

Machine learning/Apprentissage statistique

Université de Pau et des Pays de l'Adour

4 février 2026

Ce résumé de cours est essentiellement fondé sur les références suivantes :

- Trevor Hastie, Robert Tibshirani et Jerome H. Friedman, The Elements of Statistical Learning, Springer, 2009.
- Gareth James, Daniela Witten, Trevor Hastie et Robert Tibshirani, An Introduction to Statistical Learning, Springer, 2023.
- Laurent Rouvrière, Cours d'apprentissage supervisé – machine learning, Université Rennes 2, 2024.
- Emil Hvitfeldt, ISLR tidymodels labs,
<https://emilhvitfeldt.github.io/ISLR-tidymodels-labs/>, 2023.

La plupart des figures utilisées dans ce support ont été mises à disposition par les auteurs de la seconde référence sur le site de leur livre. Merci à eux !

Table des matières

1	Bases du machine learning/de l'apprentissage statistique	5
1.1	Introduction	5
1.1.1	Apprentissage supervisé	5
1.1.2	Objectifs	6
1.1.3	Philosophie	6
1.1.4	Apprentissage non supervisé	7
1.1.5	Caractère fondamental du contexte statistique	7
1.2	Apprentissage statistique supervisé : le cas de la régression	8
1.2.1	La fonction de régression	9
1.2.2	Comment estimer la fonction de régression ?	13
1.2.3	Modèles paramétriques et autres modèles structurés	19
1.2.4	Évaluation de la qualité de l'ajustement d'un modèle	23
1.2.5	Compromis/dilemme biais-variance	29
1.3	Apprentissage statistique supervisé : le cas de la "classification"	30
1.3.1	Objectifs	30
1.3.2	Le classifieur optimal de Bayes	31
1.3.3	La méthode des k plus proches voisins	32
1.3.4	Évaluation de la qualité de l'ajustement d'un classifieur	36
1.3.5	Modèles structurés	39
1.4	Apprentissage statistique supervisé : cadre général	40
1.4.1	Cadre théorique	40
1.4.2	Cadre statistique	41
1.5	Validation et validation croisée	42
1.5.1	Ensemble de validation/test	43

1.5.2	Validation croisée avec K blocs	45
1.5.3	Problèmes liés à la validation croisée	48
1.5.4	La règle d’une erreur standard	48
1.5.5	Validation croisée répétée	49
1.6	Les packages R <code>caret</code> et <code>tidymodels</code>	49
2	Analyse discriminante	53
2.1	Le “théorème de Bayes” pour la classification	53
2.2	L’analyse discriminante gaussienne dans le cas $p = 1$	55
2.3	L’analyse discriminante gaussienne dans le cas $p > 1$	57
2.4	Autres formes d’analyses discriminantes	59
2.5	Données déséquilibrées et solutions dans le cas de la classification binaire . .	62
2.6	Le classifieur bayésien naïf gaussien	66
2.7	Analyse discriminante avec <code>tidymodels</code>	67
2.7.1	Sans découpage apprentissage/test des données disponibles	68
2.7.2	Avec découpage apprentissage/test des données disponibles	69
2.7.3	Estimation de l’erreur de généralisation par validation croisée répétée	70
3	Extensions du modèle linéaire	72
3.1	Rappels sur la régression linéaire multiple	72
3.1.1	Les moindres carrés	72
3.1.2	Mesures de qualité de l’ajustement	75
3.1.3	Pratique de la régression linéaire multiple	76
3.2	Régularisation : régression linéaire ridge et Lasso	80
3.2.1	Régression linéaire ridge	80
3.2.2	Régression linéaire Lasso	83
3.3	Exemples R avec le package <code>glmnet</code>	87
3.3.1	Données simulées	87
3.3.2	Données <code>Credit</code>	89
3.4	Exemple R avec le package <code>tidymodels</code>	90
4	Arbres de décision, bagging, forêts aléatoires et boosting	93
4.1	Bases des arbres de décision dans le cas de la régression	94

4.2	Arbres de régression avec le package <code>rpart</code>	103
4.3	Arbres de décision pour la classification	105
4.4	Arbres de classification avec le package <code>rpart</code>	109
4.5	Avantages et inconvénients des arbres de décision	110
4.6	Bagging	111
4.7	Forêts aléatoires	113
4.8	Boosting	115
4.9	Bagging, forêts aléatoires et boosting avec <code>tidymodels</code>	120
4.10	Résumé	121

Chapitre 1

Bases du machine learning/de l'apprenstissage statistique

1.1 Introduction

1.1.1 Apprentissage supervisé

- Variable à expliquer (réponse, cible, de sortie, à prédire, ...) : Y
- Vecteur de p variables explicatives (variables d'entrée, prédicteurs, attributs, covariables, ...) : $X = (X_1, \dots, X_p)$.
- Il faut garder à l'esprit que les variables explicatives ne sont en réalité que *potentiellement* explicatives.
- Dans le cas de problèmes de régression, Y est quantitative (prix, tension artérielle, concentration de CO_2 , taux de cholestérol, ...).
- Dans le cas de problèmes de “classification” (voir ci-dessous), Y prend ses valeurs dans un ensemble fini non ordonné ($\{\text{solvable, non solvable}\}$, les chiffres de 0 à 9, type de cancer, ...). Il ne faut pas se laisser perturber par le fait que Y est souvent codée numériquement dans ces situations.
- Nous avons des données $(x_1, y_1), \dots, (x_n, y_n)$ qui sont des vecteurs de $\mathbb{R}^{p+1}$. Ce sont des observations (exemples, instances, réalisations) du *vecteur aléatoire* (X, Y) à $p + 1$ composantes.

Vocabulaire : Attention aux différences/analogies français/anglais suivantes :

Français	Anglais
Classification automatique Discrimination Régression	Clustering Classification / supervised classification Regression

Même si, dans la suite de ce cours, nous emploierons les termes anglo-saxons (car utilisés dans les logiciels, la documentation, etc), il est important de connaître leurs traductions en français.

1.1.2 Objectifs

En nous aidant des données disponibles, nous souhaitons :

- Prédire la valeur de Y pour une nouvelle valeur x de X .
- Comprendre quelles variables explicatives affectent Y , et comment.
- Évaluer la qualité de nos prédictions et autres inférences.

1.1.3 Philosophie

- Il est important de comprendre les idées derrière les techniques existantes afin de savoir quand et comment les utiliser.
- Il faut comprendre les méthodes simples d'abord afin de pouvoir appréhender celles qui sont plus complexes ensuite.
- Il est important d'être capable d'évaluer précisément la performance d'une méthode afin de savoir à quel point elle marche bien (ou mal). Souvent, les méthodes les plus simples marchent aussi bien que celles qui sont plus complexes.
- L'apprentissage statistique/machine learning est un domaine de recherche très stimulant ayant des applications importantes en science, dans l'industrie, en finance, ...
- C'est une compétence fondamentale du scientifique des données.
- Remarques :
 - les réseaux de neurones/l'apprentissage profond font partie de l'apprentissage statistique/machine learning ;
 - les *large language models* (LLM) utilisent des techniques de TALN en pré-traitement de méthodes neuronales.

1.1.4 Apprentissage non supervisé

- Il n’y a pas de variable à expliquer/cible, juste un ensemble de variables explicatives mesurées sur des individus statistiques.
- L’objectif est plus flou : trouver des groupes d’individus similaires, trouver des variables qui se comportent “de façon similaire”, trouver les combinaisons linéaires de variables qui varient le plus, ...
- Il est difficile d’évaluer la performance des méthodes.
- Bien que différent de l’apprentissage supervisé, l’apprentissage non supervisé peut être utile en tant que pré-traitement à de l’apprentissage supervisé.

1.1.5 Caractère fondamental du contexte statistique

- L’apprentissage statistique/machine learning se trouve à l’intersection des mathématiques/de la statistique et de l’informatique.
- Il est néanmoins fondamental de poser des hypothèses minimales sur (X, Y) et les données disponibles $(x_1, y_1), \dots, (x_n, y_n)$. Nous considérerons dans la suite que :
 - Y prend ses valeurs dans un sous-ensemble de $\mathbb{R}$ (et non pas $\mathbb{R}^q$, $q > 1$ – possible mais plus rare) ;
 - (X, Y) est un vecteur aléatoire de dimension $p + 1$; sa loi est inconnue (mais existe) ; il est également souvent nécessaire de supposer que sa matrice de covariance (inconnue) existe ;
 - les données disponibles $(x_1, y_1), \dots, (x_n, y_n)$ sont donc des vecteurs de $\mathbb{R}^{p+1}$ et seront vues comme la réalisation de n copies indépendantes $(X_1, Y_1), \dots, (X_n, Y_n)$ de (X, Y) ; on dit que $(X_1, Y_1), \dots, (X_n, Y_n)$ est un *échantillon aléatoire* ou n -échantillon de (X, Y) , ou encore $(X_1, Y_1), \dots, (X_n, Y_n)$ sont des copies *i.i.d.* (indépendantes et identiquement distribuées) de (X, Y) ;
 - dans la suite de ce cours, nous noterons $d_n = \{(x_i, y_i)\}_{i \in \{1, \dots, n\}}$ les données disponibles et $D_n = \{(X_i, Y_i)\}_{i \in \{1, \dots, n\}}$ leur version aléatoire.
- Sans ce genre d’hypothèses, le résultat de l’apprentissage statistique peut être complètement dénué de sens. Par exemple :
 - on cherche à expliquer le débit Y d’une rivière de montagne en été en fonction de covariables X mais (X, Y) a été mesuré sur une (trop) longue période (de plusieurs dizaines d’années) durant laquelle un glacier alimentant la rivière a disparu ;
 - on s’intéresse au chiffre d’affaire annuel d’une entreprise type “La Redoute” et les données ont été récoltées durant la période 1980-2020 ;

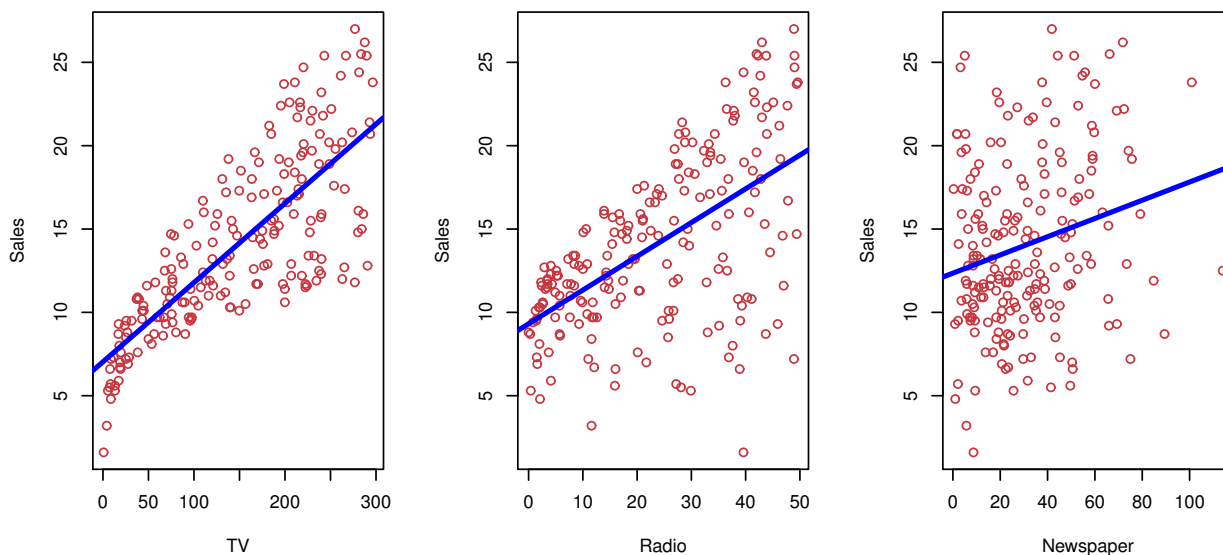
— ...

Il est fondamental que le phénomène aléatoire qui nous intéresse et sur lequel nous avons récolté des données soit stable au cours du temps.

- Les données disponibles d_n sont typiquement stockées dans une matrice n lignes $p + 1$ colonnes.

1.2 Apprentissage statistique supervisé : le cas de la régression

Ventes (Sales) en fonction du budget publicité TV, Radio et Presse écrite (Newspaper), respectivement, avec droites des moindres carrés (régressions linéaires simples) :



- Peut-on prédire Sales à l'aide de l'un de ces modèles ? Peut-être vaut-il mieux chercher à déterminer une fonction f telle que

$$Sales \approx f(TV, Radio, Newspaper) ?$$

- Ci-dessus, Sales est la variable à expliquer/à prédire/réponse, que nous noterons génériquement Y .
- TV est une variable (potentiellement) explicative (ou variable d'entrée, prédicteur, ...) que nous noterons X_1 . De même, nous utiliserons X_2 pour Radio, etc. Le vecteur des entrées/des variables explicatives est noté $X = (X_1, X_2, X_3)$.

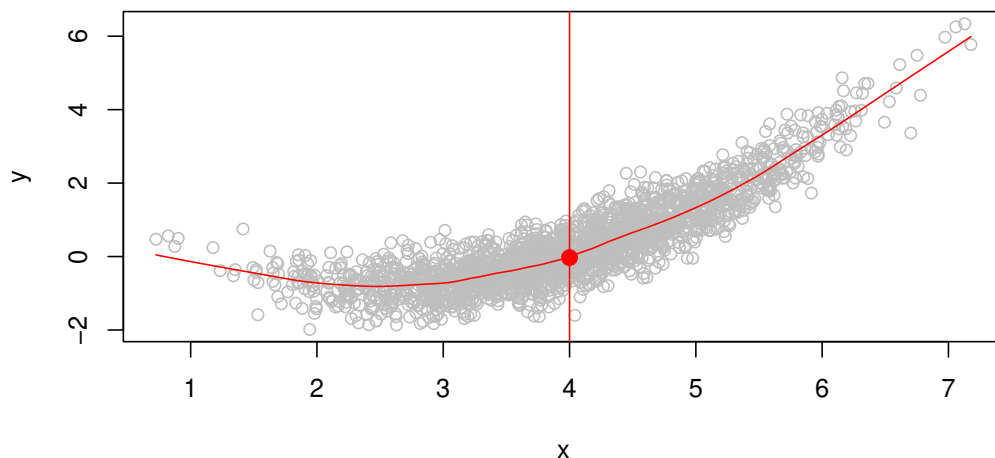
- Comme indiqué précédemment, dans la suite, nous supposons que (X, Y) est un vecteur aléatoire de dimension $p + 1$ de loi inconnue et que les observations disponibles $(x_1, y_1), \dots, (x_n, y_n)$ dans d_n sont vues comme une réalisation d'un n -échantillon de (X, Y) .
- Il peut être alors utile d'imaginer qu'en réalité il existe une fonction (en pratique inconnue) f de $\mathbb{R}^p$ dans $\mathbb{R}$ telle que,

$$Y = f(X) + \epsilon \text{ presque sûrement (p.s.),}$$

où ϵ est une variable aléatoire (pas forcément indépendante de X) représentant le caractère non déterministe de la relation entre Y et X . Cette erreur n'est pas forcément faible. Exemples ...

- Si nous connaissions la fonction f , nous pourrions faire des prédictions de Y lorsque $X = x$. Plus précisément, nous pourrions :
 - lorsque $X = x$, prédire Y par $f(x)$; cette prédiction ne serait néanmoins pas toujours de qualité (à cause de ϵ);
 - comprendre quelles composantes de $X = (X_1, X_2, \dots, X_p)$ sont importantes pour expliquer Y , et lesquelles n'ont absolument aucune utilité (le nombre d'années d'ancienneté dans un emploi et le nombre d'années d'études peuvent expliquer le niveau de salaire d'un employé mais ce n'est en général pas le cas de son statut marital);
 - en fonction de la complexité de la fonction f , comprendre même comment chaque X_j séparément affecte Y (modèles additifs versus modèles plus complexes).

1.2.1 La fonction de régression



- Existe-t-il une fonction f idéale? En particulier, relativement au graphique ci-dessus ($p = 1$), quelle serait la fonction idéale pour prédire la valeur de Y lorsque $X = 4$?

- Dans les données disponibles d_n , nous suspectons qu'il existe de nombreuses valeurs de Y (différentes les unes des autres) qui correspondent à $X \approx 4$.
- Un choix possible et plutôt naturel serait de considérer la moyenne de ces valeurs.
- Supposons $p \geq 1$ (cas général). Mathématiquement, il s'agit de trouver une fonction f de $\mathbb{R}^p$ dans $\mathbb{R}$ telle que $f(X)$ soit proche de Y en espérance au sens d'une fonction de perte ℓ choisie. Cette fonction de perte ℓ a pour objectif de pénaliser les erreurs de prédiction.
- Dans le cas de problèmes de régression (Y quantitative), la fonction de perte la plus usuelle est la perte quadratique :

$$\ell_2(y, y') = (y - y')^2.$$

[Un autre choix usuel consiste à utiliser la perte en valeur absolue : $\ell_1(y, y') = |y - y'|$.]

- L'objectif (théorique) est alors de trouver la fonction f qui minimise l'erreur de prédiction espérée :

$$\mathcal{E}_{\ell_2}(f) = \mathbb{E}[\ell_2(Y, f(X))] = \mathbb{E}[(Y - f(X))^2]. \quad (1.1)$$

Exercice 1.

1. Soit g une fonction de $\mathbb{R}^p$ dans $\mathbb{R}$ telle que $\mathcal{E}_{\ell_2}(g) < \infty$. À l'aide du théorème de l'espérance totale, exprimer l'erreur de prédiction espérée $\mathcal{E}_{\ell_2}(g)$ en fonction de la variable aléatoire $\mathbb{E}[(Y - g(X))^2 | X]$.
2. Énoncer une condition simple sous laquelle la variable aléatoire $\mathbb{E}[Y | X]$ existe.
3. Énoncer une condition simple sous laquelle la variable aléatoire $\mathbb{E}[(Y - \mathbb{E}[Y | X])^2 | X]$ existe.
4. Montrer que, pour toute fonction g de $\mathbb{R}^p$ dans $\mathbb{R}$ telle que $\mathcal{E}_{\ell_2}(g) < \infty$,

$$\mathbb{E}[(Y - \mathbb{E}[Y | X])^2 | X] \leq \mathbb{E}[(Y - g(X))^2 | X] \text{ p.s.} \quad (1.2)$$

5. En déduire que, parmi toutes les fonctions g possibles, choisir $g = f_{\ell_2}^*$ avec $f_{\ell_2}^*(x) = \mathbb{E}[Y | X = x]$ minimise $\mathcal{E}_{\ell_2}(g)$, et que toute fonction g qui minimise $\mathcal{E}_{\ell_2}(g)$ satisfait $g(X) = f_{\ell_2}^*(X)$ p.s.

□

Éléments de solution:

1. $\mathcal{E}_{\ell_2}(g) = \mathbb{E}[\mathbb{E}[(Y - g(X))^2 | X]]$.
2. $\mathbb{E}[|Y|]$ existe (c-à-d, $\mathbb{E}[|Y|] < \infty$).

3. $\mathbb{E}[Y^2]$ existe. L'inégalité de Jensen implique alors que $\mathbb{E}[|Y|]$ existe.

4. Avec la notation $f(X) = \mathbb{E}[Y | X]$, nous avons, p.s.,

$$\begin{aligned}\mathbb{E}[(Y - g(X))^2 | X] &= \mathbb{E}[(Y - f(X) + f(X) - g(X))^2 | X] \\ &= \mathbb{E}[(Y - f(X))^2 | X] + \mathbb{E}[(f(X) - g(X))^2 | X] \\ &\quad + 2\mathbb{E}[(Y - f(X))(f(X) - g(X)) | X].\end{aligned}$$

Or, p.s.,

$$\begin{aligned}\mathbb{E}[(Y - f(X))(f(X) - g(X)) | X] &= (f(X) - g(X))\mathbb{E}[Y - f(X) | X] \\ &= (f(X) - g(X))(\mathbb{E}[Y | X] - f(X)) = 0,\end{aligned}$$

ce qui implique que, p.s.,

$$\mathbb{E}[(Y - g(X))^2 | X] = \mathbb{E}[(Y - f(X))^2 | X] + (f(X) - g(X))^2. \quad (1.3)$$

Le résultat est alors simplement une conséquence du fait que $(f(X) - g(X))^2 \geq 0$ p.s.

5. Le premier résultat s'obtient par exemple en prenant l'espérance à gauche et à droite de (1.2) :

$$\mathbb{E}[(Y - \mathbb{E}[Y | X])^2] = \mathbb{E}[\mathbb{E}[(Y - \mathbb{E}[Y | X])^2 | X]] \leq \mathbb{E}[\mathbb{E}[(Y - g(X))^2 | X]] = \mathbb{E}[(Y - g(X))^2].$$

Pour le second résultat, en prenant l'espérance à gauche et à droite de (1.3), on obtient

$$\mathbb{E}[(Y - g(X))^2] = \mathbb{E}[(Y - f(X))^2] + \mathbb{E}[(f(X) - g(X))^2].$$

Si g est telle que $\mathcal{E}_{\ell_2}(g)$ est minimale, alors $\mathcal{E}_{\ell_2}(g) = \mathcal{E}_{\ell_2}(f)$. L'égalité précédente implique alors que $\mathbb{E}[(f(X) - g(X))^2] = 0$. Or, comme $(f(X) - g(X))^2$ est une variable aléatoire positive, $\mathbb{E}[(f(X) - g(X))^2] = 0$ implique que $(f(X) - g(X))^2 = 0$ p.s., c-à-d, $g(X) = f(X)$ p.s.

□

En d'autres termes, parmi toutes les fonctions f possibles, choisir pour f la fonction $f_{\ell_2}^*$ définie par $f_{\ell_2}^*(x) = \mathbb{E}[Y | X = x]$ minimise l'erreur (quadratique) de prédiction espérée $\mathcal{E}_{\ell_2}$ donnée par (1.1). Cette fonction est appelée *fonction de régression*.

Remarques :

- La fonction de régression $f_{\ell_2}^*$ est (théoriquement) bien définie si $\mathbb{E}[|Y|] < \infty$.
- Revenons au graphique précédent. Une bonne prédiction de Y lorsque $X = 4$ est donc $f_{\ell_2}^*(4) = \mathbb{E}[Y | X = 4]$. Elle s'interprète comme la moyenne des valeurs que peut prendre Y lorsque $X = 4$.

— Dans le cas de trois variables explicatives par exemple, nous avons

$$f_{\ell^2}^*(x) = f_{\ell^2}^*(x_1, x_2, x_3) = \mathbb{E}[Y \mid X_1 = x_1, X_2 = x_2, X_3 = x_3].$$

— Sachant que $X = x$, la variable aléatoire $Y - f_{\ell^2}^*(x)$ est associée à ce qui est appelé l'*erreur irréductible* : même si nous connaissions la fonction de régression $f_{\ell^2}^*$, nous ferions tout de même des erreurs de prédiction car, pour $X = x$, il y a “une distribution de valeurs de Y possibles”.

Exercice 2.

1. Soit Z une variable aléatoire telle que $\mathbb{E}[Z] < \infty$ et soit $Z_1, \dots, Z_n$ un échantillon aléatoire (ou n -échantillon) de Z . Dans le cadre de l'estimation de $\mathbb{E}[Z]$, rappeler brièvement les notions de paramètre, d'estimateur et d'estimation. Déterminer l'espérance, le biais, la variance et l'erreur quadratique moyenne de l'estimateur usuel de $\mathbb{E}[Z]$.
2. Dans la suite, nous nous intéressons à un estimateur de $f_{\ell^2}^*$ basé sur D_n , noté $\hat{f}_{D_n}$, et à une réalisation de l'estimateur précédent lorsque $D_n = d_n$, noté $\hat{f}_{d_n}$. Quel est la nature mathématique de $\hat{f}_{d_n}$? De $\hat{f}_{D_n}$? Montrer que

$$\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X = x, D_n = d_n] = (f_{\ell^2}^*(x) - \hat{f}_{d_n}(x))^2 + \mathbb{V}[Y \mid X = x]. \quad (1.4)$$

3. Lorsque l'on cherche à prédire Y sachant que $X = x$, le premier (resp. second) terme à droite de (1.4) est classiquement appelé l'*erreur réductible* (resp. l'*erreur irréductible*). Expliquer pourquoi.
4. Enfin, en déduire, que, si $Y = f_{\ell^2}^*(X) + \epsilon$ p.s., où ϵ est une variable aléatoire indépendante de X , l'erreur irréductible est simplement la variance de ϵ .

□

Éléments de solution:

1. ...
2. Avec la notation $f = f_{\ell^2}^*$, nous avons

$$\begin{aligned} \mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X = x, D_n = d_n] &= \mathbb{E}[(Y - f(X))^2 \mid X = x, D_n = d_n] \\ &\quad + \mathbb{E}[(f(X) - \hat{f}_{D_n}(X))^2 \mid X = x, D_n = d_n] \\ &\quad + 2\mathbb{E}[(Y - f(X))(f(X) - \hat{f}_{D_n}(X))^2 \mid X = x, D_n = d_n] \\ &= \mathbb{E}[(Y - f(X))^2 \mid X = x] + (f(x) - \hat{f}_{d_n}(x))^2 \end{aligned}$$

car

$$\begin{aligned} \mathbb{E}[(Y - f(X))(f(X) - \hat{f}_{D_n}(X)) \mid X = x, D_n = d_n] &= (f(x) - \hat{f}_{d_n}(x))\mathbb{E}[Y - f(X) \mid X = x] \\ &= (f(x) - \hat{f}_{d_n}(x))(f(x) - f(x)) = 0. \end{aligned}$$

Le résultat est finalement une conséquence du fait que

$$\begin{aligned}\mathbb{E}[(Y - f(X))^2 \mid X = x] &= \mathbb{E}[Y^2 - 2Yf(X) + f(X)^2 \mid X = x] \\ &= \mathbb{E}[Y^2 \mid X = x] - 2f(x)^2 + f(x)^2 \\ &= \mathbb{E}[Y^2 \mid X = x] - (\mathbb{E}[Y \mid X = x])^2,\end{aligned}$$

ou, de façon équivalente, du fait que $\mathbb{E}[Y - f(X) \mid X = x] = 0$.

3. ...

4. Le résultat est une conséquence du fait que $\mathbb{V}[Y \mid X = x]$ est alors égale à

$$\mathbb{V}[f(X) + \epsilon \mid X = x] = \mathbb{V}[\epsilon].$$

□

— Pour un n fixé et lorsque $X = x$:

— le biais de l'estimateur $\hat{f}_{D_n}(x)$ de $f_{\ell_2}^*(x)$ est :

$$\text{Biais}[\hat{f}_{D_n}(x)] = \mathbb{E}[\hat{f}_{D_n}(x)] - f_{\ell_2}^*(x),$$

— la variance de l'estimateur $\hat{f}_{D_n}(x)$ de $f_{\ell_2}^*(x)$ est :

$$\mathbb{V}[\hat{f}_{D_n}(x)] = \mathbb{E}[(\hat{f}_{D_n}(x))^2] - (\mathbb{E}[\hat{f}_{D_n}(x)])^2 = \mathbb{E}[(\hat{f}_{D_n}(x) - \mathbb{E}[\hat{f}_{D_n}(x)])^2],$$

— l'erreur quadratique moyenne de l'estimateur $\hat{f}_{D_n}(x)$ de $f_{\ell_2}^*(x)$ est :

$$\text{EQM}[\hat{f}_{D_n}(x)] = \mathbb{E}[(\hat{f}_{D_n}(x) - f_{\ell_2}^*(x))^2] = (\text{Biais}[\hat{f}_{D_n}(x)])^2 + \mathbb{V}[\hat{f}_{D_n}(x)].$$

1.2.2 Comment estimer la fonction de régression ?

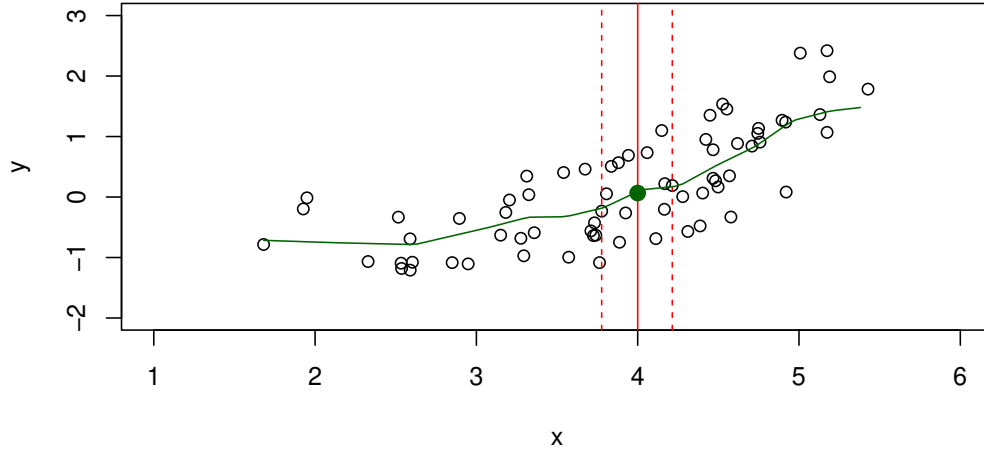
— Revenons au graphique précédent et supposons que $x = 4$. Typiquement, nous avons peu ou pas d'observations telles que $x_i = x$ exactement.

— Nous ne pouvons donc pas estimer “simplement” $\mathbb{E}[Y \mid X = x]$ par la moyenne des y_i pour lesquels $x_i = x$.

— Une première idée naturelle est de “relaxer” la définition de l'estimateur idéal et de considérer l'estimation suivante :

$$\hat{f}_{d_n}(x) = \text{Moy}(y_i \mid x_i \in \mathcal{N}(x)), \quad (1.5)$$

où $\mathcal{N}(x)$ est un voisinage adéquat de x .



Exercice 3.

Donner une définition plus explicite de l'estimation $\hat{f}_{d_n}$ de $f_{\ell^2}^*$ donnée dans (1.5). □

Éléments de solution: On a simplement

$$\hat{f}_{d_n}(x) = \frac{1}{|\{x_i \in \mathcal{N}(x)\}|} \sum_{i: x_i \in \mathcal{N}(x)} y_i.$$

□

- L'estimation de la fonction de régression donnée dans (1.5) s'apparente à l'estimation par la méthode des *k plus proches voisins*.
- Soit $k \in \{1, \dots, n\}$ fixé, soit d une distance sur $\mathbb{R}^p$ choisie et, pour tout x et $i \in \{1, \dots, n\}$, soit $d_i^x = d(x, x_i)$. Notons $d_{(1)}^x, \dots, d_{(n)}^x$ un ré-arrangement de $d_1^x, \dots, d_n^x$ tel que $d_{(1)}^x \leq \dots \leq d_{(n)}^x$, et soit

$$\mathcal{N}_k(x) = \{i : d_i^x \leq d_{(k)}^x\}.$$

- L'estimation de la fonction de régression par la méthode des *k plus proches voisins* est alors donnée par

$$\hat{f}_{d_n, k}(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i. \quad (1.6)$$

- Elle donne en général de bons résultats lorsque p est petit (disons $p \leq 4$) et n est plutôt grand.

Exercice 4.

Supposer que $p = 1$, que X est uniformément distribuée sur $] -2, 3[$ et que $Y = X^2 + \epsilon$ p.s., avec ϵ indépendante de X et normale centrée réduite.

1. Générer d_n pour $n = 50$ (c-à-d, générer $n = 50$ réalisations de (X, Y)) à l'aide de R et représenter le nuage de points correspondant ainsi que la fonction de régression vraie sur ce nuage.
2. À l'aide du package FNN, estimer $\hat{f}_{d_n, k}$ pour $k \in \{1, 5, 10\}$ et représenter $\hat{f}_{d_n, k}$ pour $k \in \{1, 5, 10\}$ sur le nuage de points précédent.
3. Ré-exécuter le code précédent plusieurs fois (sans fixer la graine aléatoire) puis adapter ce code afin d'illustrer les phrases suivantes que l'on retrouve dans la littérature :
 - (a) "la variance de l'estimateur des k plus proches voisins augmente lorsque k diminue";
 - (b) "le biais de l'estimateur des k plus proches voisins augmente lorsque k augmente".Donner une formulation plus mathématique des phrases précédentes.
4. Écrire une fonction R implémentant directement l'estimation par la méthode des plus proches voisins. Vérifier cette implémentation en la comparant avec la fonction dédiée du package FNN. Enfin, utiliser la fonction `system.time()` pour comparer les vitesses d'exécution des deux implémentations.

□

Éléments de solution:

Questions 1 et 2

```
library(FNN)
curve(x^2, from = -2, to = 3)
x.grid <- seq(from = -2, to = 3, by = 0.1)
n <- 50

x <- runif(n, min = -2, max = 3)
y <- x^2 + rnorm(n)
app.x <- data.frame(x = x)
app.y <- data.frame(y = y)
points(x, y)

## Lire l'aide de knn.reg
m1 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 1)
m5 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 5)
m10 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 10)
lines(x.grid, m1$pred, col = "blue")
```



```

lines(x.grid, m5$pred, col = "orange")
lines(x.grid, m10$pred, col = "brown")

## Question 3 (a)

M <- 100

par(mfrow=c(1,3))

curve(x^2, from = -2, to = 3)
for (i in seq_len(M)) {
  x <- runif(n, min = -2, max = 3)
  y <- x^2 + rnorm(n)
  app.x <- data.frame(x = x)
  app.y <- data.frame(y = y)
  m1 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 1)
  lines(x.grid, m1$pred, col = "blue", lwd = 0.5)
}

curve(x^2, from = -2, to = 3)
for (i in seq_len(M)) {
  x <- runif(n, min = -2, max = 3)
  y <- x^2 + rnorm(n)
  app.x <- data.frame(x = x)
  app.y <- data.frame(y = y)
  m5 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 5)
  lines(x.grid, m5$pred, col = "orange", lwd = 0.5)
}

curve(x^2, from = -2, to = 3)
for (i in seq_len(M)) {
  x <- runif(n, min = -2, max = 3)
  y <- x^2 + rnorm(n)
  app.x <- data.frame(x = x)
  app.y <- data.frame(y = y)
  m10 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 10)
  lines(x.grid, m10$pred, col = "brown", lwd = 0.5)
}

## Question 3 (b)

M <- 1000

par(mfrow=c(1,1))
res1 <- res5 <- res10 <- matrix(NA, nrow = M, ncol = length(x.grid))

```

```

curve(x^2, from = -2, to = 3)

for (i in seq_len(M)) {
  x <- runif(n, min = -2, max = 3)
  y <- x^2 + rnorm(n)
  app.x <- data.frame(x = x)
  app.y <- data.frame(y = y)
  m1 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 1)
  res1[i,] <- m1$pred
  m5 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 5)
  res5[i,] <- m5$pred
  m10 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 10)
  res10[i,] <- m10$pred
}
lines(x.grid, colMeans(res1), col = "blue", lwd = 0.5)
lines(x.grid, colMeans(res5), col = "orange", lwd = 0.5)
lines(x.grid, colMeans(res10), col = "brown", lwd = 0.5)

## Question 4

## Version avec sapply (ne marche que dans le cas p = 1)
knn <- function(x, d, k = 1) {
  sapply(x, function(x.) {
    tmp <- sort(abs(d$x - x.), index.return = TRUE)
    mean(d$y[tmp$ix[1:k]])
  })
}

## Tests de notre implémentation
d <- data.frame(x = x, y = y)
all(knn(x.grid, d = d, k = 1) == m1$pred)
all(knn(x.grid, d = d, k = 5) == m5$pred)
all(knn(x.grid, d = d, k = 10) == m10$pred)

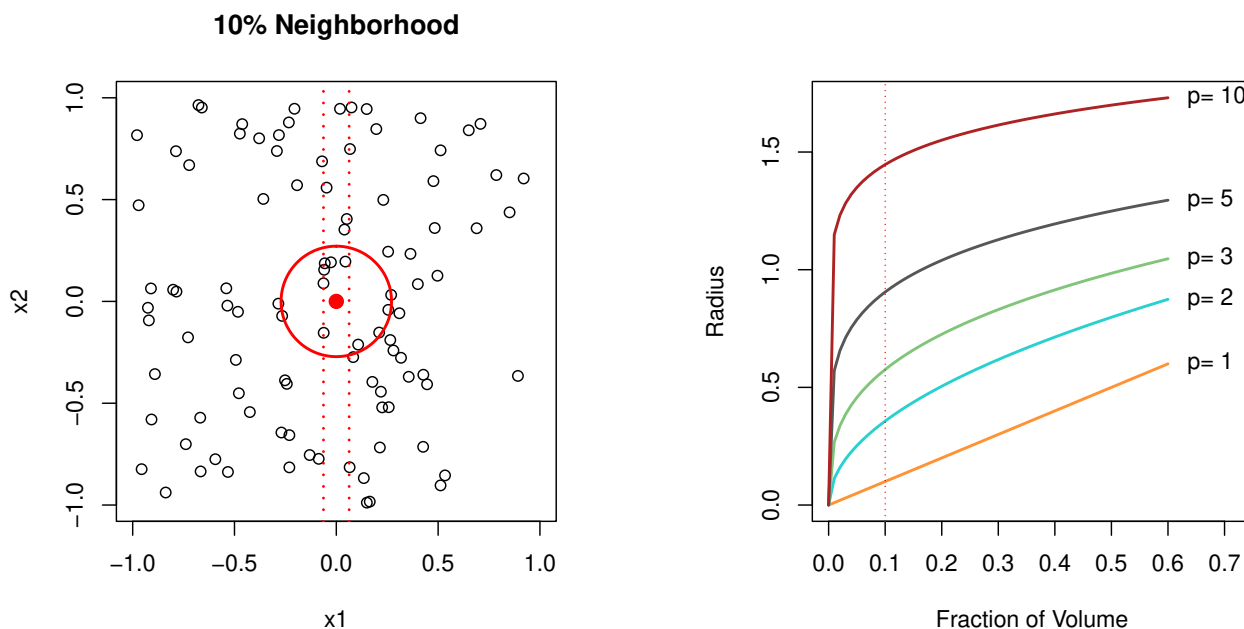
## Comparaisons peu informatives
system.time(knn(x.grid, d = d, k = 5))
system.time(m5 <- knn.reg(train = app.x, test = data.frame(x = x.grid), y = app.y, k = 5))
## Ce serait mieux d'utiliser le package "bench" par exemple

```

□

- L'estimation par la méthode des plus proches voisins peut en revanche être très mauvaise lorsque p est grand. Cela est dû à la *malédiction de la dimension* : les plus proches voisins ont tendance à être loin lorsque p est grand.
- Plus spécifiquement :

- nous avons typiquement besoin d’une proportion non négligeable (disons 10%) des n valeurs de Y disponibles à moyenner pour avoir un estimateur avec une variance raisonnable (c-à-d, un estimateur dont les réalisations ne vont pas “excessivement” changer avec d_n);
- un voisinage contenant 10% des observations en grande dimension n’est plus nécessairement local : on perd ainsi l’idée d’estimer $\mathbb{E}[Y \mid X = x]$ par des moyennes “locales”.



Exercice 5.

Supposer que X est un vecteur aléatoire uniformément distribué sur l’hypercube $] -1, 1[^p$. Donner alors du code R permettant de générer des réalisations de X . Soit n le nombre de réalisations de X et soit $x = (0, \dots, 0) \in \mathbb{R}^p$. Expliquer pourquoi, lorsque n est grand, trouver un voisinage de x (sous la forme d’un hypercube “centré” en x) contenant 10% des réalisations est approximativement équivalent à trouver un voisinage de x (sous la forme d’un hypercube “centré” en x) représentant 10% du volume de $] -1, 1[^p$. Produire enfin un graphique similaire au graphique de droite de la figure ci-dessus. $\square$

Éléments de solution: Soit B un sous-ensemble de $] -1, 1[^p$ et soient $x_1, \dots, x_n$ des réalisations de X . Pour “ n grand”, nous avons alors

$$\mathbb{P}[X \in B] \approx \frac{1}{n} \sum_{i=1}^n \mathbf{1}[x_i \in B]. \quad (1.7)$$

Cela est une conséquence de la loi forte des grands nombres si $x_1, \dots, x_n$ est la réalisation d’un échantillon aléatoire de X . Si B est un hypercube de côté $r/2$ centré en x , c-à-d,

$B =]-r/2, r/2[^p$, alors $\mathbb{P}[X \in B] = r^p/2^p$. A l'aide de (1.7), on obtient ainsi immédiatement que la "proportion de x_i dans B " est approximativement égale à $r^p/2^p$.

Volume total : 2^p ; volume hypercube : r^p ; proportion : q ; on a $q = r^p/2^p = (r/2)^p$, ce qui implique que $r = 2q^{1/p}$.

Code R :

```
curve(2 * x^(1/10), 0, 0.7, col = "brown",
      xlab = "Proportion du volume",
      ylab = "Longueur côté")
curve(2 * x^(1/2), add = TRUE, col = "cyan")
curve(2 * x^(1/3), add = TRUE, col = "green")
curve(2 * x^(1/5), add = TRUE, col = "gray")
curve(2 * x^(1/1), add = TRUE, col = "orange")
```

□

Remarque : Certains auteurs affirment que (lorsque p est petit par rapport à n) la méthode des plus proches voisins donne néanmoins de très bons résultats (parmi les meilleurs) pour environ un tiers des problèmes rencontrés.

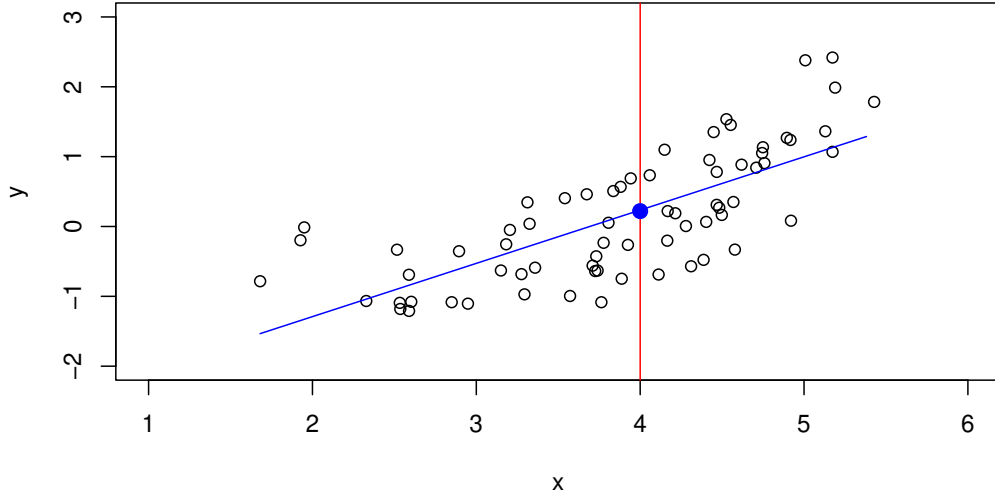
1.2.3 Modèles paramétriques et autres modèles structurés

- Le modèle linéaire est un exemple important de modèle paramétrique. Son utilisation suppose que la fonction de régression $f_{\ell^2}^*$ inconnue peut être raisonnablement bien approchée par la fonction :

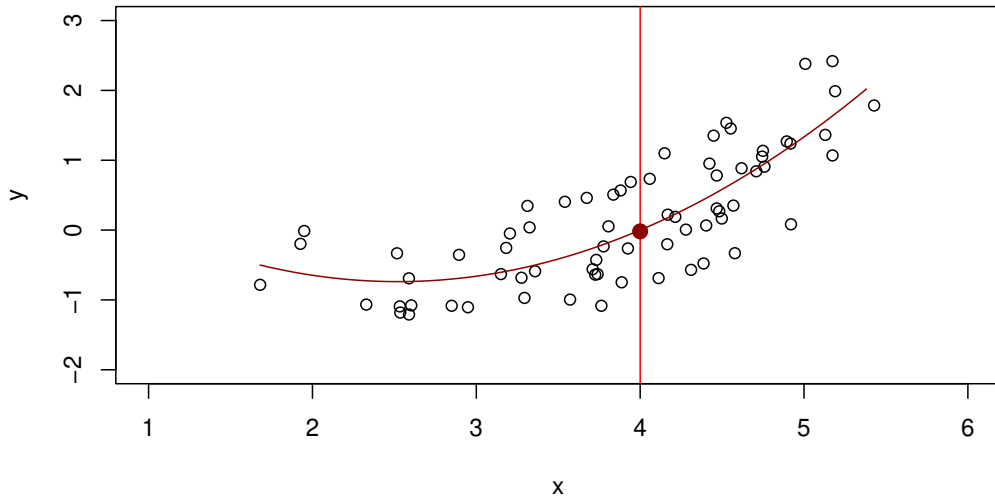
$$f_L(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p. \quad (1.8)$$

- Le modèle linéaire est spécifié par $p + 1$ paramètres $\beta_0, \beta_1, \dots, \beta_p$.
- Les paramètres sont estimés à partir des données disponibles d_n typiquement par moindres carrés.
- Bien qu'il ne soit presque jamais correct, le modèle linéaire sert souvent d'approximation raisonnable et interprétable de la fonction de régression $f_{\ell^2}^*$ inconnue.

Dans le graphique ci-dessous, le modèle linéaire estimé $\hat{f}_L(x) = \hat{\beta}_0 + \hat{\beta}_1 x$ semble conduire à une approximation raisonnable de la fonction de régression :



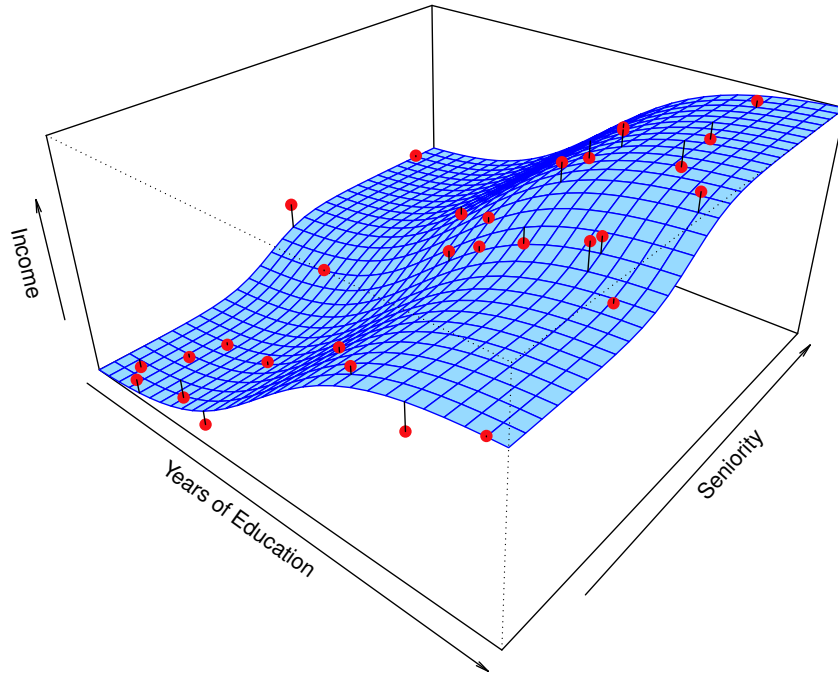
Un modèle quadratique estimé $\hat{f}_Q(x) = \hat{\beta}_0 + \hat{\beta}_1x + \hat{\beta}_2x^2$ semble conduire à une meilleure approximation de la fonction de régression :



On considère des données disponibles simulées obtenues à partir du modèle suivant :

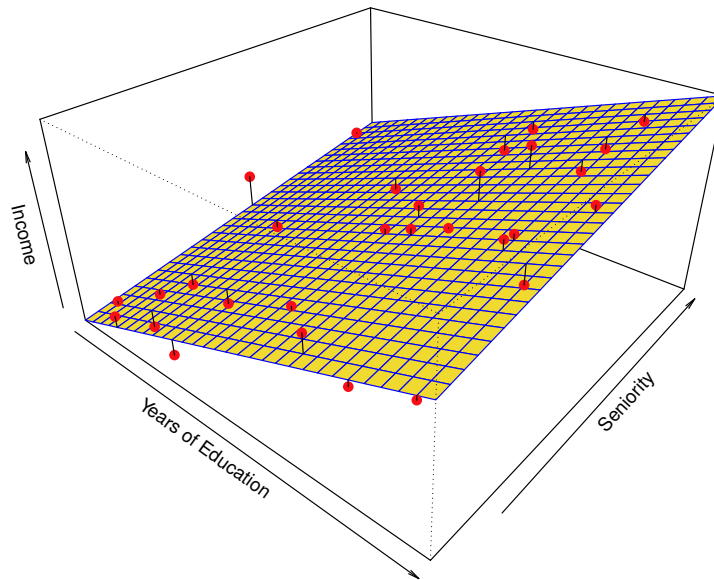
$$income = f_{\ell^2}^*(education, seniority) + \epsilon,$$

où ϵ est une variable aléatoire représentant une erreur résiduelle indépendante des variables aléatoires **education** et **seniority** telle que $\mathbb{E}[\epsilon] = 0$ et la fonction de régression $f_{\ell^2}^*$ est donnée par la surface bleue ci-dessous :

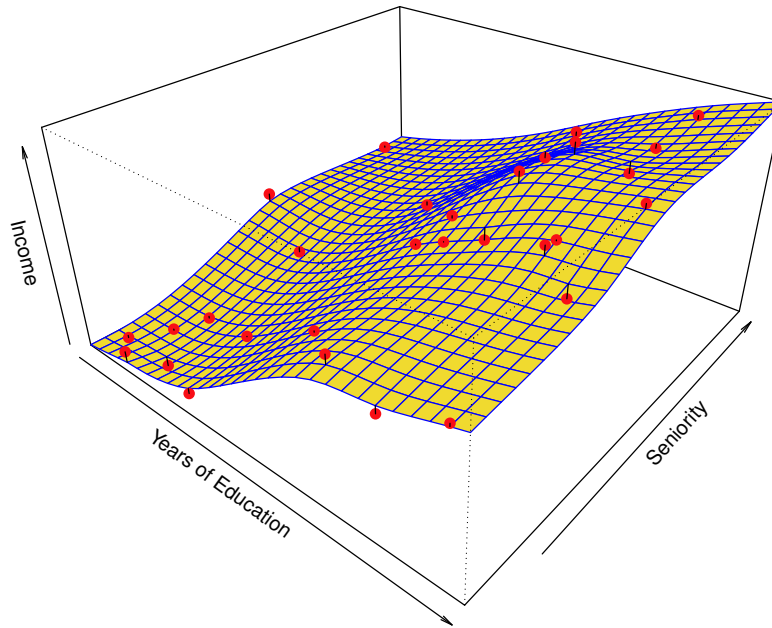


La surface jaune ci-dessous représente le modèle linéaire ajusté aux données disponibles simulées :

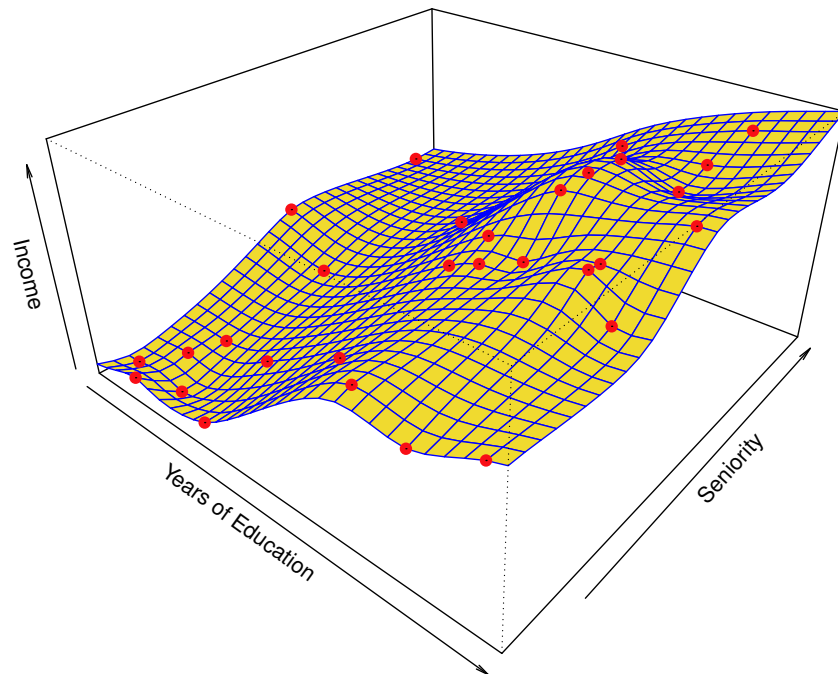
$$\hat{f}_L(education, seniority) = \hat{\beta}_0 + \hat{\beta}_1 \times education + \hat{\beta}_2 \times seniority.$$



Un modèle de régression plus flexible à base de *splines* est ajusté aux données simulées :



Enfin, une version encore plus flexible à base de *splines* est ajustée aux données simulées :

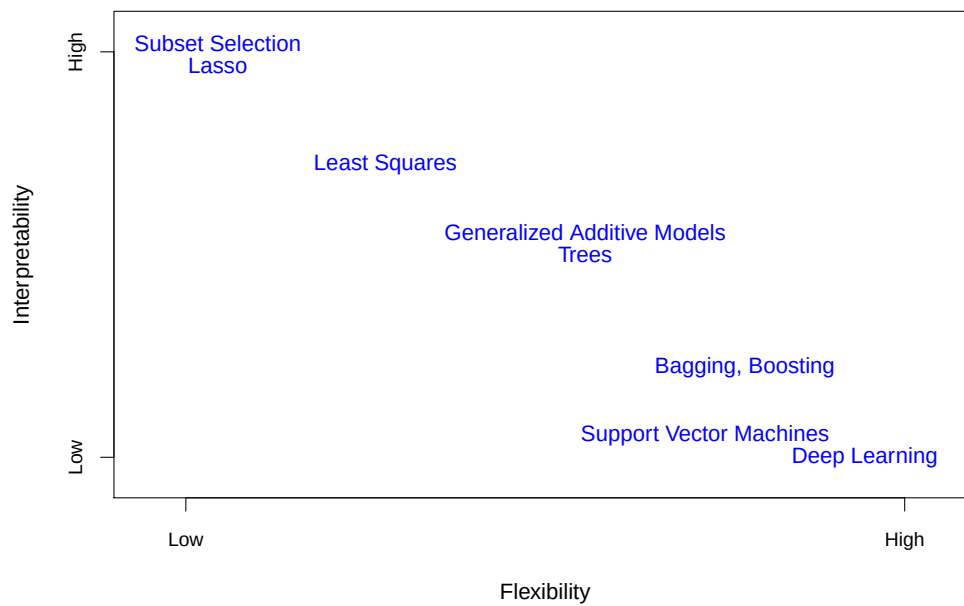


Nous voyons ci-dessus que le dernier modèle ajusté ne commet plus aucune erreur sur les données disponibles. C'est le *sur-apprentissage*.

Quelques compromis/dilemmes :

- Précision de la prédiction contre interprétabilité : les modèles linéaires sont faciles à interpréter, les modèles à bases de splines ne le sont pas.
- Bon ajustement/sur-apprentissage/sous-apprentissage : comment savoir si l'ajustement est bon ?
- Parsimonie contre boîte noire : un modèle simple incluant moins de variables explicatives est souvent préférable à une modèle “boîte noire” incluant toutes les prédicteurs disponibles. Lié à l'*IA explicable* ...

Quelques méthodes d'apprentissage statistique/machine learning évaluées en fonction de leur flexibilité/complexité (essentiellement, leur nombre de paramètres) et leur interprétabilité :



1.2.4 Évaluation de la qualité de l'ajustement d'un modèle

- Rappelons que les données disponibles sont notées d_n et sont vues comme une réalisation des données aléatoires D_n , et que nous nous intéressons à un estimateur de $f_{\ell_2}^*$ basé sur D_n , noté $\hat{f}_{D_n}$, et à une réalisation de cet estimateur lorsque $D_n = d_n$, noté $\hat{f}_{d_n}$.
- Supposons que l'on ait un modèle ajusté $\hat{f}_{d_n}$ et que l'on souhaite évaluer sa performance.
- Une mesure naturelle de sa performance prédictive est alors

$$\mathcal{E}_{\ell_2}(\hat{f}_{d_n}) = \mathbb{E}[(Y - \hat{f}_{d_n}(X))^2], \quad (1.9)$$

où $\mathcal{E}_{\ell_2}$ est définie dans (1.1) et s'interprète comme l'erreur quadratique de prédiction espérée sur une nouvelle observation (X, Y) (tirée "aléatoirement" dans la "population").

- La mesure précédente est également appelée *erreur de généralisation* de $\hat{f}_{d_n}$ dans la littérature.
- C'est une mesure théorique (même si $\hat{f}_{d_n}$ est connu) car la loi de (X, Y) est inconnue. Il faut donc essayer de l'estimer.
- Étant donnée une fonction g de $\mathbb{R}^p$ dans $\mathbb{R}$ et un ensemble e de réalisations de (X, Y) , on note

$$\begin{aligned}\text{Err}_{\ell_2}(g, e) &= \text{Moy}_{(x_i, y_i) \in e} [\ell_2(y_i, g(x_i))] = \text{Moy}_{(x_i, y_i) \in e} [(y_i - g(x_i))^2] \\ &= \frac{1}{|e|} \sum_{(x_i, y_i) \in e} (y_i - g(x_i))^2.\end{aligned}$$

Cette quantité s'interprète comme l'erreur moyenne au sens de la perte ℓ_2 de g sur l'ensemble e .

- Pour estimer l'erreur de généralisation $\mathcal{E}_{\ell_2}(\hat{f}_{d_n})$ donnée dans (1.9), nous pourrions être tentés de calculer l'erreur quadratique de prédiction moyenne sur les observations dans d_n :

$$\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n) = \text{Moy}_{(x_i, y_i) \in d_n} [(y_i - \hat{f}_{d_n}(x_i))^2]. \quad (1.10)$$

Cette quantité n'est rien d'autre que l'erreur quadratique moyenne (*Mean Squared Error* en anglais) entre le vecteur $(y_1, \dots, y_n)$ et le vecteur de prédictions $(\hat{f}_{d_n}(x_1), \dots, \hat{f}_{d_n}(x_n))$.

Exercice 6.

Expliquer intuitivement pourquoi $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$ dans (1.10) s'avère être une estimation biaisée de $\mathcal{E}_{\ell_2}(\hat{f}_{d_n})$ qui aura tendance à favoriser les modèles flexibles (et donc le sur-apprentissage).

□

- La quantité $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$ est également appelée *erreur d'apprentissage*.
- Au lieu d'utiliser $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$, nous devrions, si possible, calculer l'erreur quadratique de prédiction moyenne sur un nouvel ensemble de données t (t pour "test") non utilisé lors de l'ajustement du modèle :

$$\text{Err}_{\ell_2}(\hat{f}_{d_n}, t) = \text{Moy}_{(x_i, y_i) \in t} [(y_i - \hat{f}_{d_n}(x_i))^2]. \quad (1.11)$$

- La quantité $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ est également appelée *erreur de test*.

Exercice 7.

Expliquer intuitivement pourquoi $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ semble être une bien meilleure estimation de $\mathcal{E}_{\ell_2}(\hat{f}_{d_n}) = \mathbb{E}[(Y - \hat{f}_{d_n}(X))^2]$ que $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$. Idéalement, comment faudrait-il choisir t ? □

Exercice 8.

Dans de nombreuses situations pratiques, il est sensé d'imaginer que la relation inconnue entre X et Y est de la forme :

$$Y = g(X) + \epsilon \text{ p.s.}, \quad (1.12)$$

où ϵ est une variable aléatoire indépendante du vecteur aléatoire X représentant l'erreur de mesure et d'autres incertitudes sur la relation entre Y et X .

1. Montrer que $g(X) = f_{\ell^2}^*(X)$ p.s. est équivalent à $\mathbb{E}[\epsilon] = 0$.
2. On considère le cas usuel où $\mathbb{E}[\epsilon] = 0$. Lorsque $\mathbb{V}[\epsilon]$ est faible (resp. élevée), on dit que le rapport signal sur bruit (*signal-to-noise ratio* en anglais) est élevé (resp. faible). En finance et dans les sciences biomédicales par exemple (resp. en analyse d'images), le rapport signal sur bruit est généralement considéré comme faible (resp. élevé). Expliquer ce que cela implique en pratique.

□

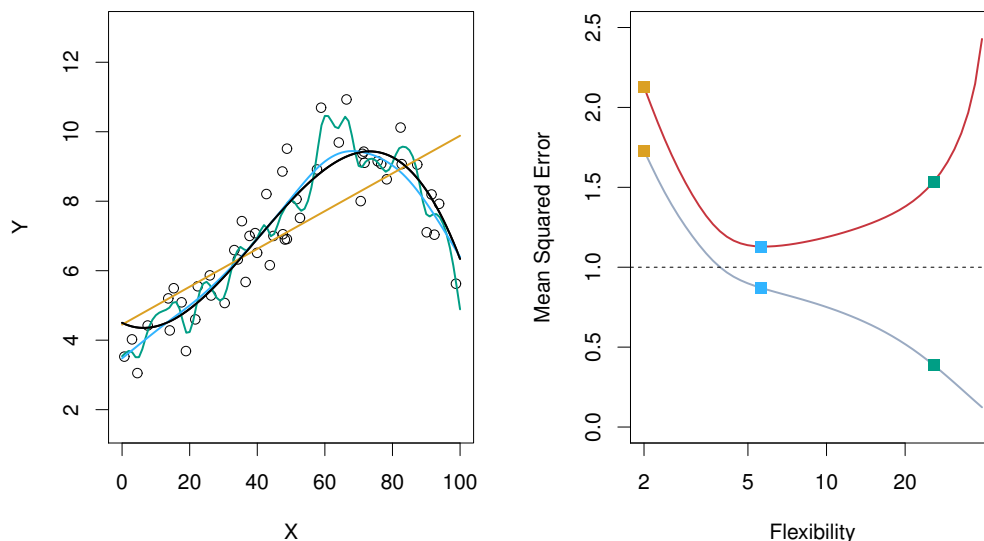
Éléments de solution: On a

$$g(X) = f_{\ell^2}^*(X) = \mathbb{E}[Y | X] \text{ p.s.} \iff \mathbb{E}[\epsilon | X] = \mathbb{E}[\epsilon] = 0 \text{ p.s.}$$

□

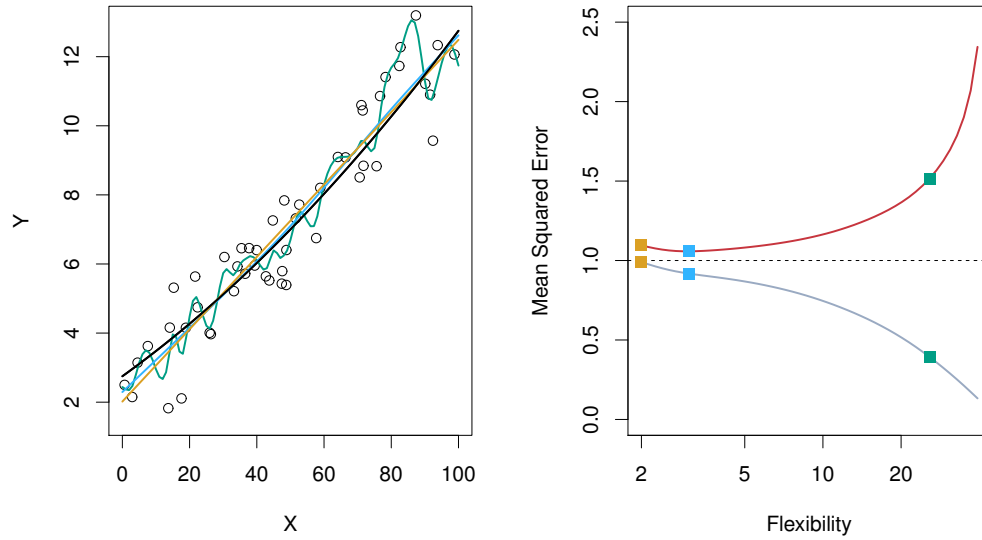
Dans le cas $p = 1$, nous allons maintenant considérer trois simulations pour lesquelles la relation entre X et Y est de la forme (1.12) avec $g = f_{\ell^2}^* / \mathbb{E}[\epsilon] = 0$.

Simulation 1 :



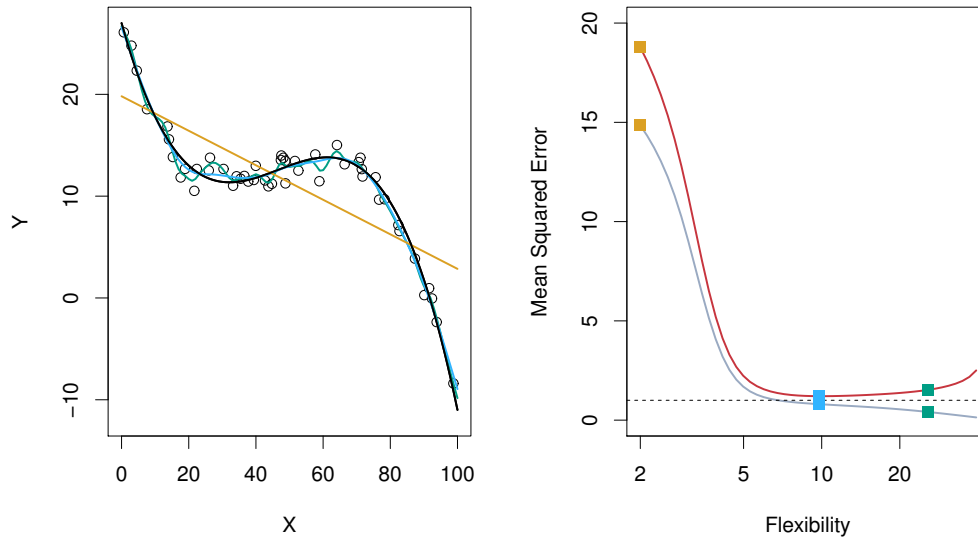
La courbe noire à gauche représente la fonction de régression vraie $f_{\ell^2}^*$. Les courbes orange, bleue et verte représentent des modèles ajustés $\hat{f}_{d_n}$ correspondant à des niveaux croissants de complexité/flexibilité du modèle (ici à base de splines). La courbe rouge (resp. grise) à droite représente $\text{Err}_{\ell^2}(\hat{f}_{d_n}, t)$ (resp. $\text{Err}_{\ell^2}(\hat{f}_{d_n}, d_n)$).

Simulation 2 :



Dans cette deuxième situation, la fonction de régression vraie $f_{\ell^2}^*$ est presque linéaire, ce qui explique pourquoi le modèle le moins complexe (proche du modèle linéaire) a l'air de très bien convenir.

Simulation 3 :



Dans cette dernière situation, la fonction de régression vraie $f_{\ell^2}^*$ est un polynôme de degré élevé, ce qui explique pourquoi des modèles plus complexes conviennent mieux.

Exercice 9.

On suppose $Y = f_{\ell^2}^*(X) + \epsilon$ p.s., avec X uniformément distribuée sur $] -3, 2[$, $f_{\ell^2}^*(x) = -0.2x^5 + x^3 - x^2 + 3x + 1$, ϵ indépendante de X et normale centrée réduite.

1. Écrire une fonction R permettant de générer des réalisations de (X, Y) .
2. Fixer la graine aléatoire à 123 et générer un “ensemble d’apprentissage” d_n de taille 40.
3. L’ajustement d’un modèle quadratique à d_n dans R peut être réalisé à l’aide du code suivant :

```
m <- lm(y ~ poly(x, degree = 2, raw = TRUE), data = app),
```

où `app` est le `data.frame` de colonnes `x` et `y` représentant d_n . En adaptant ce code, ajuster tous les modèles polynomiaux de ce type de degré 1 à 15.

4. Représenter la fonction $f_{\ell^2}^*$ sur $] -3, 2[$ ainsi que les modèles ajustés précédents de degré 1, 5 et 15.

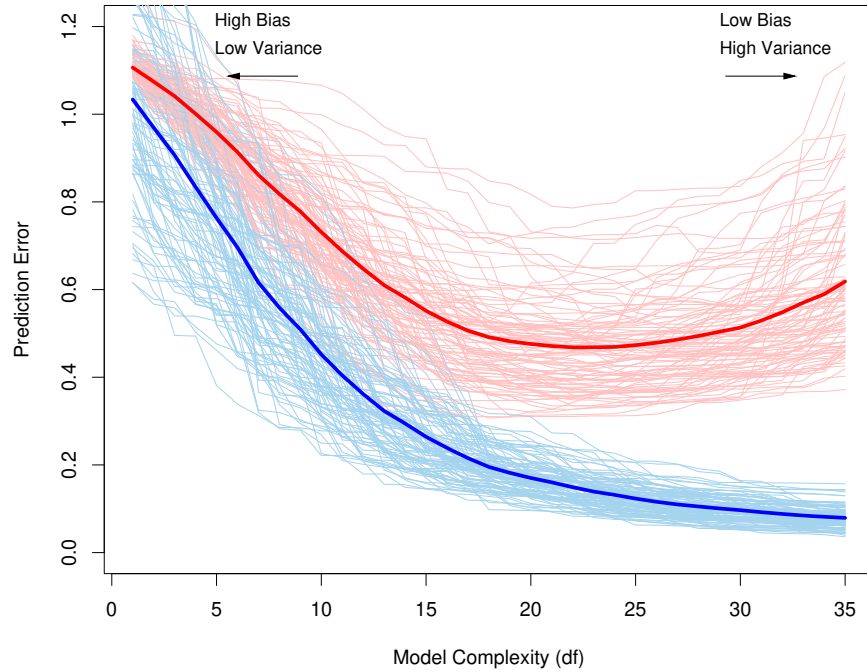
Indication : Des instructions du style `predict(m, newdata = data.frame(x = x.grid))` peuvent s’avérer utiles.

5. Générer un ensemble t de taille 1000 et représenter $\text{Err}_{\ell^2}(\hat{f}_{d_n}, d_n)$ et $\text{Err}_{\ell^2}(\hat{f}_{d_n}, t)$ en fonction de la complexité/flexibilité de la classe de modèles considérée (degré du polynôme).

6. En comparaison aux graphiques similaires pour les trois simulations ci-dessus, le dernier graphique obtenu semble “très fluctuant”. A votre avis, qu’est qui est réellement affiché dans les panneaux de droite des graphiques pour les trois simulations ?

□

Nous avons vu ci-dessus qu’un estimateur naturel de l’erreur de généralisation $\mathcal{E}_{\ell_2}(\hat{f}_{d_n})$ définie dans (1.9) est $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ défini dans (1.11), et qu’il est d’autant meilleur que l’ensemble t est grand. Pour une classe de modèles que nous verrons dans la suite du cours (l’année prochaine), les courbes bleues pâles et rouge pâles du graphique ci-après représentent $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$ et $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ en fonction de la complexité/flexibilité du modèle pour 100 réalisations d_n de l’ensemble d’apprentissage D_n (de taille 50).



Les courbes rouge et bleue foncées représentent les courbes moyennes sur les 100 échantillons d_n considérés.

Exercice 10.

En référence au graphique précédent, expliquer pourquoi, pour un niveau de complexité/flexibilité fixé, la courbe rouge foncée donne une estimation de

$$\mathbb{E} \left[\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid D_n] \right] = \mathbb{E}[(Y - \hat{f}_{D_n}(X))^2]. \quad (1.13)$$

□

Exercice 11.

En reprenant les éléments de l'exercice 9, représenter graphiquement les quantités $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$ et $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ en fonction de la complexité/flexibilité du modèle (pour un degré maximum égal à 10) pour 100 réalisations d_n de l'ensemble d'apprentissage D_n (de taille 40) ainsi que les courbes moyennes associées. $\square$

1.2.5 Compromis/dilemme biais-variance

La forme typique des courbes donnant $\text{Err}_{\ell_2}(\hat{f}_{d_n}, d_n)$ et $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ en fonction de la complexité/flexibilité du modèle peut s'expliquer théoriquement en obtenant une décomposition *biais-variance* de l'erreur de prédiction espérée/l'erreur de généralisation espérée dans (1.13). Supposons encore une fois que le modèle de régression vrai soit donné par

$$Y = f_{\ell^2}^*(X) + \epsilon \text{ p.s.,}$$

où ϵ est une variable aléatoire centrée (c-à-d, $\mathbb{E}[\epsilon] = 0$) indépendante de X représentant la variabilité/l'incertitude de la relation entre X et Y . On s'intéresse à l'erreur quadratique de prédiction espérée lorsque $X = x$, c-à-d, à la quantité $\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X = x]$. Cette dernière se décompose comme suit :

$$\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X = x] = \mathbb{V}[\epsilon] + \mathbb{V}[\hat{f}_{D_n}(x)] + (\text{Biais}[\hat{f}_{D_n}(x)])^2, \quad (1.14)$$

où $\text{Biais}[\hat{f}_{D_n}(x)] = \mathbb{E}[\hat{f}_{D_n}(x)] - f_{\ell^2}^*(x)$. Cette erreur prend en compte la variabilité de l'échantillon aléatoire d'apprentissage D_n ainsi que celle de Y lorsque $X = x$.

Le premier terme à droite dans (1.14) est la variance de Y lorsque $X = x$ et représente la variabilité de Y autour de son espérance $f_{\ell^2}^*(x)$; cette erreur étant irréductible, elle ne peut être évitée sauf si $\epsilon = 0$ p.s. (c-à-d, dans le cas de rapport bruit sur signal nul). Le troisième terme fait intervenir le *biais* qui quantifie le comportement moyen de $\hat{f}_{D_n}(x)$ autour de $f_{\ell^2}^*(x)$. Le second terme quantifie la variabilité de l'estimateur $\hat{f}_{D_n}(x)$ de $f_{\ell^2}^*(x)$ autour de son espérance $\mathbb{E}[\hat{f}_{D_n}(x)]$.

Typiquement, lorsque la flexibilité/complexité de $\hat{f}_{D_n}$ augmente, la variance de $\hat{f}_{D_n}(x)$ augmente et son biais diminue. Ainsi, choisir la flexibilité/complexité d'un modèle en se fondant sur $\text{Err}_{\ell_2}(\hat{f}_{d_n}, t)$ revient à un compromis biais-variance.

Exercice 12.

Donner une interprétation de $\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X = x]$ et démontrer (1.14). $\square$

Exercice 13.

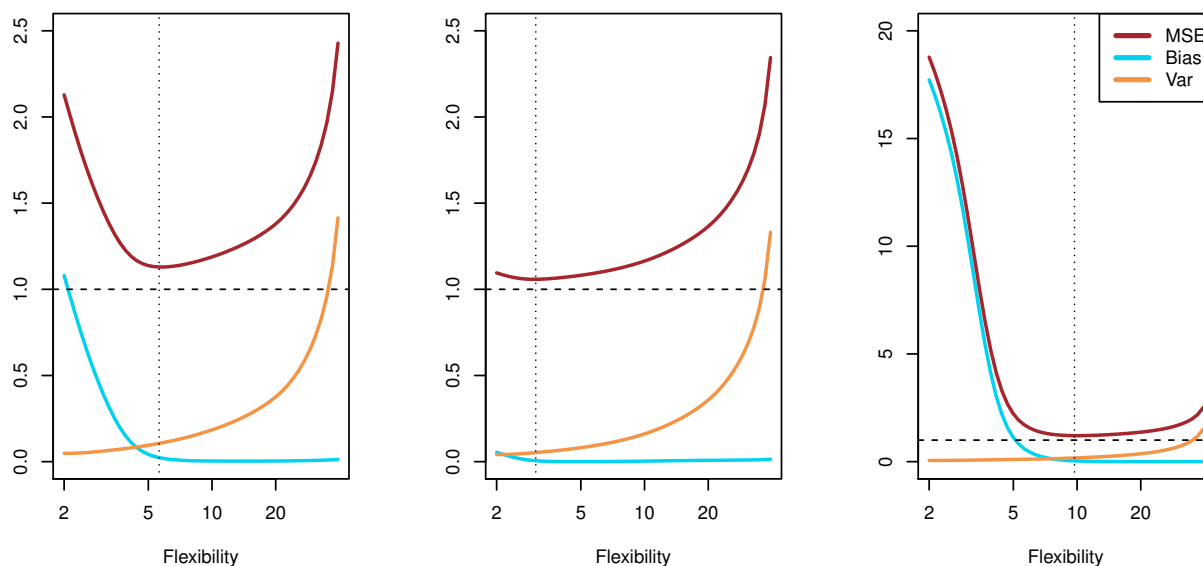
Exprimer l'erreur de généralisation espérée dans (1.13) en fonction de la variable aléatoire $\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2 \mid X]$ et en déduire que $\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2]$ satisfait une décomposition biais-variance "espérée" de la forme :

$$\mathbb{E}[(Y - \hat{f}_{D_n}(X))^2] = \mathbb{V}[\epsilon] + \mathbb{E} \left[\mathbb{V}[\hat{f}_{D_n}(X) \mid X] \right] + \mathbb{E} \left[(\mathbb{E}[\hat{f}_{D_n}(X) \mid X] - f(X))^2 \right]. \quad (1.15)$$

Donner une interprétation de cette décomposition. □

Important : Afin de ne pas confondre les différentes erreurs, il convient de se rappeler des différentes sources de variabilité et lesquelles sont fixées dans une définition donnée.

Décomposition biais-variance “espérée” en fonction de la flexibilité pour les trois exemples précédents :



1.3 Apprentissage statistique supervisé : le cas de la “classification”

Rappel sur le vocabulaire : voir la fin de la Section 1.1.1.

1.3.1 Objectifs

Dans le cas de problèmes de classification, la variable à expliquer Y est en réalité qualitative (même si elle est codée numériquement). On note $\mathcal{Y}$ ses valeurs possibles. Par exemple, $\mathcal{Y} \in \{\text{spam}, \text{non spam}\}$, $\mathcal{Y} = \{0, 1, \dots, 9\}$, $\mathcal{Y} = \{-1, 1\}$, etc.

Objectifs :

- Construire un “classifieur” C (fonction de $\mathbb{R}^p$ dans $\mathcal{Y}$) qui à chaque valeur x de X associe une “classe” dans $\mathcal{Y}$.
- Évaluer la qualité/performance/incertitude du classifieur C .

- Comprendre le rôle des différentes variables explicatives $X_1, \dots, X_p$.

1.3.2 Le classifieur optimal de Bayes

- Sans perte de généralité, on suppose que $\mathcal{Y} = \{1, \dots, K\}$, où K est un entier supérieur ou égal à 2. Soit (x, y) une réalisation de (X, Y) . Lorsque $X = x$, la vraie valeur de Y est y alors que sa valeur prédite par le classifieur C est $C(x)$. L'ensemble des K^2 situations possibles peut être représenté par une matrice dite *de confusion* :

$y \rightarrow$ $C(x) \downarrow$	1	...	K
1	décision correcte		erreur
$\vdots$	$\vdots$	$\ddots$	$\vdots$
K	erreur		décision correcte

- Afin de mesurer la performance du classifieur C , il faut définir une fonction ℓ de $\mathcal{Y} \times \mathcal{Y}$ dans $\mathbb{R}^+$ quantifiant la gravité des erreurs de prédiction.
- Cette fonction est toujours nulle “sur la diagonale de la matrice précédente”. Comme dans le cas de la régression, elle est appelée *fonction de perte*.
- Une des fonctions de perte les plus simples et les plus couramment utilisées est la perte “0/1” donnée par :

$$\ell_{0/1}(y, y') = \mathbf{1}[y \neq y'].$$

[De nombreuses généralisations sont possibles.]

- L'erreur de prédiction espérée (EPE) associée est alors

$$\mathcal{E}_{\ell_{0/1}}(C) = \mathbb{E}[\ell_{0/1}(Y, C(X))] = \mathbb{E}[\mathbf{1}[Y \neq C(X)]] = \mathbb{P}[Y \neq C(X)]. \quad (1.16)$$

- Parmi toutes les fonctions C de $\mathbb{R}^p$ dans $\mathcal{Y}$, la “fonction” pour laquelle $\mathcal{E}_{\ell_{0/1}}(C)$ est minimale est :

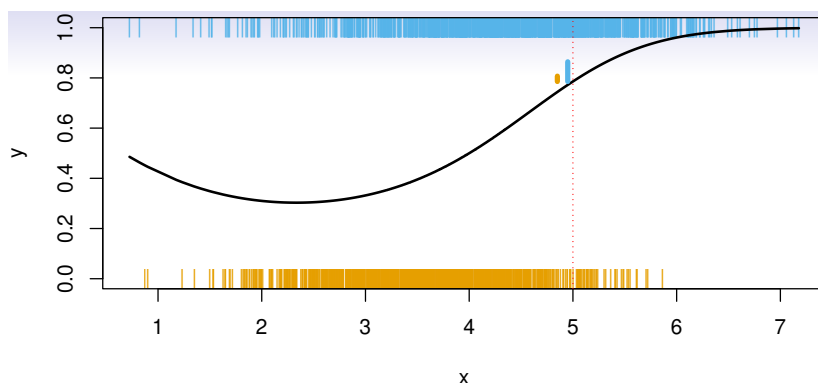
$$C_{\ell_{0/1}}^*(x) = \arg \max_{k \in \{1, \dots, K\}} p_k(x), \quad (1.17)$$

où $p_k(x) = \mathbb{P}[Y = k \mid X = x]$ est la probabilité d'être dans la classe k sachant que $X = x$.

Exercice 14.

Rappeler la définition de l'opérateur $\arg \max$. Est-ce que l'objet mathématique défini par (1.17) est bien une fonction de $\mathbb{R}^p$ dans $\mathcal{Y}$? Modulo un léger manque de rigueur donc, interpréter (1.17) et montrer que, pour tout classifieur C , $\mathcal{E}_{\ell_{0/1}}(C_{\ell_{0/1}}^*) \leq \mathcal{E}_{\ell_{0/1}}(C)$. $\square$

- En toute rigueur, il faudrait remplacer “=” par “ $\in$ ” dans (1.17). Nous gardons néanmoins la formulation telle qu’elle est car c’est celle qui est utilisée dans la littérature.
- Le classifieur donné par (1.17) est un classifieur théorique car les fonctions $p_1, \dots, p_K$ sont inconnues. Il est appelé *classifieur optimal de Bayes* (même s’il n’est optimal que pour la fonction de perte $\ell_{0/1}$).
- Considérons le cas $\mathcal{Y} = \{0, 1\}$. Le classifieur optimal de Bayes dépend alors des fonctions $p_k(x) = \mathbb{P}[Y = k \mid X = x]$, $k \in \{0, 1\}$. Clairement, $p_0(x) = \mathbb{P}[Y = 0 \mid X = x] = 1 - \mathbb{P}[Y = 1 \mid X = x] = 1 - p_1(x)$.
- Ainsi, dans le cas $\mathcal{Y} = \{0, 1\}$, conditionnellement à $X = x$, Y est une variable aléatoire de Bernoulli de paramètre $p_1(x)$.
- Imaginons un exemple avec $p = 1$ dans lequel la courbe noire du graphique ci-dessous représente la fonction p_1 :

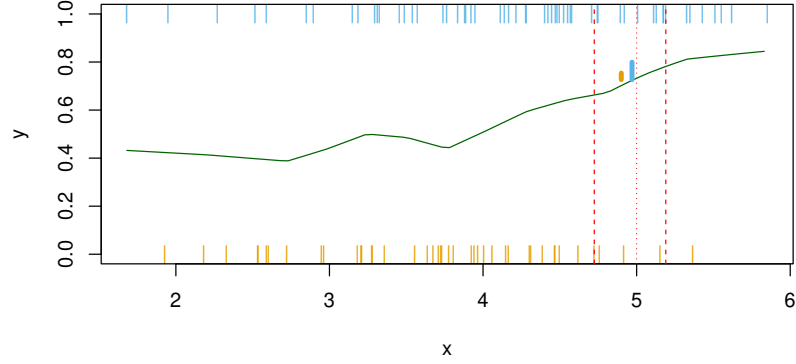


La prédiction du classifieur optimal de Bayes lorsque $X = 5$ est donc $C_{\ell_{0/1}}^*(5) = 1$ car $p_1(5) > p_0(5)$.

- Imaginons maintenant que nous ne disposions que d’un ensemble d’apprentissage d_n de (X, Y) . Ce dernier est représenté par les “ticks” oranges (classe 0) et bleus (classe 1) sur le graphique ci-dessus. Que pourrions-nous faire ?

1.3.3 La méthode des k plus proches voisins

- Pour prédire la valeur de Y lorsque $X = 5$, nous pourrions essayer d’estimer $p_0(5)$ et $p_1(5)$. Pour cela, nous pourrions utiliser les observations dans d_n pour lesquelles $x_i \approx 5$. Par exemple :



Comme ici $\hat{p}_{0,d_n}(5) < \hat{p}_{1,d_n}(5)$, la prédiction $\hat{C}(5)$ est 1.

- L'idée précédente pour estimer les probabilités conditionnelles s'apparente à l'estimation par la méthode des *k plus proches voisins*.
- Considérons le cas général $p \geq 1$ et $\mathcal{Y} = \{1, \dots, K\}$, $K \geq 2$. Soit $k \in \{1, \dots, n\}$ fixé, soit d une distance sur $\mathbb{R}^p$ choisie et, pour tout x et $i \in \{1, \dots, n\}$, soit $d_i^x = d(x, x_i)$. Notons $d_{(1)}^x, \dots, d_{(n)}^x$ un ré-arrangement de $d_1^x, \dots, d_n^x$ tel que $d_{(1)}^x \leq \dots \leq d_{(n)}^x$, et soit

$$\mathcal{N}_k(x) = \{i : d_i^x \leq d_{(k)}^x\}.$$

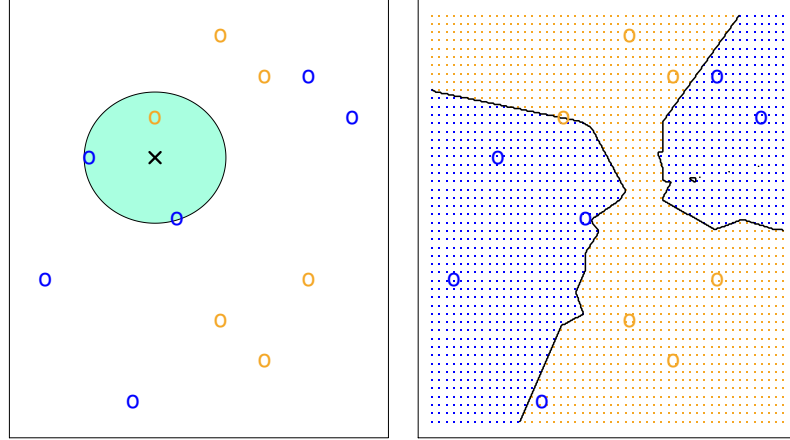
- L'estimation de p_j , $j \in \{1, \dots, K\}$, par la méthode des k plus proches voisins est alors donnée par

$$\hat{p}_{j,d_n}(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} \mathbf{1}[y_i = j]. \quad (1.18)$$

- Il s'ensuit qu'une estimation naturelle du classifieur optimal de Bayes par la méthode des k plus proches voisins est donnée par :

$$\hat{C}_{d_n,k}(x) = \arg \max_{j \in \{1, \dots, K\}} \hat{p}_{j,d_n}(x). \quad (1.19)$$

- Exemple pour $p = 2$, d la distance Euclidienne et $K = 2$:



- Cette technique souffre néanmoins toujours de la malédiction de la dimension. Cette dernière affecte plus les estimations $\hat{p}_{j,d_n}$, $j \in \{1, \dots, K\}$, que $\hat{C}_{d_n,k}$ dans (1.19).

Exercice 15.

On suppose que $Y = C_{\ell_{0/1}}^*(X)$ avec $Y \in \mathcal{Y} = \{0, 1\}$ et $X = (X_1, X_2)$ un vecteur aléatoire dont les deux composantes sont indépendantes et uniformément distribuées sur $]0, 1[$, et

$$p_1(x) = \min(\|x\|_2, \|(1, 1) - x\|_2, 1), \quad x \in]0, 1[^2.$$

1. Interpréter la fonction p_1 et donner la fonction p_0 .
2. Donner du code R permettant de représenter la fonction p_1 . La fonction `contour()` pourra par exemple être utilisée. En déduire, une représentation graphique de la fonction $C_{\ell_{0/1}}^*$.
3. Écrire une fonction R permettant de calculer $C_{\ell_{0/1}}^*$. Cette fonction représente le classifieur théorique.
4. Écrire une fonction R permettant de générer des réalisations de (X, Y) , fixer la graine aléatoire à 321 et générer un ensemble d_n de taille $n = 100$.
5. L'objectif est maintenant d'estimer $C_{\ell_{0/1}}^*$ à partir de d_n en utilisant la méthode des plus proches voisins. Utiliser la fonction `knn()` se trouvant dans le package `class` (installé par défaut) et représenter $\hat{C}_{d_n,k}$ dans (1.19) sur une grille de points (régulière et fine) dans $]0, 1[^2$ pour $k = 1, 20, 50, 100$. Identifier les situations qui correspondent à un sous-apprentissage/à un sur-apprentissage.

□

Éléments de solution:

```

## Fonction p1 donnant la probabilité d'être de la classe 1 au point (x1,x2)
p1 <- function(x1, x2) pmin(sqrt(x1^2 + x2^2), sqrt((1-x1)^2 + (1-x2)^2), 1)

## Grille fine et régulière dans [0,1]^2
m <- 100
x1.grid <- x2.grid <- seq(from = 0, to = 1, length.out = m)
grid <- expand.grid(x = x1.grid, y = x2.grid)

## Valeurs de p1 sur la grille
p1.val <- matrix(p1(x1 = grid[,1], x2 = grid[,2]), nrow = m, ncol = m)

## Contours de p1
contour(x = x1.grid, y = x2.grid, z = p1.val)

## Génération de réalisations de (X,Y)
## Pour générer une réalisation, on génère d'abord x1 et x2
## Sachant que X1 = x1 et X2 = x2, Y est une variable aléatoire
## de Bernoulli de paramètre p1(x1, x2)
rmod <- function(n) {
  x1 <- x2 <- y <- numeric(n)
  for (i in seq_len(n)) {
    x1[i] <- runif(1)
    x2[i] <- runif(1)
    y[i] <- rbinom(1, size = 1, prob = p1(x1 = x1[i], x2 = x2[i]))
  }
  data.frame(x1, x2, y)
}

## Le code précédent est volontairement sous-optimal
## Sa lisibilité est privilégiée

## Un ensemble d'apprentissage
set.seed(321)
app <- rmod(100)

## Représentation graphique
plot(app$x1, app$x2, pch = "")
text(app$x1, app$x2, app$y, col = app$y + 1)

library(class)

## Estimation de la classe par la méthode des p.p.v.
par(mfrow = c(2, 2))
for (k in c(1, 10, 20, 50)) {
  y.ppv <- knn(train = app[,1:2], test = grid, cl = app$y, k = k)
  plot(grid, col = y.ppv, main = paste("k =", k))
}

```

}

La situation $k = 1$ (resp. $k = 50$) semble correspondre
à du sur-apprentissage (resp. sous-apprentissage)

□

1.3.4 Évaluation de la qualité de l'ajustement d'un classifieur

- Étant donnée une fonction g de $\mathbb{R}^p$ dans $\mathcal{Y}$ et un ensemble e de réalisations de (X, Y) , on note

$$\text{Err}_{\ell_{0/1}}(g, e) = \text{Moy}_{(x_i, y_i) \in e} [\ell_{0/1}(y_i, g(x_i))] = \text{Moy}_{(x_i, y_i) \in e} \mathbf{1}[y_i \neq g(x_i)].$$

Cette quantité s'interprète comme l'erreur au sens de la perte $\ell_{0/1}$ de g sur l'ensemble e .

- Par analogie avec le cas de la régression, pour mesurer la performance d'un classifieur ajusté $\hat{C}_{d_n}$, il est naturel d'utiliser

$$\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n}, t) = \text{Moy}_{i \in t} \mathbf{1}[y_i \neq \hat{C}_{d_n}(x_i)], \quad (1.20)$$

où t est un ensemble de réalisations de (X, Y) non utilisé lors de l'ajustement du modèle. C'est l'erreur de test.

- Si t était de cardinal “infini”, c-à-d, si t représentait la “population”, c'est le classifieur optimal de Bayes (utilisant les vraies probabilités conditionnelles $p_1, \dots, p_K$ inconnues) qui aurait la plus faible erreur.
- La quantité $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n}, t)$ est clairement une estimation de l'erreur de généralisation du classifieur ajusté $\hat{C}_{d_n}$:

$$\mathcal{E}_{\ell_{01}}(\hat{C}_{d_n}) = \mathbb{E}[\ell_{0/1}(Y, \hat{C}_{d_n}(X))] = \mathbb{P}[Y \neq \hat{C}_{d_n}(X)],$$

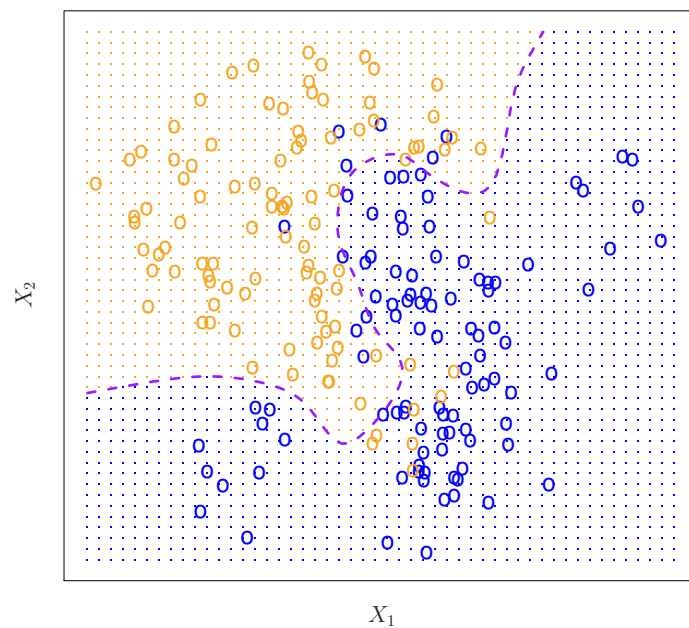
où $\mathcal{E}_{\ell_{01}}$ est donné par (1.16).

- Par analogie avec le cas de la régression, une sous-estimation de cette dernière (en général) est donnée par l'erreur d'apprentissage :

$$\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n}, d_n) = \text{Moy}_{i \in d_n} \mathbf{1}[y_i \neq \hat{C}_{d_n}(x_i)]. \quad (1.21)$$

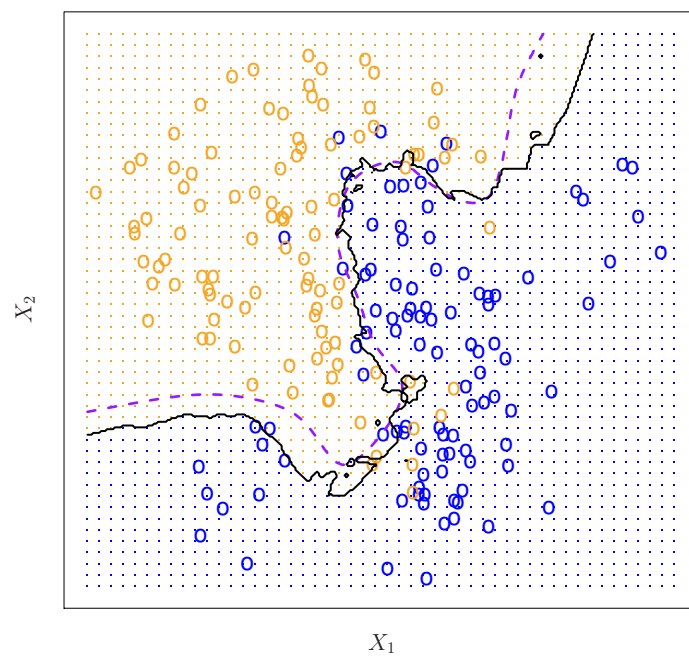
Exemple : Avec $p = 2$ et $K = 2$.

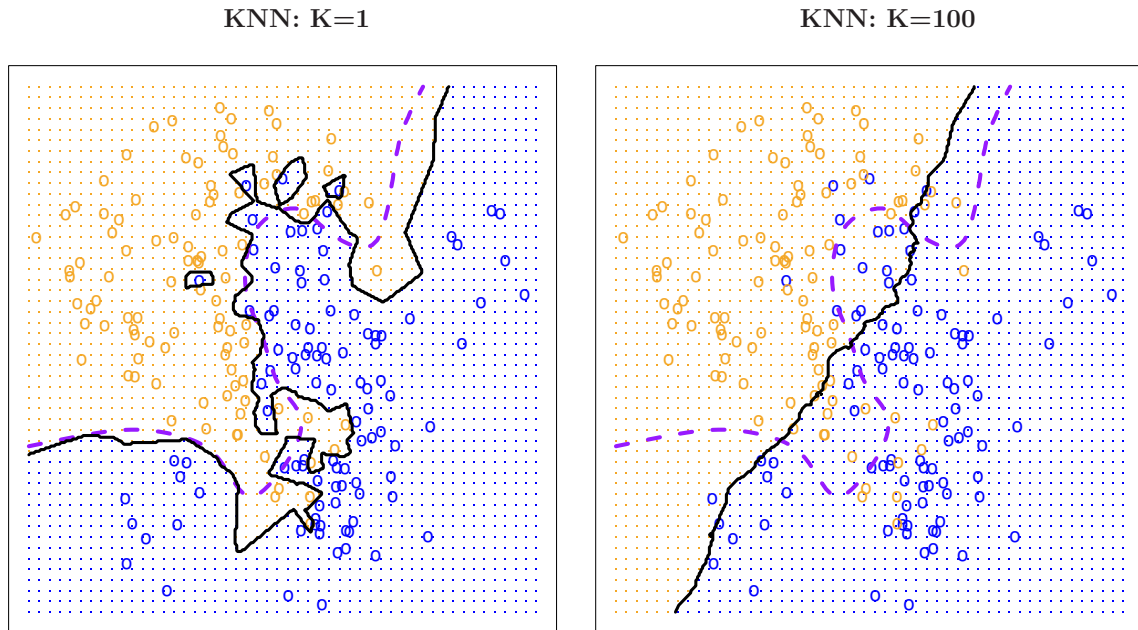
- Les données disponibles d_n et “la frontière de décision” du classifieur optimal de Bayes :



— Le classifieur de k plus proches voisins avec $k = 10$, $k = 1$ et $k = 100$:

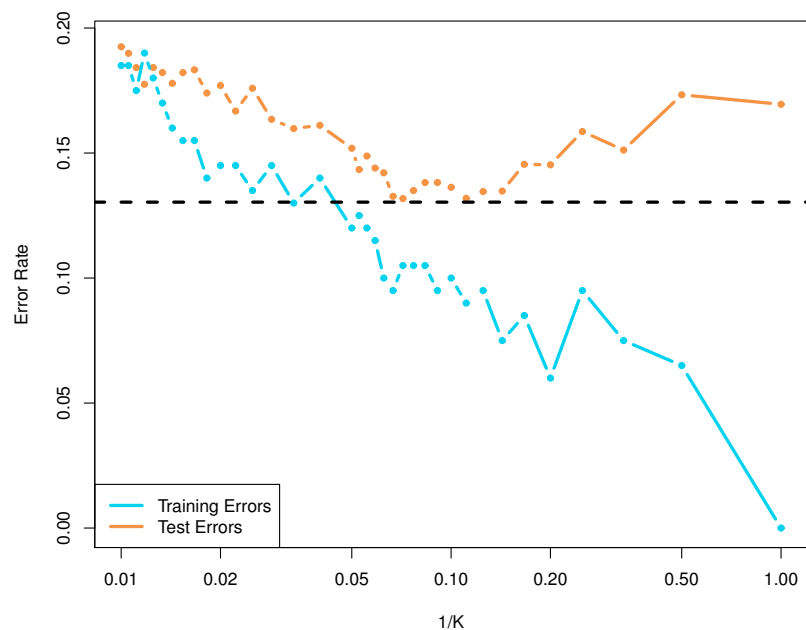
KNN: K=10





Quelle situation ci-dessus correspond à du sur-apprentissage/sous-apprentissage ?

- L'erreur d'apprentissage (en bleu) et l'erreur de test (en rouge) en fonction du "paramètre de complexité/flexibilité" $1/k$. Au fait, que représentent les pointillés ?



Exercice 16.

En partant du contexte de l'exercice [15](#) :

1. Générer un sous-ensemble de test t de taille 1000 et représenter graphiquement dans R les quantités $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n,k}, d_n)$ et $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n,k}, t)$ en fonction de $1/k$.
2. Pour quelle(s) raison(s) l'erreur de test n'est-elle pas plus faible ?
3. En partant du code précédent, représenter graphiquement les quantités $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n,k}, d_n)$ et $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n,k}, t)$ en fonction de $1/k$ pour 100 réalisations d_n de l'ensemble d'apprentissage D_n (de taille 100) ainsi que les courbes moyennes associées.

□

Éléments de solution:

```
## Un ensemble de test
test <- rmod(1000)

## Fonction pour calculer l'erreur
erreur <- function(y, ppv) mean(abs(y + 1 - as.numeric(ppv)))

## Erreur d'apprentissage et de l'erreur de test
k.val <- seq_len(50)
err.app <- err.test <- numeric(length(k.val))
for (k in k.val) {
  y.ppv.app <- knn(train = app[,1:2], test = app[,1:2], cl = app$y, k = k)
  err.app[k] <- erreur(app$y, y.ppv.app)
  y.ppv.test <- knn(train = app[,1:2], test = test[,1:2], cl = app$y, k = k)
  err.test[k] <- erreur(test$y, y.ppv.test)
}
plot(1/k.val, err.app, type = "l", xlab = "1/k", ylab = "Error",
      ylim = c(0, 0.5), log = "x") # Échelle logarithmique
points(1/k.val, err.test, type = "l", col = "red")
```

□

- L'exercice précédent illustre le fait que $\mathbb{E}[\ell_{0/1}(Y, \hat{C}_{D_n,k}(X))]$ semble satisfaire une sorte de décomposition biais-variance “espérée”.
- Comme dans le cas de la régression, choisir la flexibilité/complexité d'un modèle de classification en se fondant sur $\text{Err}_{\ell_{0/1}}(\hat{C}_{d_n,k}, t)$ revient à une sorte de compromis biais-variance.

1.3.5 Modèles structurés

- Les machines à vecteurs support sont par exemple des modèles structurés pour C .

- Il est aussi classique de considérer des modèles structurés estimant les vraies probabilités conditionnelles $p_1, \dots, p_K$ (par ex., régression logistique, analyse discriminante Gaussienne, modèles additifs généralisés, ...).

1.4 Apprentissage statistique supervisé : cadre général

1.4.1 Cadre théorique

- Étant donné un vecteur aléatoire (X, Y) de loi inconnue et en notant $\mathcal{Y}$ l'ensemble des valeurs possible pour Y , on cherche une fonction f de $\mathbb{R}^p$ dans $\mathcal{Y}$ telle que la variable aléatoire $f(X)$ soit “aussi proche que possible” de la variable aléatoire Y dans un certain sens.
- Une telle fonction est parfois appelée *fonction de prévision*.
- La proximité entre $f(X)$ et Y est mesurée à l'aide d'une *fonction de perte* ℓ de $\mathcal{Y} \times \mathcal{Y}$ dans $\mathbb{R}^+$ telle que

$$\ell(y, y') = \begin{cases} 0 & \text{si } y = y', \\ > 0 & \text{sinon.} \end{cases}$$

- Étant donnée une fonction de perte ℓ , la qualité/performance d'une fonction de prévision f est mesurée par l'erreur de prévision espérée :

$$\mathcal{E}_\ell(f) = \mathbb{E}[\ell(Y, f(X))], \quad (1.22)$$

parfois également appelée *risque*.

- Étant donnée la nature de $Y/\mathcal{Y}$ et une fonction de perte ℓ , un objectif théorique est alors de trouver une fonction f_ℓ^* telle que, pour toute fonction f , $\mathcal{E}_\ell(f_\ell^*) \leq \mathcal{E}_\ell(f)$, c-à-d, telle que

$$f_\ell^* \in \arg \min_f \mathcal{E}_\ell(f).$$

- Comme nous l'avons vu dans les sections précédentes :
 - lorsque Y est une variable aléatoire continue et $\ell = \ell_2$, f_ℓ^* est la fonction de régression donnée par $f_{\ell_2}^*(x) = \mathbb{E}[Y \mid X = x]$;
 - lorsque Y est une variable aléatoire discrète avec $|\mathcal{Y}| = K$ et $\ell = \ell_{0/1}$, f_ℓ^* est le classifieur optimal de Bayes $C_{\ell_{0/1}}^*$ donné dans (1.17).
- De nombreux autres choix sont possibles pour ℓ :
 - Par exemple, lorsque Y est une variable aléatoire continue et $\ell = \ell_1$ avec $\ell_1(y, y') = |y - y'|$, f_ℓ^* est donné par la médiane théorique conditionnelle, c-à-d, $f_\ell^*(x) = \text{Median}[Y \mid X = x]$.

- Un autre choix de ℓ dans le cas de la régression est la *perte de Huber* ...
- Dans le cas de la classification, comme nous le verrons, la perte $\ell_{0/1}$ n'est pas forcément adaptée aux situations où $\min_{k \in \{1, \dots, K\}} \mathbb{P}[Y = k] \ll \max_{k \in \{1, \dots, K\}} \mathbb{P}[Y = k]$ (classes “très rares” versus classes “fréquentes”).
- Étant donnée la nature de $Y/\mathcal{Y}$ et une fonction de perte ℓ , même si l'on parvient mathématiquement à trouver f_ℓ^* , cette dernière dépendra très souvent de la loi inconnue de (X, Y) et ne sera donc pas calculable sans cette information.
- Il faut donc se placer dans un cadre statistique et utiliser la réalisation d_n de l'échantillon aléatoire D_n .

1.4.2 Cadre statistique

- Nous disposons d'une réalisation d_n de l'échantillon aléatoire D_n . Étant donnée la nature de $Y/\mathcal{Y}$, il nous faut choisir une fonction de perte ℓ .
- Soit $\hat{f}_{D_n}$ un estimateur de f_ℓ^* . Idéalement, nous souhaitons que cet estimateur soit consistant. Informellement, cela revient à souhaiter que, plus n est grand, plus la fonction aléatoire $\hat{f}_{D_n}$ est proche de la fonction f_ℓ^* théorique inconnue dans un certain sens.
- Cela n'est évidemment possible que si l'estimateur considéré est suffisamment flexible. Par exemple, dans le cas de la régression basée sur la perte ℓ_2 , un estimateur type régression linéaire ne sera certainement pas consistant si la fonction de régression $f_{\ell_2}^*$ vraie inconnue est en réalité non linéaire.
- Pour un n fixé et lorsque $X = x$, dans le cas de la régression, nous pouvons définir :
 - le biais de l'estimateur $\hat{f}_{D_n}(x)$ de $f_\ell^*(x)$:

$$\text{Biais}[\hat{f}_{D_n}(x)] = \mathbb{E}[\hat{f}_{D_n}(x)] - f_\ell^*(x),$$

- la variance de l'estimateur $\hat{f}_{D_n}(x)$ de $f_\ell^*(x)$:

$$\mathbb{V}[\hat{f}_{D_n}(x)] = \mathbb{E}[(\hat{f}_{D_n}(x))^2] - (\mathbb{E}[\hat{f}_{D_n}(x)])^2 = \mathbb{E}\left[(\hat{f}_{D_n}(x) - \mathbb{E}[\hat{f}_{D_n}(x)])^2\right],$$

- l'erreur quadratique moyenne de l'estimateur $\hat{f}_{D_n}(x)$ de $f_\ell^*(x)$:

$$\text{EQM}[\hat{f}_{D_n}(x)] = \mathbb{E}\left[(\hat{f}_{D_n}(x) - f_\ell^*(x))^2\right] = (\text{Biais}[\hat{f}_{D_n}(x)])^2 + \mathbb{V}[\hat{f}_{D_n}(x)].$$

- Les notions précédentes ont particulièrement du sens lorsque $\ell = \ell_2$.

- Ce n'est plus forcément le cas pour des problèmes de classification (puisque Y est en réalité une variable quantitative codée numériquement). Par exemple, comment interpréter $\text{Biais}[\hat{f}_{D_n}(x)]$ et $\mathbb{V}[\hat{f}_{D_n}(x)]$ si $\mathcal{Y} = \{1, \dots, K\}$?
- Néanmoins, on est tout de même amené à s'interroger sur la “précision moyenne” et la variabilité de l'estimateur $\hat{f}_{D_n}(x)$. Par abus de langage, on retrouve parfois l'utilisation des termes *biais* et *variance* dans la littérature alors que ces derniers ne sont pas appropriés.
- Évidemment, puisque f_ℓ^* dépend de la perte choisie, un estimateur $\hat{f}_{D_n}$ avec de bonnes propriétés statistiques pour une perte donnée n'aura pas forcément de bonnes propriétés statistiques pour une autre perte.

1.5 Validation et validation croisée

- Étant donnée la nature de $Y/\mathcal{Y}$ et une fonction de perte ℓ choisie, en pratique, nous ne pouvons pas calculer la fonction de prévision idéale f_ℓ^* minimisant l'erreur de prédiction espérée/de généralisation donnée par (1.22).
- Nous avons uniquement des données d_n et des modèles candidats (de régression ou de classification) à notre disposition.
- Ces modèles peuvent être ajustés aux données d_n (par exemple, par moindres carrés dans le cas de modèles de régression linéaire). Notons $\hat{f}_{d_n}^{(1)}, \hat{f}_{d_n}^{(2)}, \dots$, les versions de ces modèles ajustées aux données d_n .
- Quel modèle estimé choisir alors ? Soit ℓ une fonction de perte pertinente pour notre problème. Idéalement, il faudrait pouvoir calculer l'erreur de prédiction espérée/de généralisation $\mathcal{E}_\ell(\hat{f}_{d_n}^{(i)}) = \mathbb{E}[\ell(Y, \hat{f}_{d_n}^{(i)}(X))]$ pour chaque modèle ajusté candidat.
- Étant donnée une fonction g de $\mathbb{R}^p$ dans $\mathcal{Y}$ et un ensemble e de réalisations de (X, Y) , rappelons la notation :

$$\text{Err}_\ell(g, e) = \text{Moy}_{(x_i, y_i) \in e} [\ell(y_i, g(x_i))] = \frac{1}{|e|} \sum_{(x_i, y_i) \in e} \ell(y_i, g(x_i)).$$

- Si nous étions en mesure d'obtenir un ensemble t de réalisations de (X, Y) différent de d_n , $\text{Err}_\ell(\hat{f}_{d_n}^{(i)}, t)$ donnerait une estimation raisonnable de $\mathcal{E}_\ell(\hat{f}_{d_n}^{(i)})$ dans de nombreuses situations pratiques, d'autant meilleure que $|t|$ est grand.
- Comme nous l'avons déjà vu, la quantité $\text{Err}_\ell(\hat{f}_{d_n}^{(i)}, t)$ est appelée *erreur de test* (du modèle ajusté $\hat{f}_{d_n}^{(i)}$).

- Si nous utilisons toutes les données disponibles pour ajuster un modèle candidat, la seule erreur que nous pouvons calculer est l'*erreur d'apprentissage* donnée par $\text{Err}_\ell(\hat{f}_{d_n}^{(i)}, d_n)$. Comme déjà évoqué à plusieurs reprises, cette dernière est généralement une (sous-)estimation médiocre de l'erreur de prédiction espérée/de généralisation $\mathcal{E}_\ell(\hat{f}_{d_n}^{(i)})$ du modèle $\hat{f}_{d_n}^{(i)}$.
- La solution idéale serait donc de disposer d'un grand ensemble de test t en plus de d_n mais cela n'est que rarement possible.
- Certaines approches (pour certains modèles) font des ajustements mathématiques de l'erreur d'apprentissage afin d'obtenir une meilleure estimation de l'erreur de généralisation. Dans le cas du modèle linéaire, c'est le cas par exemple du C_p de Mallows, du AIC (acronyme de l'anglais *Akaike Information Criterion*), du BIC (acronyme de l'anglais *Bayesian Information Criterion*), etc (certains de ces critères sont généralisables à d'autres modèles).
- Nous considérons ci-après une approche générale qui consiste à estimer l'erreur de généralisation à partir d'un sous-ensemble des données disponibles (qui ne sera pas utilisé pour l'apprentissage).

1.5.1 Ensemble de validation/test

- L'idée est de diviser aléatoirement les observations disponibles d_n en deux sous-ensembles : un ensemble a pour l'apprentissage et un ensemble t dit de *validation/test*. On a donc $d_n = a \cup t$.
- Anglais : *hold-out validation set*.
- Division aléatoire (des indices) des observations dans d_n : à gauche pour l'ensemble "d'apprentissage" a , à droite de pour l'ensemble de validation/test t :

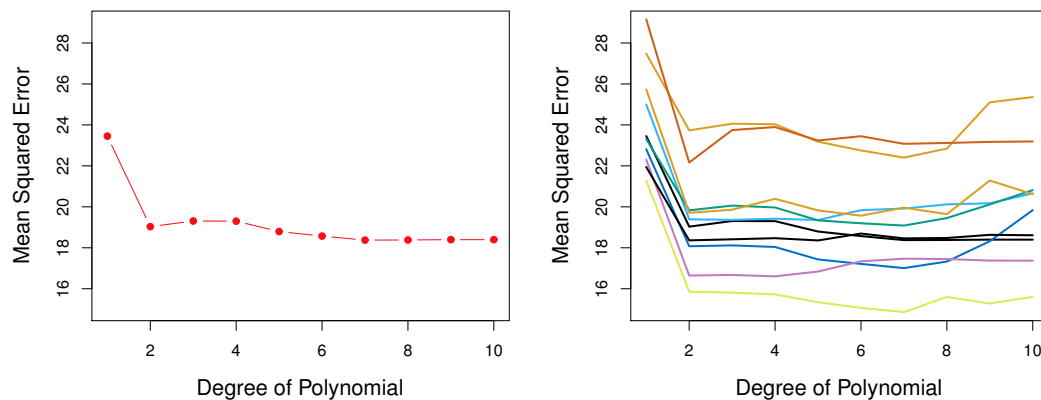


- Les modèles candidats sont ensuite entraînés/ajustés/estimés sur l'ensemble d'apprentissage a et les modèles ajustés sont utilisés pour prédire les Y des observations dans l'ensemble de validation/test t .
- Après avoir été comparées aux vraies valeurs de Y , ces prédictions permettent d'obtenir les erreurs de test $\text{Err}_\ell(\hat{f}_a^{(i)}, t)$ estimant les erreurs de généralisation $\mathcal{E}_\ell(\hat{f}_a^{(i)})$ (et non pas $\mathcal{E}_\ell(\hat{f}_{d_n}^{(i)})$).

- Le modèle choisi est souvent l'un de ceux qui a l'une des erreurs de test $\text{Err}_\ell(\hat{f}_a^{(i)}, t)$ les plus faibles. Il est ensuite typiquement réajusté sur toutes les données disponibles d_n .

Etude empirique sur un exemple simple :

- Données **auto** (disponibles sur Elearn).
- Objectif : choisir le degré maximum d dans une régression polynomiale pour prédire la consommation (**mpg**) en fonction de la puissance (**horsepower**).
- Division aléatoire des 392 observations en deux sous-ensembles : un ensemble d'apprentissage a de 196 observations et un ensemble de validation/test t contenant les 196 observations restantes.
- Dix modèles candidats : $\hat{f}_a^{(1)}, \dots, \hat{f}_a^{(10)}$ correspondants à $d = 1, \dots, d = 10$, respectivement.
- A gauche : $\text{Err}_{\ell_2}(\hat{f}_a^{(d)}, t)$ en fonction de d pour une division aléatoire de d_n . A droite : graphes similaires pour d'autres divisions aléatoires.



Exercice 17.

Reproduire l'étude empirique précédente dans R.

□

Inconvénients de l'approche précédente :

- L'estimation de l'erreur de généralisation basée sur l'ensemble de validation/test peut être très variable, en fonction de quelles observations sont incluses dans l'ensemble d'apprentissage (resp. l'ensemble de validation/test).
- Seul un sous-ensemble des observations disponibles – celles qui sont dans l'ensemble d'apprentissage a – sont utilisées pour ajuster/entraîner/estimer le modèle.

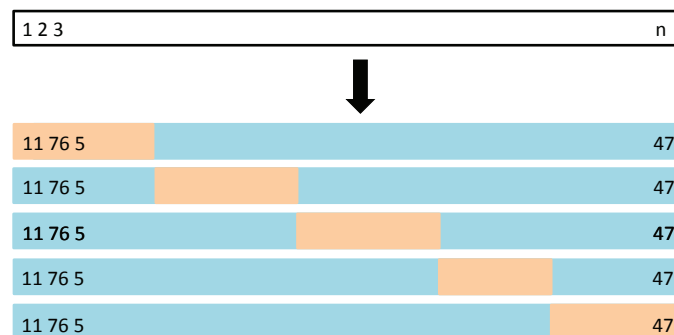
- Cela suggère que l'erreur calculée sur l'ensemble de validation/test a tendance à sur-estimer l'erreur de test (par rapport à un modèle dont l'ajustement utiliserait toutes les observations disponibles).

Remarque :

- On retrouve dans la littérature l'idée de découper les données disponibles non pas en deux mais en trois sous-ensembles : un ensemble d'apprentissage (disons 50%), un ensemble de validation (disons 25%) et un ensemble de test (disons 25%).
- L'idée est alors d'utiliser l'ensemble de validation pour la sélection d'un modèle parmi les modèles candidats et l'ensemble de test pour estimer l'erreur de généralisation du modèle final choisi.

1.5.2 Validation croisée avec K blocs

- Approche très fréquemment utilisée pour estimer l'erreur de généralisation d'un modèle ajusté.
- Anglais : *K-fold cross-validation*.
- Comme dans le cas *hold-out*, les estimations peuvent être utilisées pour sélectionner un modèle parmi les modèles candidats et avoir une idée de son erreur de généralisation.
- L'idée consiste à diviser aléatoirement les données en K sous-ensembles approximativement égaux. Ensuite, sans utiliser le bloc k , le modèle est ajusté sur l'union des $K - 1$ blocs restants et évalué sur le bloc k (qui sert de sous-ensemble de test).
- Cette opération est répétée pour $k \in \{1, \dots, K\}$ et les résultats sont combinés.



- Soit $b_1, \dots, b_K$ les blocs d'observations résultant de la division aléatoire de d_n . On a donc $d_n = b_1 \cup \dots \cup b_K$. Soit n_k le nombre d'observations dans le bloc k . Les blocs sont choisis de sorte que $n_1 \approx \dots \approx n_K$. Si n est un multiple de K , alors $n_k = n/K$.

— On calcule alors

$$CV_{\ell,K}(\hat{f}_{d_n}) = \sum_{k=1}^K \frac{n_k}{n} \text{Err}_{\ell}(\hat{f}_{-b_k}, b_k) = \text{ou} \approx \frac{1}{K} \sum_{k=1}^K \text{Err}_{\ell}(\hat{f}_{-b_k}, b_k),$$

où $\hat{f}_{-b_k}$ est le modèle d'intérêt ajusté sans le bloc d'observations b_k .

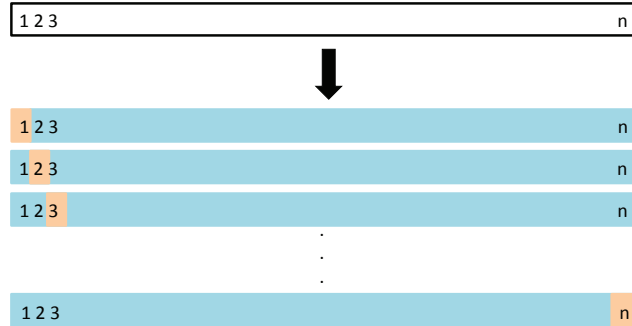
— Attention, le calcul de $CV_{\ell,K}(\hat{f}_{d_n})$ **ne nécessite pas** l'ajustement du modèle sur toutes les observations disponibles, c-à-d, le calcul de $\hat{f}_{d_n}$. La notation $CV_{\ell,K}(\hat{f}_{d_n})$ reflète juste le fait que cette quantité est perçue comme une estimation de l'erreur de généralisation $\mathcal{E}_{\ell}(\hat{f}_{d_n})$.

— Étant donnés des modèles candidats, les quantités $CV_{\ell,K}(\hat{f}_{d_n}^{(1)})$, $CV_{\ell,K}(\hat{f}_{d_n}^{(2)})$, ... sont typiquement utilisées pour choisir l'un d'entre eux. Le modèle choisi est ensuite classiquement réajusté sur toutes les données disponibles d_n .

— Une estimation (imparfaite) de l'erreur standard de $CV_{\ell,K}(\hat{f}_{d_n})$ peut alors être calculée comme suit :

$$\widehat{\text{SE}}[CV_{\ell,K}(\hat{f}_{d_n})] = \sqrt{\frac{1}{K(K-1)} \sum_{k=1}^K (\text{Err}_{\ell}(\hat{f}_{-b_k}, b_k) - CV_{\ell,K}(\hat{f}_{d_n}))^2}. \quad (1.23)$$

— Le cas particulier $K = n$ est appelé *leave-one out cross-validation (LOOCV)* en anglais :



Exercice 18.

1. Soit Z une variable aléatoire telle que $\mathbb{E}[Z] < \infty$ et soit $Z_1, \dots, Z_n$ un échantillon aléatoire de Z . Dans le cadre de l'estimation de $\mathbb{E}[Z]$, rappeler brièvement l'espérance, la variance et l'erreur standard de l'estimateur usuel de $\mathbb{E}[Z]$.
2. La formule donnant $\widehat{\text{SE}}[CV_{\ell,K}(\hat{f}_{d_n})]$ dans (1.23) a-t-elle plus de sens maintenant ? Expliquer.

3. Supposer maintenant que $Z_1, \dots, Z_n$ ont la même loi que Z mais ne sont plus indépendantes. Que vaut la variance de $\bar{Z}_n$?
4. Pour $\beta \in [0, 1]$, on considère le cas où $Z_1 = \varepsilon_1$ et $Z_{i+1} = \beta Z_i + \varepsilon_i$, pour $i \in \{2, \dots, n\}$, où $\varepsilon_1, \dots, \varepsilon_n$ est un échantillon aléatoire de la loi normale centrée réduite. À l'aide d'une simulation R, étudier empiriquement l'influence du coefficient β sur la variance de $\bar{Z}_n$ dans ce cas.
5. Expliquer pourquoi cela pousse à s'inquiéter de la qualité de l'estimateur $\widehat{\text{SE}}[\text{CV}_{\ell, K}(\hat{f}_{d_n})]$? Pour quelle raison est-ce dans le cas de la LOOCV ($K = n$) qu'il est typiquement le plus problématique ?

□

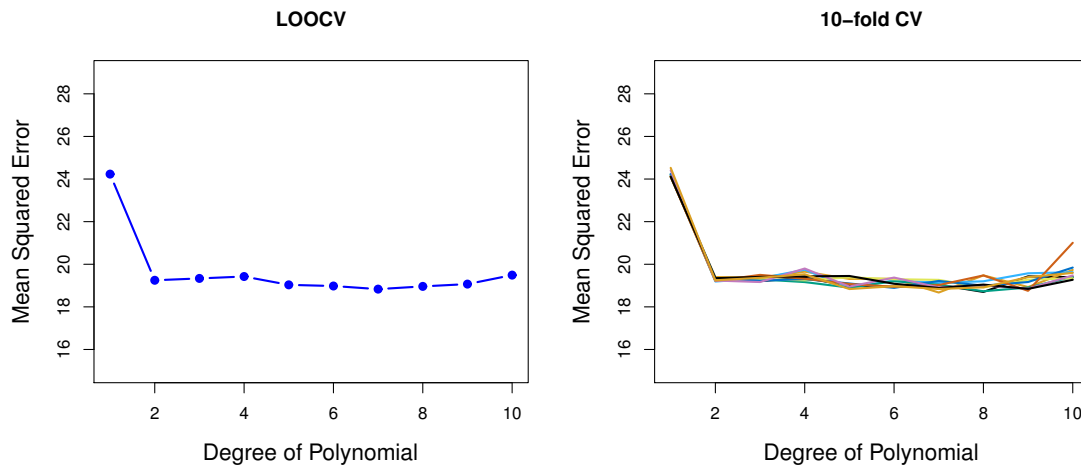
- Dans le cas de régression linéaire ou polynomiale, de façon inattendue, le calcul de la LOOCV ne nécessite qu'un ajustement du modèle sur toutes les données ! En effet, il est possible de montrer que :

$$\text{CV}_{\ell_2, n}(\hat{f}_{d_n}) = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{f}_{d_n}(x_i)}{1 - h_i} \right)^2$$

où $\hat{f}_{d_n}$ est le modèle d'intérêt ajusté sur toutes les données par moindres carrés and h_i est l'influence (*leverage*) de la i ème observation (voir diagonale de la matrice “chapeau” – “*hat*” matrix). Le terme $1 - h_i$ pénalise les observations très influentes.

- La LOOCV est parfois utile mais, typiquement, elle ne perturbe pas suffisamment les données. Les estimations $\text{Err}_{\ell}(\hat{f}_{-b_1}, b_1), \dots, \text{Err}_{\ell}(\hat{f}_{-b_n}, b_n)$ sont en effet très fortement corrélées, ce qui implique que leur moyenne a une forte variance.
- Un meilleur choix est $\text{CV}_{\ell, K}(\hat{f}_{d_n})$ avec $K = 5$ ou 10 .

Retour sur les données auto : À gauche, la LOOCV. À droite, dix exécutions de la validation croisée avec $K = 10$.



Exercice 19.

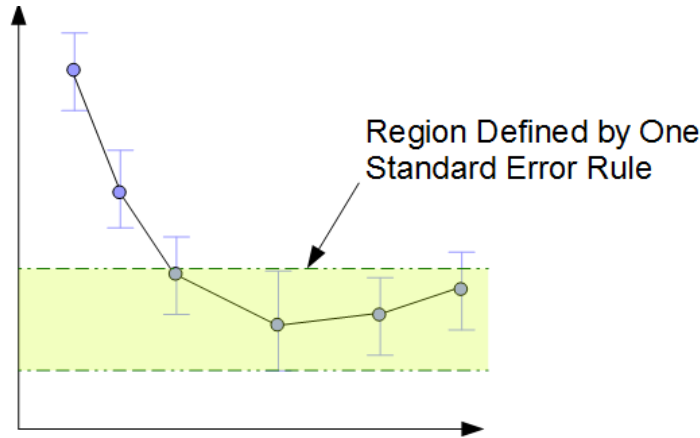
Reprendre le contexte de l'exercice 17 et reproduire le graphique de droite de la figure précédente.

1.5.3 Problèmes liés à la validation croisée

- Puisque chaque ensemble d'apprentissage ne représente que $(K - 1)/K$ en proportion de l'ensemble des données disponibles d_n , les estimations de l'erreur de généralisation par validation croisée vont typiquement sur-estimer l'erreur de généralisation.
- Ce biais est minimisé lorsque $K = n$ (LOOCV). Mais ce cas correspond à une forte variance comme indiqué précédemment.
- $K = 5$ ou 10 semble conduire à un bon compromis biais-variance.

1.5.4 La règle d'une erreur standard

- Certains auteurs suggèrent d'utiliser la *règle d'une erreur standard*.
- Elle consiste à choisir le modèle de complexité minimale parmi ceux dont l'erreur de généralisation estimée (par validation croisée) ne dépasse pas la plus faible erreur de généralisation estimée (par validation croisée) plus une erreur standard :



“Flexibilité/complexité” en abscisse. Erreur de généralisation estimée (par validation croisée) en ordonnée. Les barres verticales représentent plus ou moins une erreur standard estimée.

1.5.5 Validation croisée répétée

- Étant donné que l’estimation de l’erreur standard de $CV_{\ell,K}(\hat{f}_{d_n})$ donnée dans (1.23) est “sous-optimale”, une solution plus sûre que l’approche précédente (lorsqu’elle est réalisable en pratique) consiste à répéter la validation croisée plusieurs fois sur des découpages différents (mais avec le même K).
- Soit $CV_{\ell,K}^{(1)}(\hat{f}_{d_n}), \dots, CV_{\ell,K}^{(M)}(\hat{f}_{d_n})$ les estimations de l’erreur de généralisation estimée par M répétitions de la validation croisée avec K blocs. Une estimation “moins variable” de l’erreur de généralisation estimée est alors donnée par

$$\frac{1}{M} \sum_{m=1}^M CV_{\ell,K}^{(m)}(\hat{f}_{d_n}).$$

1.6 Les packages R caret et tidymodels

- Ces deux packages permettent de mettre en oeuvre les techniques du machine learning/apprentissage statistique dans R . Ils ont le même architecte principal : Max Kuhn.
- Le package `caret` est un package assez ancien fondé sur du “R historique”.
- Le package `tidymodels` semble être le successeur de `caret`. Il repose sur plusieurs packages spécialisés et a adopté la philosophie `tidyverse` (opérateur “pipeline”, etc).

- Pour une introduction au `tidyverse`, voir par exemple l'excellent document de Julien Barnier : https://juba.github.io/tidyverse/introduction_au_tidyverse.pdf

Exemple avec `caret` :

```
library(caret)

## Fonction p1 donnant la probabilité d'être de la classe 1 au point (x1, x2)
p1 <- function(x1, x2) if (x1 < 0.4 && x2 < 0.5) 0.9 else 0.1

## Génération de réalisations de (X,Y)
rmod <- function(n) {
  x1 <- runif(n)
  x2 <- runif(n)
  y <- numeric(n)
  for (i in 1:n)
    y[i] <- rbinom(1, size = 1, prob = p1(x1 = x1[i], x2 = x2[i]))
  data.frame(x1 = x1, x2 = x2, y = factor(y))
}

## Données
d <- rmod(1000)
plot(d[,1:2], col = d$y)

## Valeurs possibles pour le nombre de plus proches voisins
k.val <- data.frame(k = seq(1, 50, by = 1))

## Approche validation/test pour le choix de k
tc1 <- trainControl(method = "LGOCV", number = 1, p = 0.5)
out1 <- train(y ~ ., data = d, method = "knn", trControl = tc1, tuneGrid = k.val)
out1

## "Nouvelles données" sans y
d.new <- rmod(100)[,1:2]
## Prédiction du "meilleur" modèle
predict(out1$finalModel, newdata = d.new)

## Approche validation/test avec 5 répétitions
tc2 <- trainControl(method = "LGOCV", number = 5, p = 0.5)
out2 <- train(y ~ ., data = d, method = "knn", trControl = tc2, tuneGrid = k.val)
out2

## Validation croisée avec K = 10 blocs
tc3 <- trainControl(method = "cv", number = 10)
out3 <- train(y ~ ., data = d, method = "knn", trControl = tc3, tuneGrid = k.val)
```

out3

```
## Validation croisée répétée 5 fois
tc4 <- trainControl(method = "repeatedcv", number = 10, repeats = 5)
out4 <- train(y ~ ., data = d, method = "knn", trControl = tc4, tuneGrid = k.val)
out4
```

Plus de détails ici : <https://topepo.github.io/caret/>

Exemple avec tidymodels :

```
library(tidymodels)

## Fonction p1 donnant la probabilité d'être de la classe 1 au point (x1, x2)
p1 <- function(x1, x2) if (x1 < 0.4 && x2 < 0.5) 0.9 else 0.1

## Génération de réalisations de (X,Y)
rmod <- function(n) {
  x1 <- runif(n)
  x2 <- runif(n)
  y <- numeric(n)
  for (i in 1:n)
    y[i] <- rbinom(1, size = 1, prob = p1(x1 = x1[i], x2 = x2[i]))
  tibble(x1 = x1, x2 = x2, y = factor(y))
}

## Données
d <- rmod(1000)
ggplot(d) +
  geom_point(aes(x = x1, y = x2, color = y))

## Définition du modèle et de son implémentation
mod_knn <- nearest_neighbor(neighbors = tune(), weight_func = "rectangular") |>
  set_mode("classification") |>
  set_engine("kkn")

## Le "workflow"
knn_wf <- workflow() |> add_model(mod_knn) |> add_formula(y ~ .)

## Pour la validation croisée répétée
set.seed(123)
cv <- vfold_cv(d, v = 10, repeats = 5)
cv |> head()
```

```

## Valeurs de k
k_val <- tibble(neighbors = seq(1, 50, by = 1))

## Validation croisée répétée
knn_cv <- knn_wf |>
  tune_grid(resamples = cv, grid = k_val, metrics = metric_set(accuracy))
res <- knn_cv |> collect_metrics()
ggplot(res) + aes(x=neighbors,y=mean) + geom_line() + ylab("Accuracy")

## La "meilleure" valeur de k
knn_cv |> show_best(metric = "accuracy") |> select(1:6)
best_k <- knn_cv |> select_best(metric = "accuracy")
best_k

## Avec la règle d'un écart-type
knn_cv |> select_by_one_std_err(desc(neighbors), metric = "accuracy")

## Réajustement du modèle avec toutes les données
final_knn <- knn_wf |> finalize_workflow(best_k) |> fit(data = d)

## "Nouvelles données" sans y
d_new <- rmod(100) |> select(1,2)
## Prédiction du "meilleur" modèle
final_knn |> predict(new_data = d_new)
## Pour rajouter les prédictions aux données
final_knn |> augment(new_data = d_new)

```

Plus de détails ici : <https://www.tidymodels.org/>

Chapitre 2

Analyse discriminante

- Ensemble de méthodes/modèles pour la classification.
- Sans perte de généralité, on suppose que $\mathcal{Y} = \{1, \dots, K\}$ avec $K \geq 2$.
- La plupart des versions de l’analyse discriminante nécessitent de supposer que X est un vecteur aléatoire absolument continu.
- L’idée de ces méthodes est de modéliser la loi de X pour chacune des classes et d’utiliser ensuite le “théorème de Bayes” pour obtenir des estimations des fonctions $p_k(x) = \mathbb{P}[Y = k \mid X = x]$, $k \in \{1, \dots, K\}$.
- Une fois estimées, ces dernières peuvent être utilisées pour obtenir une estimation du classifieur optimal de Bayes donné par (1.17).
- Lorsque l’on base la modélisation sur la loi normale (multivariée), on parle d’*analyse discriminante linéaire* ou *quadratique*.
- Cet ensemble de modèles de classification est néanmoins très vaste : d’autres lois que la loi normale peuvent être utilisées (et on peut même adopter une approche non paramétrique).
- Lorsque l’on fait l’hypothèse que les variables explicatives sont indépendantes conditionnellement à Y , on obtient le classifieur de Bayes naïf qui donne souvent de bons résultats lorsque p est grand.

2.1 Le “théorème de Bayes” pour la classification

- Thomas Bayes était un mathématicien Britannique du 18e siècle.
- Ses travaux sont notamment à l’origine de la statistique bayésienne (par opposition à

la statistique fréquentiste).

- Nous allons nous intéresser à un résultat simple connu sous le nom de “théorème de Bayes”.
- Supposons temporairement que X soit un vecteur aléatoire discret. Il est alors facile de vérifier que

$$\mathbb{P}(Y = k \mid X = x) = \frac{\mathbb{P}[X = x \mid Y = k] \mathbb{P}[Y = k]}{\mathbb{P}[X = x]}.$$

- Pour $k \in \{1, \dots, K\}$, notons $\pi_k = \mathbb{P}[Y = k]$ la probabilité (inconnue) d’avoir une observation (X, Y) de la classe k .
- Dans l’essentiel de cette section, nous supposons que X est un vecteur aléatoire **absolument continu**. Notons alors f_k la densité (inconnue) de la loi conditionnelle de X sachant que $Y = k$.
- Les idées derrière le “théorème de Bayes” nous permettent alors d’écrire, pour tout $k \in \{1, \dots, K\}$:

$$p_k(x) = \mathbb{P}(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{\ell=1}^K \pi_\ell f_\ell(x)}. \quad (2.1)$$

- Supposons un instant que nous connaissions les probabilités $\pi_1, \dots, \pi_K$ et les densités $f_1, \dots, f_K$.
- A l’aide de (2.1), nous pourrions alors utiliser le classifieur optimal de Bayes donné par

$$C_{\ell_{0/1}}^*(x) = \arg \max_{k \in \{1, \dots, K\}} p_k(x),$$

pour prédire la classe Y lorsque $X = x$.

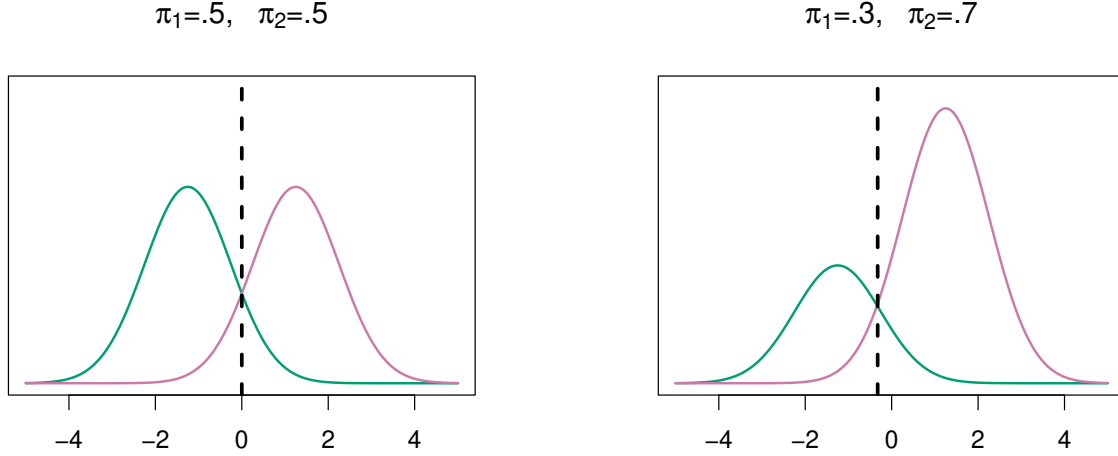
Dans le cas où X est un vecteur aléatoire absolument continu et en notant f_k la densité de $X \mid Y = k$, le classifieur optimal de Bayes est donné par

$$C_{\ell_{0/1}}^*(x) = \arg \max_{k \in \{1, \dots, K\}} \pi_k f_k(x). \quad (2.2)$$

Exercice 20.

Démontrer l’affirmation précédente. □

Exemple : supposons que $p = 1$, $K = 2$, f_1, f_2 densités de certaines lois normales et $(\pi_1, \pi_2) = (0.5, 0.5)$ (gauche) / $(\pi_1, \pi_2) = (0.3, 0.7)$ (droite) :



Les courbes vertes (resp. rouges) représentent les fonctions $\pi_1 f_1$ (resp. $\pi_2 f_2$). La droite verticale en pointillés est la *frontière de décision* : les x pour lesquels $\pi_1 f_1(x) > \pi_2 f_2(x)$ (resp. $\pi_2 f_2(x) > \pi_1 f_1(x)$) seront classifiés en tant que classe 1 (resp. 2).

2.2 L'analyse discriminante gaussienne dans le cas $p = 1$

- L'hypothèse du modèle est que, pour tout $k \in \{1, \dots, K\}$:

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp \left[-\frac{1}{2} \left(\frac{x - \mu_k}{\sigma_k} \right)^2 \right],$$

où $\mu_k = \mathbb{E}[X \mid Y = k]$ et $\sigma_k^2 = \mathbb{V}[X \mid Y = k]$.

- Évidemment, plus cette hypothèse est plausible, plus le modèle sera susceptible de donner de bons résultats.
- D'après (2.2), pour prédire Y lorsque $X = x$, il nous faut alors comparer les quantités $\pi_k f_k(x)$, ou, de façon équivalente, les quantités $g(\pi_k f_k(x))$ pour n'importe quelle fonction strictement croissante g de $]0, \infty[$ dans $\mathbb{R}$.
- Supposons que $\sigma_1 = \dots = \sigma_K = \sigma$ (c'est l'hypothèse d'*homoscédasticité*) et notons

$$\delta_k(x) = \frac{\mu_k}{\sigma^2} x - \frac{\mu_k^2}{2\sigma^2} + \log \pi_k$$

le *score (de discrimination)* de la classe k .

- On peut alors montrer que

$$\arg \max_{k \in \{1, \dots, K\}} \pi_k f_k(x) = \arg \max_{k \in \{1, \dots, K\}} \delta_k(x). \quad (2.3)$$

- Remarquer que la fonction δ_k est une fonction affine : on parle alors de *discrimination linéaire*.

Exercice 21.

Démontrer l'égalité (2.3). Dans le cas $K = 2$, $\pi_1 = \pi_2 = 0.5$, $\mu_1 \neq \mu_2$, vérifier que la frontière de décision est $x = \frac{\mu_1 + \mu_2}{2}$. $\square$

- En pratique, les paramètres $\pi_1, \dots, \pi_K$, $\mu_1, \dots, \mu_K$ et σ du modèle sont inconnus et doivent être estimés à partir des données disponibles d_n .

Exercice 22.

Dans le contexte considéré, proposer des estimations naturelles des paramètres $\pi_1, \dots, \pi_K$ et $\mu_1, \dots, \mu_K$ à partir de d_n dans le cas où D_n est un échantillon aléatoire de (X, Y) . Des idées pour estimer σ ? $\square$

- Notons $n_k = \sum_{i=1}^n \mathbf{1}(y_i = k)$, $k \in \{1, \dots, K\}$.
- Pour estimer σ^2 (hypothèse d'homoscédasticité), on utilise l'estimation dite *pooled variance estimation* en anglais :

$$\hat{\sigma}^2 = \frac{1}{n - K} \sum_{k=1}^K \sum_{i: y_i = k} (x_i - \hat{\mu}_k)^2 = \sum_{k=1}^K \frac{n_k - 1}{n - K} \hat{\sigma}_k^2,$$

où

$$\hat{\sigma}_k^2 = \frac{1}{n_k - 1} \sum_{i: y_i = k} (x_i - \hat{\mu}_k)^2$$

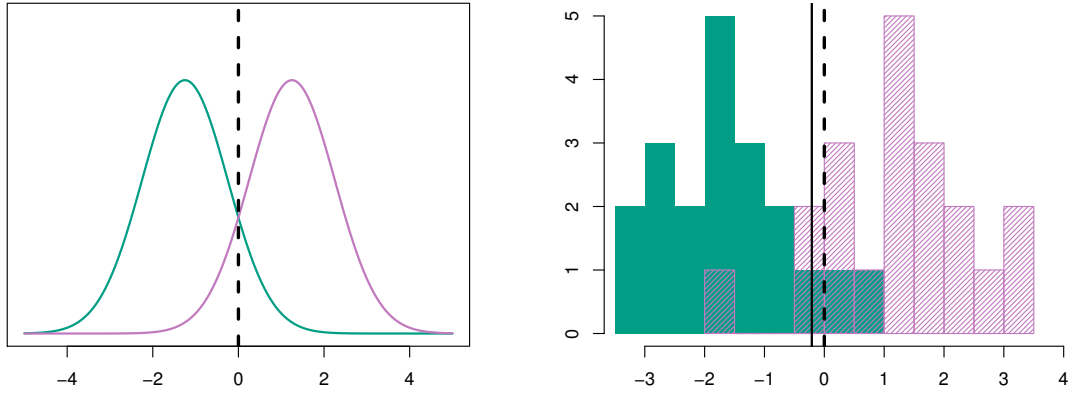
est la variance empirique (version non biaisée) des x_i dans la classe k .

- Noter que si $n_1 = \dots = n_K$ alors $(n_k - 1)/(n - K) = 1/K$ pour tout $k \in \{1, \dots, K\}$, et donc

$$\hat{\sigma}^2 = \frac{1}{K} \sum_{k=1}^K \hat{\sigma}_k^2.$$

- Pour plus de détails, voir par exemple https://en.wikipedia.org/wiki/Pooled_variance.

Exemple : Avec $K = 2$, $\mu_1 = -1.5$, $\mu_2 = 1.5$, $\pi_1 = \pi_2 = 0.5$ et $\sigma^2 = 1$:



La frontière de décision théorique est donnée par la droite verticale en pointillées. La frontière de décision estimée (graphique de droite) est donnée par la droite verticale en trait continu.

Exercice 23.

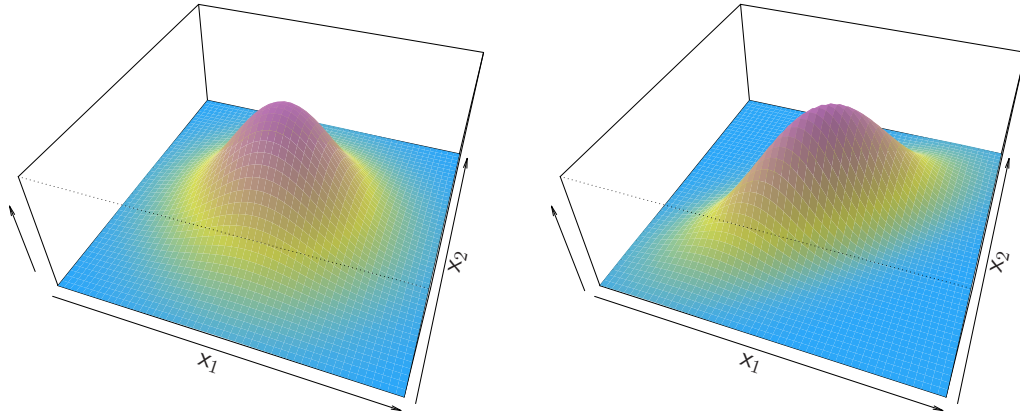
Donner du code R permettant de générer des réalisations d'un échantillon aléatoire D_n de (X, Y) dans le cadre de l'exemple précédent. Générer une réalisation d_n de D_n , estimer les paramètres du modèle et produire un graphique similaire au graphique précédent.

2.3 L'analyse discriminante gaussienne dans le cas $p > 1$

- Les principes sont les mêmes sauf que, comme $p > 1$, c'est la densité de la loi normale multivariée qui est utilisée.
- Nous supposons ainsi que, pour tout $k \in \{1, \dots, K\}$:

$$f_k(x) = \frac{1}{(2\pi)^{p/2} |\Sigma_k|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k) \right],$$

où $\mu_k = \mathbb{E}[X \mid Y = k] \in \mathbb{R}^p$ et $\Sigma_k = \text{Cov}[X \mid Y = k] \in \mathbb{R}^{p \times p}$.



- Supposons dans un premier temps l'homoscédasticité, c-à-d, que $\Sigma_1 = \dots = \Sigma_K = \Sigma$.
- Le *score (de discrimination)* de la classe k est alors donné par :

$$\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k. \quad (2.4)$$

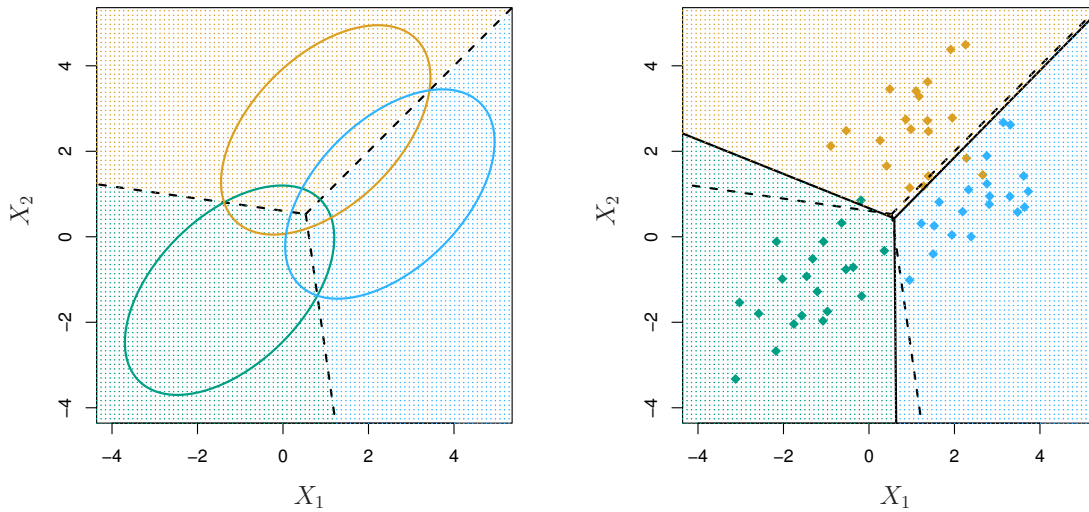
Exercice 24.

Démontrer que

$$\arg \max_{k \in \{1, \dots, K\}} \pi_k f_k(x) = \arg \max_{k \in \{1, \dots, K\}} \delta_k(x).$$

Les fonctions $\delta_1, \dots, \delta_K$ sont-elles toujours affines ? □

Exemple : $p = 2$, $K = 3$, $\pi_1 = \pi_2 = \pi_3 = 1/3$ et ...



Les lignes pointillées (resp. continues) représentent les frontières de décision (resp. frontières de décision estimées).

Exercice 25.

Par analogie au cas $p = 1$, proposer des estimations $\hat{\delta}_{k,d_n}$, $k \in \{1, \dots, K\}$, des scores théoriques inconnus δ_k , $k \in \{1, \dots, K\}$.

- On peut transformer les scores estimés en probabilités conditionnelles estimées :

$$\hat{p}_{k,d_n}(x) = \frac{e^{\hat{\delta}_{k,d_n}(x)}}{\sum_{\ell=1}^K e^{\hat{\delta}_{\ell,d_n}(x)}}, \quad k \in \{1, \dots, K\}. \quad (2.5)$$

Noter l'analogie avec la fonction *softmax* (utilisée en régression logistique multinomiale, en réseaux de neurones, ...).

- L'analyse discriminante gaussienne avec $\Sigma_1 = \dots = \Sigma_K = \Sigma$ est appelée *analyse discriminante linéaire* (pourquoi?).
- L'acronyme anglais est LDA : *linear discriminant analysis*.
- Lorsque les classes sont bien séparées, les estimateurs des paramètres de la régression logistique sont particulièrement instables. L'analyse discriminante linéaire ne souffre pas de ce problème.
- Si n est petit et X est approximativement normalement distribué conditionnellement à Y , l'analyse discriminante linéaire est plus stable que la régression logistique dans tous les cas.
- L'analyse discriminante linéaire peut être également présentée comme une approche géométrique/factorielle cherchant à déterminer des hyperplans qui séparent au mieux les classes.
- Dans ce cadre plus descriptif, elle est parfois nommée *analyse factorielle discriminante* (AFD).

2.4 Autres formes d'analyses discriminantes

- Le point de départ de cette catégorie de méthodes de classification est l'expression (2.1) qui en ensuite combinée avec le classifieur optimal de Bayes pour donner le classifieur théorique (2.2).
- Les versions ajustées/estimées de ce classifieur théorique seront particulièrement performantes si les densités paramétriques choisies pour représenter les densités conditionnelles vraies $f_1, \dots, f_K$ ont été choisies de façon adéquate.

- Choisir des densités gaussiennes ne convient donc évidemment pas dans tous les cas.
- Autres formes d'analyses discriminantes :
 - En gardant des densités gaussiennes pour $f_1, \dots, f_K$ mais en abandonnant la contrainte $\Sigma_1 = \dots = \Sigma_K = \Sigma$, c-à-d, en optant pour une hypothèse d'hétéroscédasticité, on obtient l'analyse discriminante quadratique.
 - Les scores théoriques inconnus sont alors donnés par :

$$\delta_k(x) = -\frac{1}{2}(x - \mu_k)^\top \Sigma_k^{-1}(x - \mu_k) + \log \pi_k - \frac{1}{2} \log |\Sigma_k|, \quad k \in \{1, \dots, K\}.$$

Ils contiennent des termes quadratiques (d'où viennent-ils ?).

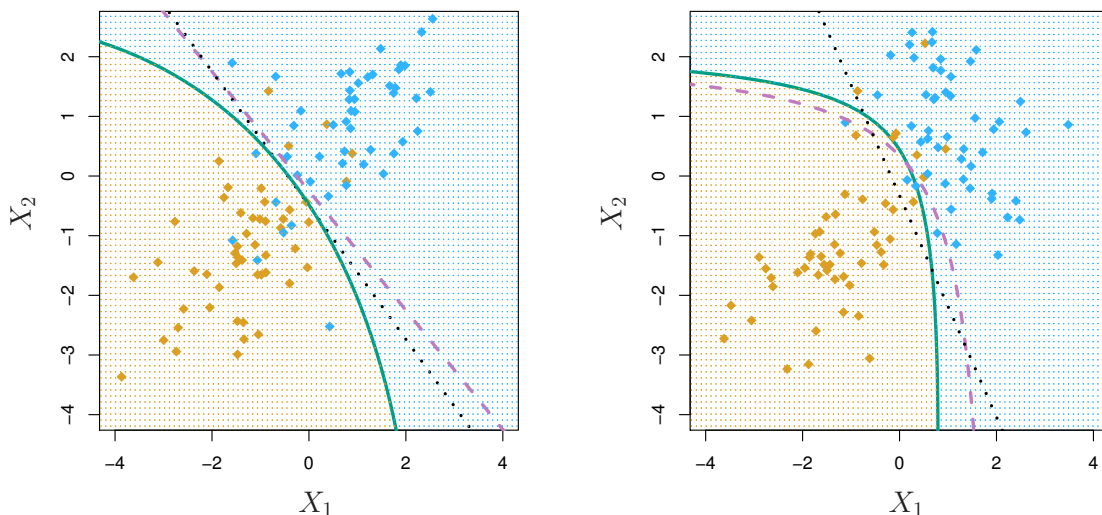
- L'acronyme anglais est QDA : *quadratic discriminant analysis*.
- Si l'on suppose que chaque densité multivariée f_k satisfait :

$$f_k(x) = \prod_{j=1}^p f_{k,j}(x_j), \quad (2.6)$$

où $f_{k,1}, \dots, f_{k,p}$ sont les densités marginales de f_k , on obtient la catégorie des classifieurs *bayésiens naïfs* (*naive Bayes* en anglais).

- Cela revient à supposer que les p variables explicatives sont indépendantes conditionnellement à Y . Dans le cas gaussien, cela revient à supposer que les matrices de covariance Σ_k sont diagonales.
- Il existe de nombreuses autres formes d'analyse discriminante en fonction du choix fait pour estimer f_k : paramétrique, non paramétrique, etc.

Exemple d'analyse discriminante quadratique : $p = 2$, $K = 2$ et ...



Les lignes (LDA) et courbes (QDA) pointillées (resp. continues) représentent les frontières de décision (resp. frontières de décision estimées).

Exercice 26.

1. Écrire une fonction R nommée `rAD()` prenant en entrée un entier n , un réel `prob` dans $(0, 1)$, deux vecteurs $\mu_1, \mu_2 \in \mathbb{R}^p$ et deux matrices de covariances $\Sigma_1, \Sigma_2 \in \mathbb{R}^{p \times p}$, et renvoyant un `data.frame` de colonnes `X1, ..., Xp, y` contenant en ligne n réalisations d'un vecteur aléatoire (X, Y) tel que $Y \in \mathcal{Y} = \{1, 2\}$ avec $\mathbb{P}[Y = 1] = \text{prob}$, $X | Y = 1 \sim N_p(\mu_1, \Sigma_1)$ et $X | Y = 2 \sim N_p(\mu_2, \Sigma_2)$. Pour manipuler la loi normale multivariée, utiliser le package `mvtnorm`.
2. Testez votre fonction avec le code suivant :

```
## Tests
n <- 1000
prob <- 0.5
mu1 <- c(1, 1)
mu2 <- c(-1, -1)
Sigma1 <- matrix(c(2, 0, 0, 2), nrow = 2, ncol = 2)
Sigma2 <- matrix(c(0.5, 0, 0, 0.5), nrow = 2, ncol = 2)
d <- rAD(n = n, prob = prob, mu1 = mu1, mu2 = mu2,
         Sigma1 = Sigma1, Sigma2 = Sigma2)

## LDA et QDA avec l'un des packages historiques
library(MASS)
l <- lda(y ~ ., data = d)
q <- qda(y ~ ., data = d)

## Prédictions sur les données d'apprentissage
pred.y.l <- predict(l, newdata = d)$class
pred.y.q <- predict(q, newdata = d)$class

## Graphiques
par(mfrow = c(1, 3))
plot(d[,1:2], col = d$y, main = "Vraie classe")
plot(d[,1:2], col = pred.y.l, main = "Classe prédite par LDA")
plot(d[,1:2], col = pred.y.q, main = "Classe prédite par QDA")
```

Dans la situation ci-dessus, quel type d'analyse discriminante semble marcher le mieux ? Expliquer pourquoi.

3. Donner du code R permettant de calculer les matrices de confusion des deux modèles sur les données d'apprentissage ainsi que les erreurs d'apprentissage correspondantes.
4. Écrivez votre propre implémentation de l'analyse discriminante quadratique et, dans

le cadre du code de test ci-dessus, comparez ses résultats avec ceux renvoyés par `qda`.

2.5 Données déséquilibrées et solutions dans le cas de la classification binaire

- Afin d’illustrer le problème des données dites “déséquilibrées”, supposons que $K = 2$ et reprenons le cadre de l’exercice 26 mais avec `prob = 0.95` cette fois.
- Cela signifie que la probabilité d’avoir une observation de la classe 1 (resp. 2) est de 0.95 (resp. 0.05).
- Les observations de la classe 1 seront donc très fortement représentées dans les données.
- Nous ne nous intéresserons qu’à l’analyse discriminante linéaire ci-après.
- Le code suivant génère un ensemble d’apprentissage et un ensemble de test en utilisant la fonction `rAD()` précédemment programmée.
- L’ensemble d’apprentissage est utilisé pour ajuster le modèle d’analyse discriminante linéaire.
- L’ensemble de test est utilisé pour évaluer la qualité du modèle via le calcul de la matrice de confusion et de l’erreur de classification.

```
## Pour pouvoir reproduire les résultats
set.seed(111)

## Paramètres pour générer des données
n <- 1000
prob <- 0.95
mu1 <- c(1, 1)
mu2 <- c(-1, -1)
Sigma1 <- matrix(c(2, 0, 0, 2), nrow = 2, ncol = 2)
Sigma2 <- matrix(c(0.5, 0, 0, 0.5), nrow = 2, ncol = 2)

## Ensemble d'apprentissage
app <- rAD(n = n, prob = prob, mu1 = mu1, mu2 = mu2,
          Sigma1 = Sigma1, Sigma2 = Sigma2)
table(app$y)

## Ajustement
library(MASS)
l <- lda(y ~ ., data = app)
```

```
## Ensemble de test
test <- rAD(n = n, prob = prob, mu1 = mu1, mu2 = mu2,
           Sigma1 = Sigma1, Sigma2 = Sigma2)

## Prédiction sur les données de test
pred.y.l <- predict(l, newdata = test)

## Graphiques
par(mfrow = c(1, 2))
plot(test[,1:2], col = test$y, main = "Vraie classe")
plot(test[,1:2], col = pred.y.l$class, main = "Classe prédite par LDA")

## Matrice de confusion et erreur de test associée
mat.conf.l <- table(pred.y.l$class, test$y); mat.conf.l
errClass(mat.conf.l)
```

- Supposons que le cadre applicatif soit celui de la prédiction du risque de non remboursement d’un client d’une banque demandant un crédit.
- Spécifiquement, appartenir à la classe 1 (resp. 2) s’interprète comme “Risque de non remboursement = non” (resp. “Risque de non remboursement = oui”).
- La matrice de confusion sur l’ensemble de test résume alors la situation concernant le risque de non remboursement :

	Vérité	
Prédiction	Non	Oui
Non	947	40
Oui	7	6

- L’erreur de classification sur l’ensemble de test est de $(40 + 7)/1000 = 4.7\%$.
- Sans trop réfléchir, cela semble être une faible erreur de classification, le modèle ne se trompant que dans 4.7% des cas.
- Sans faire de modélisation, en nous basant uniquement sur les proportions des classes dans l’ensemble d’apprentissage ($949/1000 = 94.9\%$ de “non” et $51/1000 = 5.1\%$ de “oui”), nous pourrions considérer le classifieur naïf qui consiste à toujours prédire la classe majoritaire de l’ensemble d’apprentissage.
- En procédant ainsi sur l’ensemble de test, nous serions grosso modo équivalent au modèle ajusté en terme d’erreur de classification !
- Pour comprendre le comportement du modèle ajusté, il faut décomposer l’erreur de classification sur l’ensemble de test en deux erreurs :

- parmi les vrais “non”, le taux de ceux pour lesquels le modèle prédit “oui”, appelé le “taux de faux positifs” (*false positive rate* en anglais – acronyme FPR) ;
- et, parmi les vrais “oui”, le taux de ceux pour lesquels le modèle prédit “non”, appelé le “taux de faux négatifs” (*false negative rate* en anglais – acronyme FNR).
- En partant de la matrice de confusion, on trouve $\text{FPR} = 7/(947 + 7) \approx 0.7\%$ et $\text{FNR} = 40/(40 + 6) \approx 87\%$ (!!!).
- On voit ainsi que le modèle prédit très bien Y pour les clients ne présentant pas de risque mais classifie extrêmement mal les clients auxquels il ne faudrait pas donner de crédit.
- Il est néanmoins possible d’améliorer le modèle dans le contexte de classification binaire considéré.
- Comme nous l’avons vu dans les sections précédentes, pour prédire la classe d’une nouvelle observation pour laquelle $X = x$ à l’aide de l’analyse discriminante, il faut calculer les scores de discrimination $\hat{\delta}_{k,d_n}(x)$, $k \in \{1, 2\}$, et renvoyer comme prédiction de Y le k pour lequel $\hat{\delta}_{k,d_n}(x)$ est le plus grand.
- Nous avons également vu que les scores de discrimination estimés $\hat{\delta}_{k,d_n}$ peuvent être facilement transformés en probabilités conditionnelles estimées $\hat{p}_{k,d_n}$ à l’aide de (2.5).
- Ces probabilités conditionnelles estimées (parfois appelées *posterior probabilities* en anglais) sont par exemple disponibles dans la liste renvoyée par la fonction `predict()` lorsqu’elle est appelée sur l’objet renvoyé par `lda()` :

```
head(pred.y.1$posterior)
```

- Comme ici $K = 2$, nous avons $\hat{p}_{2,d_n}(x) = 1 - \hat{p}_{1,d_n}(x)$ et la règle de classification peut simplement s’écrire comme :

$$\text{SI } \hat{p}_{1,d_n}(x) \geq 0.5 \text{ ALORS } \hat{y} = 1 \text{ SINON } \hat{y} = 2.$$

[Il faut trancher le cas $\hat{p}_{1,d_n}(x) = 0.5$ de façon arbitraire.]

- Pour s’en convaincre sur notre exemple :

```
all(ifelse(pred.y.1$posterior[,1] >= 0.5, 1, 2) == pred.y.1$class)
```

- Pour améliorer le modèle de classification, l’idée est de modifier le seuil de 0.5 dans la règle ci-dessus.
- Nous allons tout d’abord écrire une fonction `R` qui renvoie la matrice de confusion pour un seuil $s \in [0, 1]$ puis ensuite calculer $\text{FPR}(s)$ et $\text{FNR}(s)$, le taux de faux positifs et

le taux de faux négatifs pour un seuil s , et enfin représenter $FPR(s)$ et $FNR(s)$ en fonction de s :

```
## Fonction renvoyant la matrice de confusion pour un seuil s
mcSeuil <- function(s) {
  pred.class <- ifelse(pred.y.l$posterior[,1] >= s, 1, 2)
  table(factor(pred.class, levels = c(1, 2)), factor(test$y, levels = c(1, 2)))
}

## Calculs de FPR(s) et FNR(s) pour s dans {0, 0.01, ..., 0.99, 1}
s <- seq(from = 0, to = 1, by = 0.01)
fpr <- fnr <- numeric(length(s))
for (i in seq_along(s)) {
  mc <- mcSeuil(s = s[i])
  fpr[i] <- mc[2,1] / (mc[1,1] + mc[2,1])
  fnr[i] <- mc[1,2] / (mc[1,2] + mc[2,2])
}

## FPR(s) et FNR(s) en fonction de s
plot(s, fpr, type = "l", xlab = "seuil", ylab = "FPR, FNR")
points(s, fnr, type = "l", col = "red")
```

- Nous voyons par exemple que le taux de faux positifs reste relativement “faible” tant que $s \leq 0.8$. En revanche, pour $s = 0.8$, le taux de faux négatifs n’est plus que de 40% environ (par rapport à 87% environ pour $s = 0.5$).
- Le choix final du seuil dépend évidemment du contexte applicatif.
- Dans le contexte applicatif considéré, les faux positifs ne se verront pas attribuer de crédit alors qu’ils ne présentent pas de risque en réalité.
- Les faux négatifs en revanche se verront attribuer un crédit mais ne seront pas en mesure de le rembourser.
- Les deux types d’erreurs de classification conduisent à une perte financière pour la banque mais la seconde est susceptible d’être plus grave que la première surtout s’il s’agit de crédits impliquant des sommes conséquentes.
- Une façon de résumer la performance d’un classifieur binaire basé par exemple sur l’analyse discriminante indépendamment du choix du seuil s consiste à représenter la courbe paramétrique formée des points $\{(FPR(s), 1 - FNR(s)) : s \in [0, 1]\}$.
- Cette courbe s’appelle la courbe ROC (pour *receiver operating characteristic* en anglais).
- Nous pouvons facilement en afficher une approximation dans R :

```
## Courbe ROC approchée
plot(fpr, 1 - fnr, type = "l", xlab = "FPR", ylab = "TPR = 1 - FNR")
abline(0, 1, lty = 2)
```

L'acronyme TPR correspond à *true positive rate* : parmi les vrais “oui”, le taux de ceux pour lesquels le modèle prédit “oui”, appelé le “taux de vrais positifs”.

Exercice 27.

À quoi ressemblerait la courbe ROC d'un classifieur binaire de type analyse discriminante ne commettant aucune erreur quel que soit le choix du seuil $s \in]0, 1[$? Expliquer. $\square$

- Pour résumer la courbe ROC d'un classifieur binaire de type analyse discriminante, on peut mesurer l'aire sous la courbe ROC, noté AUC (*area under the curve* en anglais).
- Plus AUC est proche de 1, meilleur est le classifieur.
- Autres termes que l'on retrouve dans les logiciels :
 - un synonyme de *true positive rate* (TPR) est *sensitivity* ;
 - on trouve aussi l'acronyme TNR qui correspond à *true negative rate* : parmi les vrais “non”, le taux de ceux pour lesquels le modèle prédit “non”, appelé le “taux de vrais négatifs” ;
 - un synonyme de *true negative rate* (TNR) est *specificity*.

2.6 Le classifieur bayésien naïf gaussien

- Lorsque p est grand, l'analyse discriminante linéaire et l'analyse discriminante quadratique ne donnent pas de bons résultats en général.
- Même si l'hypothèse d'indépendance des p variables explicatives conditionnellement à Y est une hypothèse très forte, le classifieur bayésien naïf gaussien donne souvent de bons résultats lorsque p est grand.
- Les scores théoriques inconnus sont alors donnés par

$$\delta_k(x) = -\frac{1}{2} \sum_{j=1}^p \left[\frac{(x_j - \mu_{k,j})^2}{\sigma_{k,j}^2} + \log \sigma_{k,j}^2 \right] + \log \pi_k, \quad (2.7)$$

où $\Sigma_k = \text{diag}(\sigma_{k,1}^2, \dots, \sigma_{k,p}^2)$.

- À cause de l'indépendance des p variables explicatives conditionnellement à Y , ce modèle peut être facilement étendu au cas où certaines variables explicatives sont qualitatives : si la variable X_j est qualitative, il suffit de remplacer $f_{k,j}$ dans (2.6) par la fonction de masse conditionnelle de X_j sachant que $Y = k$.
- Au fait, pourquoi l'hypothèse d'indépendance conditionnelle ?
 - Il est difficile d'estimer correctement des densités multivariées lorsque p est grand.
 - L'hypothèse d'indépendance des p variables explicatives conditionnellement à Y permet de n'avoir à estimer que des densités univariées pour chaque classe.

2.7 Analyse discriminante avec tidymodels

Nous allons donner des exemples de code R utilisant le package `tidymodels` pour réaliser diverses opérations lors de l'utilisation de méthodes d'analyse discriminante. Nous commençons par générer des données avec la fonction `rAD()` programmée précédemment.

```
## Pour pouvoir reproduire les résultats
set.seed(222)

## Paramètres pour générer des données
n <- 1000
prob <- 0.4
mu1 <- c(1, 1)
mu2 <- c(-1, -1)
Sigma1 <- matrix(c(2, 0, 0, 2), nrow = 2, ncol = 2)
Sigma2 <- matrix(c(0.5, 0, 0, 0.5), nrow = 2, ncol = 2)

## Données disponibles
d <- rAD(n = n, prob = prob, mu1 = mu1, mu2 = mu2,
        Sigma1 = Sigma1, Sigma2 = Sigma2)
```

Nous allons supposer que nous sommes toujours dans le contexte applicatif de la prédiction du risque de non remboursement d'un client d'une banque demandant un crédit. Pour que les interprétations soient plus explicites, nous recodons la variable à expliquer après l'avoir transformée en `factor` :

```
d$y <- factor(d$y)
levels(d$y)
levels(d$y) <- c("oui", "non") # choix arbitraire
levels(d$y)
```

2.7.1 Sans découpage apprentissage/test des données disponibles

Nous supposons maintenant que les données `d` nous ont été fournies et que nous souhaitons estimer un modèle de type analyse discriminante pour prédire la classe `y` à partir de `X1` et de `X2`. Nous commençons “de façon naïve”, sans découper les données disponibles en ensemble d’apprentissage et ensemble de test.

```
library(tibble)
library(tidymodels)
library(discrim) # package supplémentaire interfaçant avec tidymodels
                  # les principales méthodes d'analyse discriminantes

## On transforme le data.frame en tibble pour être "plus tidyverse"
d <- as_tibble(d)

## Le modèle LDA -----
lda_model <- discrim_linear() |> set_engine("MASS") |> set_mode("classification")

## On crée le "workflow" correspondant
## C'est l'un des objets clés : il combine une formule
## et une spécification de modèle
lda_wf <- workflow() |> add_model(lda_model) |> add_formula(y ~ .)

## On ajuste le modèle aux données
lda_fit <- lda_wf |> fit(data = d)

## Prédiction pour l'ensemble d'apprentissage
lda_results <- lda_fit |> predict(new_data = d)

## On rajoute les y vrais au tibble lda_results
lda_results <- lda_results |> add_column(y = d$y)

## On calcule la matrice de confusion
lda_cm <- conf_mat(lda_results, truth = y, estimate = .pred_class)
lda_cm
## puis un ensemble de mesures de qualité du modèle
summary(lda_cm)
## La 1e est accuracy = 1 - erreur de classification
## La 3e est la "sensitivity" = TPR = 1 - FNR
## La 4e est la "specificity" = TNR = 1 - FPR
## Pour la "sensitivity"/"specificity", levels[1] semble
## toujours être interprété comme "oui"/"positif"

## Le modèle QDA -----
qda_model <- discrim_quad() |> set_engine("MASS") |> set_mode("classification")
```

```

## On crée le "workflow" correspondant
qda_wf <- workflow() |> add_model(qda_model) |> add_formula(y ~ .)

## On ajuste le modèle aux données
qda_fit <- qda_wf |> fit(data = d)

## Prédiction pour l'ensemble d'apprentissage
qda_results <- qda_fit |> predict(new_data = d) |> add_column(y = d$y)

## On calcule la matrice de confusion
qda_cm <- conf_mat(qda_results, truth = y, estimate = .pred_class)
qda_cm
summary(qda_cm)

## Comme on pouvait s'y attendre, QDA a une
## erreur d'apprentissage plus faible que LDA

## Courbes ROC et AUC -----
## Pour obtenir automatiquement les courbes ROC, nous avons besoin
## des probabilités conditionnelles estimées
lda_results <-
  lda_fit |> predict(new_data = d, type = "prob") |> add_column(y = d$y)
qda_results <-
  qda_fit |> predict(new_data = d, type = "prob") |> add_column(y = d$y)

## Les courbes ROC + AUC
roc_curve(lda_results, truth = y, .pred_oui) |> autoplot()
roc_auc(lda_results, truth = y, .pred_oui)
roc_curve(qda_results, truth = y, .pred_oui) |> autoplot()
roc_auc(qda_results, truth = y, .pred_oui)
## Sans surprise, QDA est perçu comme meilleur sur l'ensemble d'apprentissage

```

2.7.2 Avec découpage apprentissage/test des données disponibles

Comme nous savons que l'erreur d'apprentissage peut avoir tendance à sous-estimer grossièrement l'erreur de généralisation d'un modèle, nous découpons les données disponibles en ensemble d'apprentissage et ensemble de test préalablement à l'ajustement des modèles. Cela se fait avec des fonctions de haut niveau dans `tidymodels` et donne un code assez compact :

```

set.seed(333)
## Découpage 75% / 25% en "préservant" les proportions des classes
## L'utilisation de "strata" est particulièrement utile dans le cas de
## données déséquilibrées (pas le cas ici)

```

```

split <- initial_split(d, prop = 0.75, strata = y)
## Pour voir les ensembles d'apprentissage et de test:
## app <- split /> training()
## test <- split /> testing()

## Le modèle LDA et le "workflow" correspondant
lda_model <- discrim_linear() |> set_engine("MASS") |> set_mode("classification")
lda_wf <- workflow() |> add_model(lda_model) |> add_formula(y ~ .)

## Ajustement du modèle sur l'ensemble d'apprentissage
lda_fit <- lda_wf |> last_fit(split = split)
## Prédiction sur l'ensemble de test
lda_results <- lda_fit |> collect_predictions()

## Matrice de confusion, courbe ROC et AUC
lda_cm <- conf_mat(lda_results, truth = y, estimate = .pred_class)
summary(lda_cm)
roc_curve(lda_results, truth = y, .pred_oui) |> autoplot()
roc_auc(lda_results, truth = y, .pred_oui)

```

Exercice 28.

Adapter le code précédent pour ajuster un modèle QDA puis un classifieur bayésien naïf gaussien. Des trois modèles ajustés, lequel semble être le meilleur ? Est-ce surprenant ? Expliquer.

2.7.3 Estimation de l'erreur de généralisation par validation croisée répétée

Les mesures de qualités des modèles ajustés ci-dessus dépendent, comme nous le savons, du découpage apprentissage/test considéré. Pour s'affranchir de cet effet et obtenir une meilleure estimation de leurs erreurs de généralisation, il vaut mieux procéder par validation croisée, possiblement répétée plusieurs fois :

```

## Blocs pour la validation croisée répétée
set.seed(444)
folds <- vfold_cv(d, v = 10, repeats = 5)
folds

## Pour LDA
lda_model <- discrim_linear() |> set_engine("MASS") |> set_mode("classification")
lda_wf <- workflow() |> add_model(lda_model) |> add_formula(y ~ .)
lda_fit_rs <- lda_wf |> fit_resamples(folds)
collect_metrics(lda_fit_rs)

```

Exercice 29.

Adapter le code précédent pour évaluer les erreurs de généralisation d'un modèle QDA et d'un classifieur bayésien naïf gaussien. Des trois modèles considérés, lequel semble être le meilleur ? Est-ce une surprise ? Expliquer.

Chapitre 3

Extensions du modèle linéaire

3.1 Rappels sur la régression linéaire multiple

- Nous sommes dans le contexte de la régression fondée sur la perte quadratique ℓ_2 .
- On rappelle que l'objectif (théorique) est de trouver la fonction f qui minimise l'erreur de prédiction espérée $\mathcal{E}_{\ell_2}(f) = \mathbb{E}[\ell_2(Y, f(X))] = \mathbb{E}[(Y - f(X))^2]$.
- Comme nous l'avons vu, parmi toutes les fonctions f possibles, choisir $f_{\ell_2}^*$ définie par $f_{\ell_2}^*(x) = \mathbb{E}[Y \mid X = x]$ minimise $\mathcal{E}_{\ell_2}$. Cette fonction est appelée *fonction de régression*.
- L'hypothèse principale lorsqu'on utilise le modèle de régression linéaire est que la fonction de régression vraie inconnue s'écrit comme :

$$f_{\ell_2}^*(x) = \mathbb{E}[Y \mid X = x] = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p. \quad (3.1)$$

- C'est une hypothèse forte. Même si elle n'est que rarement complètement valide, elle conduit souvent à des modèles ayant à la fois une bonne performance prédictive et qui sont faciles à interpréter.

3.1.1 Les moindres carrés

- L'estimation des paramètres du modèle se fait par minimisation des *moindres carrés*.
- Soit $\hat{\beta} = (\hat{\beta}_0, \dots, \hat{\beta}_p)$ l'estimation du vecteur de paramètres $\beta = (\beta_0, \dots, \beta_p)$ obtenue à partir des données disponibles $d_n = \{(x_i, y_i)\}_{i \in \{1, \dots, n\}}$. On a

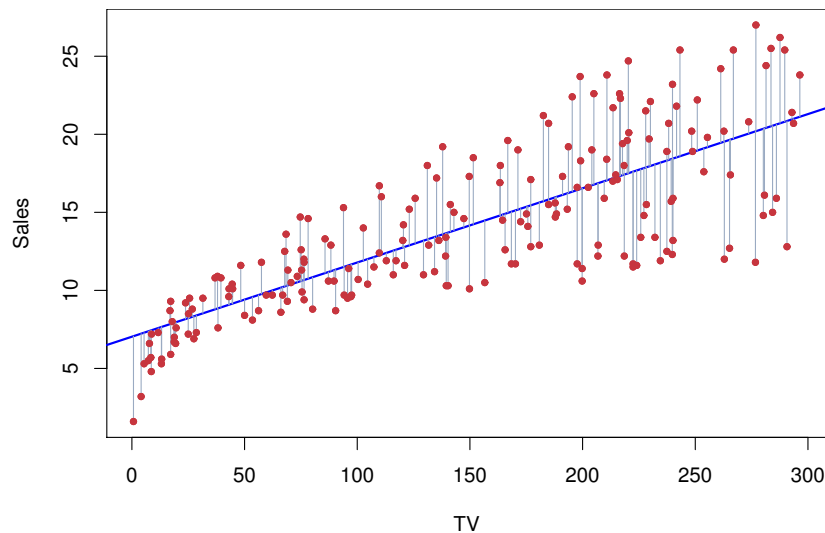
$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \cdots - \beta_p x_{ip})^2.$$

— Soient

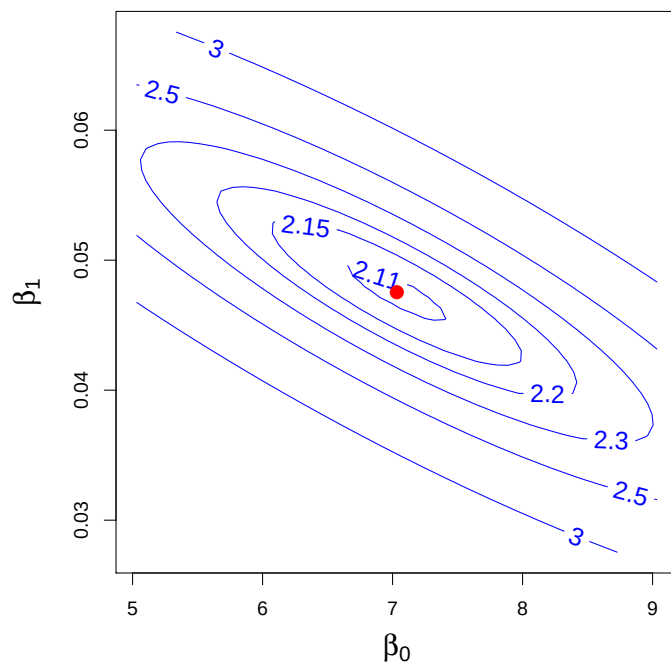
$$\mathbb{Y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \quad \text{et} \quad \mathbb{X} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1p} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \dots & x_{np} \end{bmatrix}.$$

Si $\mathbb{X}$ est de rang $p + 1$, alors $\hat{\beta} = (\mathbb{X}^\top \mathbb{X})^{-1} \mathbb{X}^\top \mathbb{Y}$.

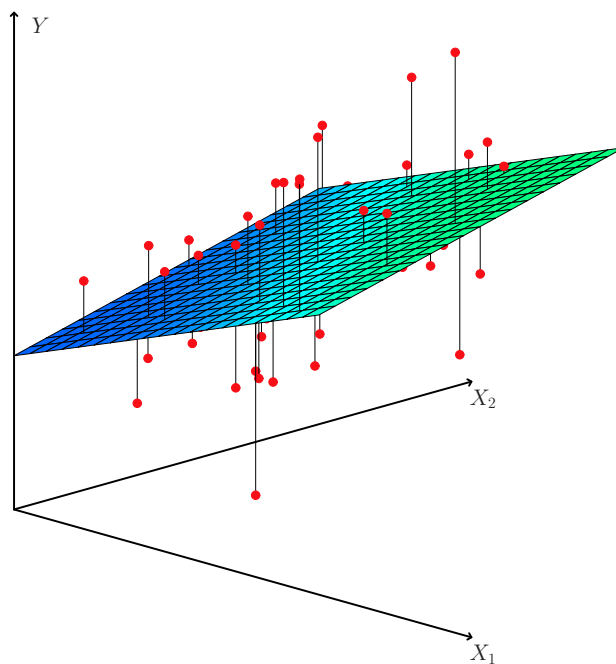
- Soit $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_p x_{ip}$ la prédiction de Y renvoyée par le modèle ajusté lorsque $X = x_i$. Les quantités $e_1 = y_1 - \hat{y}_1, \dots, e_n = y_n - \hat{y}_n$ sont souvent appelées les *résidus*.
- Une propriété importante est que les résidus ont une somme/moyenne nulle.
- **Exemple lorsque $p = 1$** : Le graphique suivant représente la droite des moindres carrés d'équation $y = \hat{\beta}_0 + \hat{\beta}_1 x$ lors de la modélisation par régression linéaire de **sales** en fonction de **TV** pour les données **Advertising** déjà utilisées.



La minimisation des moindres carrés est un problème d'optimisation convexe (voir exercice ci-dessous) :



- **Exemple lorsque $p = 2$:** Dans le cas de deux variables explicatives, le modèle ajusté correspond au plan d'équation $y = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2$:



Exercice 30.

On rappelle que l'on dispose de données $d_n = \{(x_i, y_i)\}_{i \in \{1, \dots, n\}}$. On considère uniquement le cas $p = 1$ à des fins pédagogiques.

1. Soit $\mathcal{X}$ un sous-ensemble convexe de $\mathbb{R}^q$, $q \geq 1$, et soit g une fonction de $\mathcal{X}$ dans $\mathbb{R}$. On rappelle que g est strictement convexe sur $\mathcal{X}$ si et seulement si, pour tout $\lambda \in]0, 1[$ et pour tout $w_1, w_2 \in \mathcal{X}$, $w_1 \neq w_2$,

$$g(\lambda w_1 + (1 - \lambda)w_2) < \lambda g(w_1) + (1 - \lambda)g(w_2).$$

Vérifier que la somme de deux fonctions strictement convexes sur $\mathcal{X}$ est strictement convexe sur $\mathcal{X}$.

2. En utilisant le fait que la fonction $x \mapsto x^2$ est strictement convexe sur $\mathbb{R}$ (argument simple?), vérifier que, pour tout $i \in \{1, \dots, n\}$, $f_i : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par $f_i(b_0, b_1) = (y_i - b_0 - b_1 x_i)^2$ est strictement convexe sur $\mathbb{R}^2$.
3. En déduire que la fonction $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par

$$f(b_0, b_1) = \sum_{i=1}^n f_i(b_0, b_1) = \sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2$$

est strictement convexe sur $\mathbb{R}^2$.

4. En déduire que f a au plus un minimiseur sur $\mathbb{R}^2$ et proposer une approche simple pour le déterminer par le calcul s'il existe.
5. En déduire une expression simple de l'estimation de moindres carrés dans le cas $p = 1$. Montrer en particulier que $\hat{\beta}_1 = s_{xy}/s_x^2$ et $\hat{\beta}_0 = \bar{y}_n - \hat{\beta}_1 \bar{x}_n$, où s_{xy} est la covariance empirique, s_x^2 est la variance empirique des x_i et, $\bar{y}_n$ et $\bar{x}_n$ sont les moyennes empiriques des y_i et des x_i respectivement.

3.1.2 Mesures de qualité de l'ajustement

- On définit RSS (acronyme anglais de *residual sum of squares*) par

$$\text{RSS} = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (3.2)$$

- Deux mesures de la qualité du modèle ajusté sont RSE (acronyme anglais de *residual standard error*) donné par :

$$\text{RSE} = \sqrt{\frac{1}{n-2} \text{RSS}} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n e_i^2} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2},$$

et le R^2 , s'interprétant comme la *proportion de variance expliquée*, donné par :

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}},$$

où $\text{TSS} = \sum_{i=1}^n (y_i - \bar{y}_n)^2$ est l'acronyme anglais de *total sum of squares*.

- Plus RSE est proche de zéro/plus R^2 est proche de 1, plus l'hyperplan représentant le modèle ajusté approche bien les points des données (d'apprentissage).
- Attention, les deux quantités précédentes sont relatives aux données d'apprentissage. Elles peuvent néanmoins être adaptées dans le cadre de l'utilisation d'un ensemble de test.

Exercice 31.

On se place dans le cas $p = 1$.

1. Vérifier que

$$R^2 = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y}_n)^2}{\sum_{i=1}^n (y_i - \bar{y}_n)^2}$$

2. En déduire que R^2 est égale au carré du coefficient de corrélation linéaire calculé à partir de d_n .

□

3.1.3 Pratique de la régression linéaire multiple

- Sous une hypothèse de normalité et d'indépendance des erreurs (indépendance vis-à-vis de X et indépendance sérielle), on connaît la loi (asymptotique) des estimateurs de $\beta_0, \beta_1, \dots, \beta_p$.
- On peut alors estimer des erreurs standards, obtenir des intervalles de confiances pour les β_j et réaliser des tests à la fois sur la nullité des β_j et sur la pertinence globale du modèle.
- Pour illustrer tout ceci, on considère les données **Advertising** déjà utilisées. On cherche à modéliser **sales** en fonction des autres variables à l'aide d'un modèle de régression linéaire multiple :

```
setwd("~/Documents/Ens/MachineLearning/Data/")
adv <- read.csv("Advertising.csv", row.names = 1)
head(adv)
m <- lm(sales ~ ., data = adv)
summary(m)
```

- Sous réserve de normalité et d'indépendance des erreurs, on peut conclure :
 - à l'aide de la dernière ligne de `summary(m)` qui donne la p-valeur du test de Fisher, que le modèle constant est significativement plus mauvais que le modèle complet avec trois variables explicatives ;
 - en regardant la p-valeur relative à `newspaper`, que le modèle sans `newspaper` n'est pas significativement plus mauvais que le modèle complet ;
 - que le R^2 vaut 0.9 environ.

- Cela suggère de considérer le modèle sans `newspaper` :

```
m2 <- lm(sales ~ . - newspaper, data = adv)
summary(m2)
```

- Lorsque l'on pratique la régression linéaire multiple, on est amené à se poser plusieurs questions :
 1. Est-ce qu'au moins une des variables (potentiellement) explicatives $X_1, \dots, X_p$ est utile pour prédire Y ? Comme nous l'avons vu ci-dessus, pour répondre à cette question, on peut utiliser le test de Fisher.
 2. Est-ce que toutes les variables explicatives doivent être gardées dans le modèle ou seulement un sous-ensemble de ces dernières ? C'est le problème de la *sélection de variables*. Avec p variables potentiellement explicatives, il y a 2^p modèles possibles. Lorsque p est grand, il n'est généralement pas possible de considérer tous les modèles possibles et on a recours à des heuristiques :
 - *forward selection* : on part du modèle constant et on ajoute une variable à chaque étape selon un certain critère jusqu'à satisfaire un critère d'arrêt ;
 - *backward selection* : on part du modèle complet et on enlève une variable à chaque étape selon un certain critère jusqu'à satisfaire un critère d'arrêt ;
 - parmi les critères d'arrêt, on retrouve le C_p de Mallow, le critère AIC (acronyme de l'anglais *Akaike Information Criterion*), le critère BIC (acronyme de l'anglais *Bayesian Information Criterion*), le R^2 ajusté et l'utilisation de la validation croisée.

Nous n'aborderons pas les méthodes de sélection de variables *pas-à-pas* dans ce cours car elles reposent dans certains cas sur des hypothèses peu solides. Nous verrons en revanche que la régression linéaire multiple *régularisée* permet de faire de la sélection de variables.

 3. Lorsque $X = x$, quelle est la précision de notre prédiction ? Sous réserve de normalité et d'indépendance des erreurs, il est possible de renvoyer des intervalles de prédiction. Par exemple pour les données `Advertising` :

```
## Un nouveau scénario pour la publicité
new.adv <- data.frame(TV = 200, radio = 40, newspaper = 20)
## Prédiction ponctuelle
predict(m2, newdata = new.adv)
## Intervalle de prédiction
predict(m2, newdata = new.adv, interval = "prediction")
```

- Une autre considération d'ordre pratique est la possibilité d'étendre le modèle de régression linéaire multipliée en incluant des termes d'*interaction* et des termes *non linéaires*.
- Exemple d'inclusion d'un terme d'interaction pour les données Advertising :

```
## Pour plus d'informations sur les formules R, voir par ex. ?formula
m3 <- lm(sales ~ . - newspaper + TV:radio, data = adv)
summary(m3)
```

Le modèle considéré ci-dessus est :

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times (\text{radio} \times \text{TV}) + \epsilon.$$

- Exemple d'inclusion d'un terme quadratique pour les données Auto :

```
setwd("~/Documents/Ens/MachineLearning/Data/")
## Valeur manquante était codée par ?
a <- read.csv("Auto.csv", na.strings = "?")
head(a)
## Tendance quadratique
plot(a$horsepower, a$mpg)
m4 <- lm(mpg ~ horsepower + I(horsepower^2), data = a)
summary(m4)
```

Le modèle considéré ci-dessus est :

$$\text{mpg} = \beta_0 + \beta_1 \times \text{horsepower} + \beta_2 \times \text{horsepower}^2 + \epsilon.$$

- Une dernière considération d'ordre pratique est l'inclusion de variables explicatives qualitatives : ceci se fait simplement en procédant à un *codage disjontif complet* de ces variables.
- Pour illustrer ce type de transformation, on considère les données **Carseats**. Il s'agit de données de ventes de sièges de bébés pour voitures. On cherche à modéliser **Sales** en fonction des autres variables à l'aide d'un modèle de régression linéaire multiple :

```
setwd("~/Documents/Ens/MachineLearning/Data/")
cs <- read.csv("Carseats.csv")
head(cs)
## Transformation des variables qualitatives en "factors"
```

```

cs$ShelveLoc <- factor(cs$ShelveLoc)
cs$Urban <- factor(cs$Urban)
cs$US <- factor(cs$US)
## On rajoute le terme d'interaction \beta_j * Income * Advertising
m5 <- lm(Sales ~ . + Income:Advertising, data = cs)
summary(m5)
contrasts(cs$ShelveLoc)

```

Lors de l'appel à `lm()` ci-dessus, les variables qualitatives à deux modalités `Urban` et `US` ont été automatiquement remplacées par les variables quantitatives `1(Urban == "Yes")` et `1(US == "Yes")`, respectivement. Quant à la variable qualitative à trois modalités `ShelveLoc` (type de position sur les étagères de vente), elle a été automatiquement remplacée par les deux variables quantitatives `1(ShelveLoc == "Good")` et `1(ShelveLoc == "Medium")`.

- Il est également possible d'ajouter des termes d'interaction combinant une variable quantitative et une variable qualitative (voir exercice ci-dessous).

Exercice 32.

Dans le cas où $p = 2$, X_1 est quantitative et X_2 est qualitative, écrire l'équation du modèle lorsque :

1. X_2 a deux modalités "a" et "b" ;
2. X_2 a trois modalités "a", "b" et "c" ;
3. lorsque X_2 a deux modalités et on rajoute un terme d'interaction combinant X_1 et X_2 .

□

Exercice 33.

En utilisant la fonction `model.matrix`, programmer en deux lignes une version simplifiée de la fonction `lm` ne renvoyant que l'estimation $\hat{\beta}$ de β . Tester l'implémentation obtenue en la comparant à `lm` sur certains des exemples vus ci-dessus.

□

Exercice 34.

1. En partant du code précédent, vérifier empiriquement que lorsque l'on multiplie la colonne des données correspondant à X_j par un coefficient $\alpha > 0$, l'estimation correspondante de β_j est divisée par α .
2. Effectuer la même vérification théoriquement.
3. De même vérifier rapidement (théoriquement) que lorsque l'on multiplie la colonne des données correspondant à Y par un coefficient $\alpha > 0$, toutes les estimations sont multipliées par α .

4. Du coup, quel est l'effet de multiplier toutes les valeurs dans d_n par une constante $\alpha > 0$?

□

3.2 Régularisation : régression linéaire ridge et Lasso

- Malgré ses avantages (simplicité, interprétabilité) lorsque l'hypothèse de linéarité est justifiée/plausible, le modèle de régression linéaire fondé sur les moindres carrés souffre “d'une grande variance” lorsque p est grand et “se rapproche” de n .
- Pour illustrer ceci, considérons la situation où $Y = \tilde{\beta}^\top X + \epsilon$, avec $\epsilon \sim N(0, 1)$ indépendante de X , les composantes $X_1, \dots, X_p$ de X indépendantes et toutes $N(0, 1)$, et $\tilde{\beta} = (\beta_1, 0, \dots, 0) \in \mathbb{R}^p$ avec $\beta_1 = 1$. Ainsi, Y ne dépend que de X_1 .

Exercice 35.

1. Relativement au modèle théorique spécifié ci-dessus, écrire une fonction `unAjust` prenant en entrée deux entiers n et p , générant n réalisations de (X, Y) , et renvoyant uniquement l'estimation de β_1 après ajustement du modèle complet $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$ aux données générées par moindres carrés.
 2. Pour $n = 200$ et $p = 1$, à l'aide de la fonction `replicate`, effectuer $M = 1000$ ajustements et représenter la variabilité de $\hat{\beta}_1$ à l'aide d'une boîte à moustaches.
 3. Réaliser la même opération pour $p = 150$ et $p = 175$.
- Les boîtes à moustaches obtenues illustrent bien l'augmentation de la variance de l'estimateur de β_1 lorsque le nombre de variables inutiles augmente.
 - Dans le contexte de la simulation précédente, nous souhaiterions en pratique être capable de sélectionner la variable pertinente X_1 et enlever les variables $X_2, \dots, X_p$ du modèle qui n'apportent que du bruit.
 - Enlever les variables $X_2, \dots, X_p$ du modèle revient évidemment à mettre leurs coefficients (c-à-d, $\beta_2, \dots, \beta_p$) à zéro.

3.2.1 Régression linéaire ridge

- La régression linéaire *ridge* consiste à minimiser les moindres carrés pénalisés par la norme euclidienne (au carré) du vecteur des coefficients.

- Soit $\hat{\beta}^R = (\hat{\beta}_0^R, \dots, \hat{\beta}_p^R)$ l'estimation du vecteur de paramètres $\beta = (\beta_0, \dots, \beta_p)$ obtenue à partir des données disponibles $d_n = \{(x_i, y_i)\}_{i \in \{1, \dots, n\}}$. On a

$$\hat{\beta}^R = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left[\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right],$$

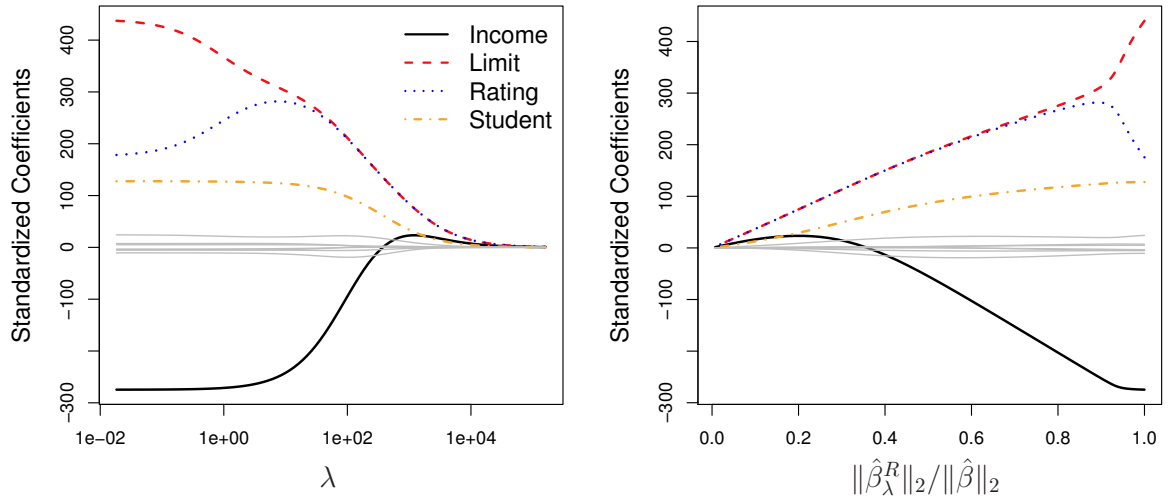
où $\lambda \geq 0$ est un paramètre qui doit être fixé par l'utilisateur.

- Comme pour les moindres carrés, l'objectif de la régression linéaire ridge est de trouver $\hat{\beta}^R$ tel que les points disponibles dans d_n soient “proches” de l'hyperplan d'équation $y = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_i$.
- Le terme de *régularisation* ou de *pénalisation* $\lambda \sum_{j=1}^p \hat{\beta}_j^2$ force néanmoins les éléments de $\hat{\beta}^R$ à ne pas être trop grands.
- Le choix du paramètre λ est donc crucial : d'un côté, pour $\lambda = 0$, $\hat{\beta}^R$ est l'estimation des moindres carrés ; d'un autre côté, plus λ est grand, plus la norme euclidienne de $\hat{\beta}^R$ sera petite.
- En pratique, le choix du paramètre λ se fait typiquement par validation croisée.
- De façon équivalente, l'estimation des coefficients peut s'écrire sous la forme d'un problème d'optimisation sous contraintes :

$$\hat{\beta}^R = \arg \min_{\substack{\beta \in \mathbb{R}^{p+1} \\ \|\beta\|_2 \leq M}} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2,$$

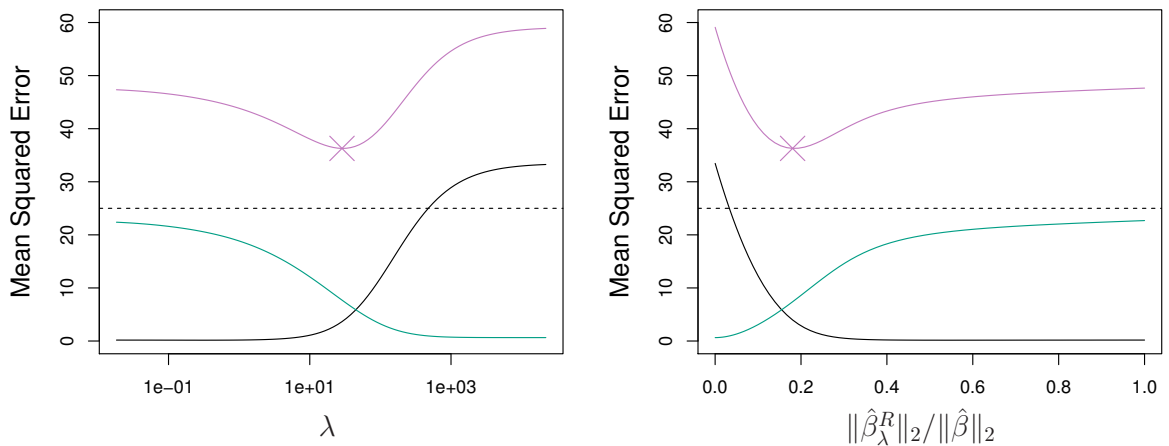
où M joue un rôle similaire à λ et s'interprète comme le “budget” maximum alloué à la norme euclidienne des coefficients.

- **Exemple :** sélection du paramètre λ par validation croisée pour des données portant sur l'utilisation de cartes de crédit (voir données similaires dans la section 3.3) :



Dans le panneau de gauche, chaque courbe représente la valeur d'un coefficient $\hat{\beta}_j^R$ en fonction de λ . Dans le panneau de droite, chaque courbe représente la valeur d'un coefficient $\hat{\beta}_j^R$ en fonction de $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$, où $\hat{\beta}$ est l'estimation des moindres carrés.

- Comme nous l'avons vu dans l'exercice 34, d'éventuelles transformations linéaires des variables affectent les estimations des moindres carrés de façon prévisible : si l'on sait résoudre le problème initial, on sait comment obtenir les estimations pour le problème transformé linéairement. Ce n'est pas le cas des estimations ridge à cause de la présence de la norme euclidienne au carré dans la fonction objectif.
- Il est ainsi important de procéder à une réduction des données avec d'effectuer une régression ridge. En pratique, cela est en général fait automatiquement dans les logiciels.
- Compromis/dilemme biais-variance dans le cas de λ :



Pour illustrer le compromis/dilemme biais-variance mentionné, on considère des simu-

lations faites à partir d'un modèle linéaire avec $p = 45$ variables explicatives ayant toutes un coefficient $\beta_j \neq 0$. La taille des échantillons générés est fixée à $n = 50$. Les courbes violettes, vertes et noires ci-dessus renvoient l'erreur de généralisation (quadratique) estimée, le biais empirique au carré et la variance empirique sur un ensemble test (moyennés sur de nombreuses répétitions) en fonction de λ (gauche) et en fonction de $\|\hat{\beta}^R\|_2/\|\hat{\beta}\|_2$ (droite). Les croix marquent le minimum de l'erreur de généralisation estimée moyennée, représentant un “compromis biais/variance”. Revoir aussi la section 1.2.5.

- Comme on peut le voir sur la figure donnant l'évolution des coefficients $\hat{\beta}_j^R$ en fonction de λ pour les données sur les cartes de crédit, la régression ridge fait tendre les coefficients $\hat{\beta}_j^R$ vers 0 **de façon continue** lorsque λ tend vers $+\infty$.
- En conséquence, même lorsque λ est grand, aucun des coefficients $\hat{\beta}_j^R$ n'est exactement égale à zéro.
- La régression ridge ne permet donc pas vraiment la sélection de variables.

3.2.2 Régression linéaire Lasso

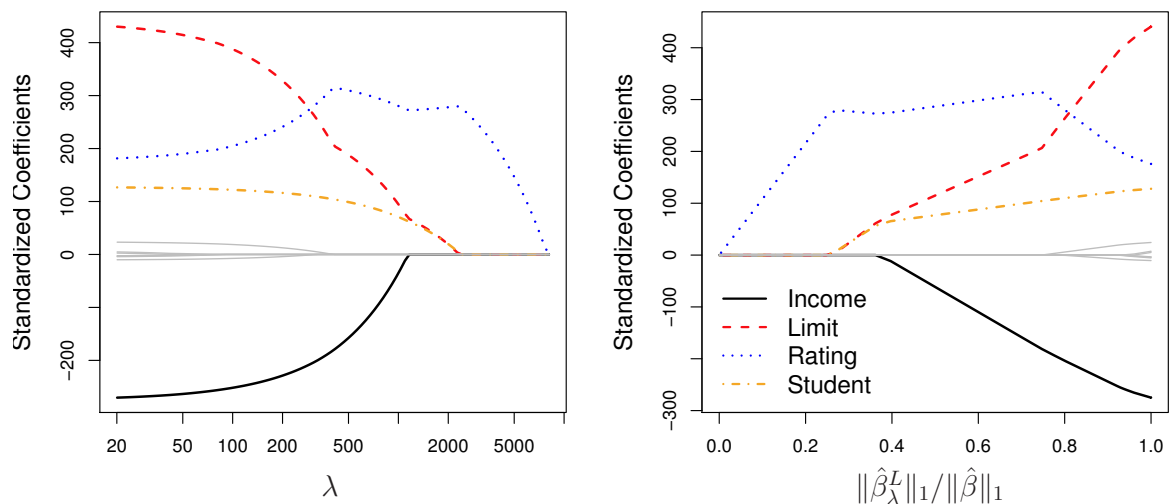
- L'approche Lasso est une alternative relativement récente à l'approche ridge qui a l'avantage de potentiellement réaliser une sélection de variables.
- Soit $\hat{\beta}^L = (\hat{\beta}_0^L, \dots, \hat{\beta}_p^L)$ l'estimation du vecteur de paramètres $\beta = (\beta_0, \dots, \beta_p)$ obtenue à partir des données disponibles $d_n = \{(x_i, y_i)\}_{i \in \{1, \dots, n\}}$. On a

$$\hat{\beta}^L = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left[\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right],$$

où $\lambda \geq 0$ est un paramètre qui doit être fixé par l'utilisateur.

- En d'autres termes, la régression linéaire Lasso est l'analogue de la régression linéaire ridge avec une pénalité fondée sur la norme L_1 au lieu de la norme L_2 .
- Ainsi, comme pour la régression ridge, la régression Lasso avec $\lambda > 0$ “réduit” la norme des coefficients estimés par rapport aux estimations des moindres carrés.
- Néanmoins, contrairement à ridge, pour λ suffisamment grand, l'approche Lasso a la propriété de forcer certains des coefficients estimés à être exactement égaux à zéro.
- Cela revient à une sélection de variables.
- On dit que l'approche Lasso conduit potentiellement à des modèles *sparses* (la traduction française de “sparse” est “clairsemés”), c-à-d, des modèles avec peu de variables.

- Comme dans le cas de la régression ridge, le choix du paramètre λ est crucial et se fait typiquement par validation croisée.
- **Exemple** : sélection du paramètre λ par validation croisée pour les données sur les cartes de crédit :

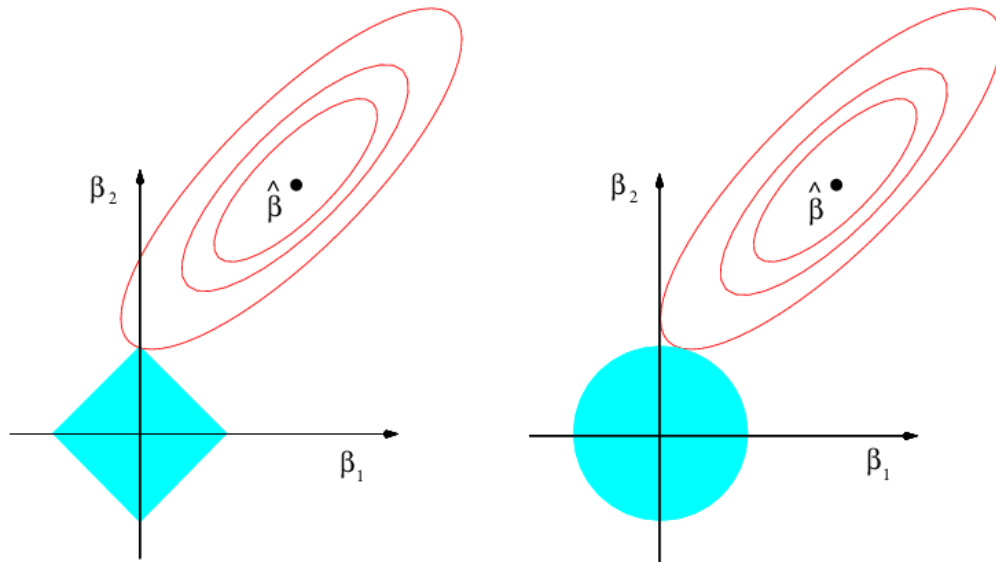


- Comme pour la régression ridge, l'estimation des coefficients peut s'écrire sous la forme d'un problème d'optimisation sous contraintes :

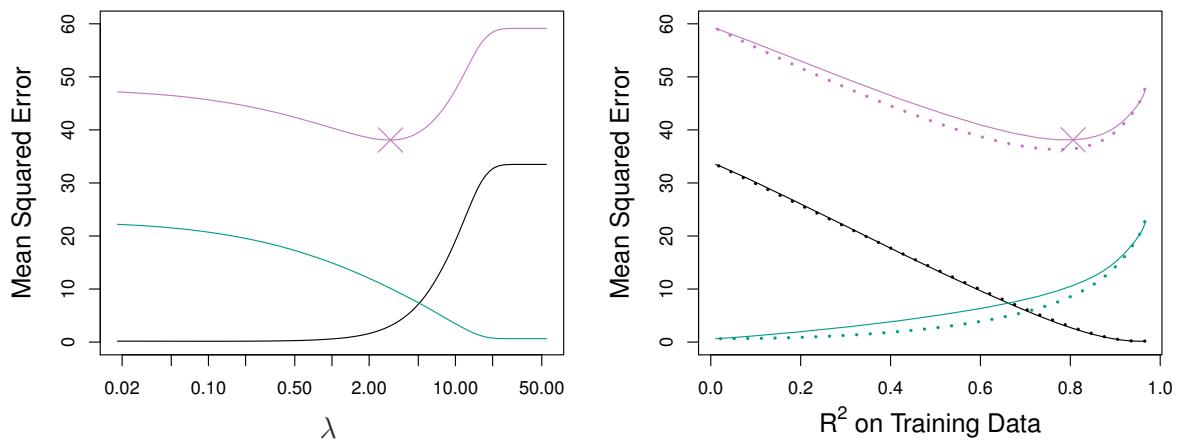
$$\hat{\beta}^L = \arg \min_{\substack{\beta \in \mathbb{R}^{p+1} \\ \|\beta\|_1 \leq M}} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2,$$

où M joue un rôle similaire à λ et s'interprète comme le “budget” maximum alloué à la norme L_1 des coefficients.

- Cette formulation permet de mieux comprendre pourquoi l'approche Lasso conduit à mettre des estimations à zéro lorsque λ augmente :



— Compromis/dilemme biais-variance et première comparaison avec la régression ridge :

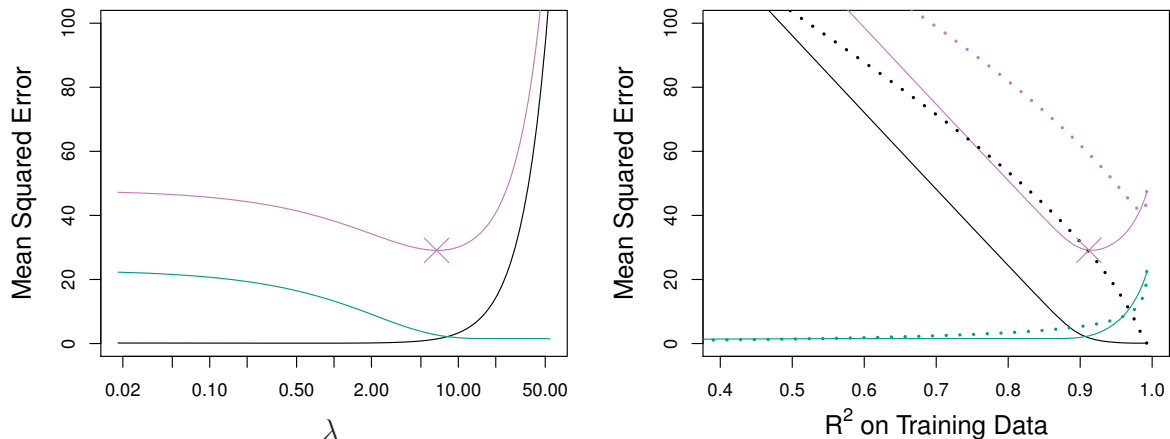


Pour illustrer le compromis/dilemme biais-variance dans le cas Lasso, on considère comme précédemment des simulations faites à partir d'un modèle linéaire avec $p = 45$ variables explicatives ayant toutes un coefficient $\beta_j \neq 0$. La taille des échantillons générés est fixée à $n = 50$. Dans le panneau de gauche, les courbes violette, verte et noire ci-dessus renvoient l'erreur de généralisation (quadratique) estimée, le biais empirique au carré et la variance empirique sur un ensemble test (moyennés sur de nombreuses répétitions) en fonction de λ . Le panneau de droite compare ces mêmes quantités avec celles obtenues pour la régression rigde (courbes en pointillés). Les modèles sont ordonnés en fonction de leur R^2 sur les données d'apprentissage.

Exercice 36.

Relativement au panneau de droite de la figure précédente, expliquer intuitivement pourquoi cela n'aurait pas de sens d'ordonner les modèles en fonction de λ . Aussi, quel est la nature du meilleur modèle en terme d'erreur de généralisation (quadratique) estimée (moyennée sur plusieurs répétitions) ? $\square$

- Comparaison avec la régression ridge dans le cas où le modèle vrai est *sparse* :



Le contexte est le même que précédemment excepté que seules deux variables parmi les $p = 45$ variables ont des coefficients non nuls (c-à-d, expliquent Y). Le panneau de gauche montre que le meilleur compromis biais/variance est obtenu pour une valeur de λ supérieure (par rapport à la situation précédente). Aussi, d'après le panneau de droite, cette fois-ci la régression Lasso est bien meilleure que la régression ridge.

- Les deux exemples précédents illustrent le fait qu'aucune des deux techniques de régularisation n'est uniformément meilleure que l'autre.
- En général, on peut s'attendre à ce que la régression Lasso conduise à un meilleur modèle lorsque Y dépend d'un nombre relativement faible de X_j parmi les p variables potentiellement explicatives.
- En revanche, si le modèle vrai est *dense* (par opposition à *sparse*), on peut s'attendre à ce que ce soit la régression ridge qui conduise à un meilleur modèle.
- Cette information n'est malheureusement jamais disponible dans le cas de problèmes de régression réels.
- L'utilisation de techniques comme la validation croisée peut néanmoins être utilisée pour déterminer laquelle des deux approches conduit au meilleur modèle de régression linéaire.

3.3 Exemples R avec le package glmnet

3.3.1 Données simulées

Nous allons d'abord reproduire certaines des simulations présentées précédemment. Le premier objectif est de programmer une fonction pour générer des données. Le modèle est le suivant :

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_q X_q + \epsilon,$$

avec $\beta_0 = \beta_1 = \cdots = \beta_q = 1$, $X_1, \dots, X_p, \epsilon$ $p + 1$ variables aléatoires $N(0, 1)$ indépendantes et $q \leq p$.

Exercice 37.

Programmer une fonction `rRegLin` prenant en entrée q , n (avec 50 comme valeur par défaut) et p (avec 45 comme valeur par défaut) et renvoyant n réalisations de (X, Y) lorsque le modèle ci-dessous est satisfait. Ces réalisations seront renvoyées sous forme de `list` composée d'une matrice `x` de dimension $n \times p$ contenant les réalisations de X en ligne et un vecteur `y` de longueur n contenant les réalisations correspondantes de Y . $\square$

En prenant q "petit", nous serons dans le cas *sparse* et la régression Lasso devrait donner les meilleurs résultats. En prenant $q = p$, nous serons dans le cas *dense* et la régression ridge devrait donner les meilleurs résultats. À l'aide du package `glmnet`, nous commençons par ajuster un grand nombre de modèles ridge et Lasso pour de nombreuses valeurs de λ sans utiliser un ensemble test ou la validation croisée :

```
## Chargement du package glmnet (co-écrit par les "inventeurs" du Lasso)
library(glmnet)

## Données d'apprentissage "dense"
set.seed(111)
app45 <- rRegLin(q = 45)

## Régression ridge (alpha = 0)
rr <- glmnet(x = app45$x, y = app45$y, alpha = 0)
## Régression Lasso (alpha = 1)
rl <- glmnet(x = app45$x, y = app45$y, alpha = 1)

## Note: alpha dans ]0,1[ conduit à un "mélange" ridge/Lasso
## appelé ElasticNet

## Quelques graphiques
par(mfrow = c(2, 2))
## Modèles ordonnés selon la valeur de lambda
## Attention, lambda n'a pas la même signification dans les deux graphes
```



```
plot(rr, xvar = "lambda", label = TRUE)
plot(rl, xvar = "lambda", label = TRUE)
## Modèles ordonnés selon le  $R^2$  (coincide avec "deviance")
plot(rr, xvar = "dev", label = TRUE)
plot(rl, xvar = "dev", label = TRUE)
```

Exercice 38.

Produire des graphiques similaires dans le cas “sparse” avec $q = 2$. □

Pour trouver le “meilleur” modèle en terme d’erreur de généralisation, on va utiliser la validation croisée, d’abord dans le cas “dense” puis dans le cas “sparse” :

```
## Vecteur de valeurs de lambda et graine aléatoire
lambda.val <- exp(seq(from = -3, to = 4, by = 0.1))
set.seed(123)
## Validation croisée pour la régression ridge
vc.rr <- cv.glmnet(x = app45$x, y = app45$y, alpha = 0, lambda = lambda.val)
## Validation croisée pour la régression Lasso
vc.rl <- cv.glmnet(x = app45$x, y = app45$y, alpha = 1, lambda = lambda.val)
par(mfrow = c(1, 2))
## Modèles ordonnés selon log(lambda)
## Attention, lambda n'a pas la même signification dans les deux graphes
plot(vc.rr)
plot(vc.rl)
## Valeurs "optimales" selon la règle d'un écart-type
lambda.rr <- vc.rr$lambda.1se
lambda.rl <- vc.rl$lambda.1se

## Des modèles "finaux"
rr.final <- glmnet(x = app45$x, y = app45$y, alpha = 0, lambda = lambda.rr)
rl.final <- glmnet(x = app45$x, y = app45$y, alpha = 1, lambda = lambda.rl)
## Et leurs coefficients
coef(rr.final)
coef(rl.final)
## Pour utiliser les modèles, on peut travailler, comme d'habitude,
## avec la fonction predict(). Par exemple:
predict(rr.final, newx = app45$x) # à comparer avec app45$y
## Vérification
max(abs(coef(rr.final)[1] + app45$x %*% coef(rr.final)[-1] - predict(rr.final, newx = app45$x)))
```

Exercice 39.

Produire des graphiques et des calculs similaires dans le cas “sparse” avec $q = 2$. Confirmer que, comme on pouvait s’y attendre, la régression ridge (resp. Lasso) donne les meilleurs résultats dans le cas “dense” (resp. “sparse”). □

3.3.2 Données Credit

Le contexte de l'exemple est celui de l'utilisation de cartes de crédit (en France, la plupart des cartes bleues sont des cartes à débit immédiat). L'objectif est d'expliquer la variable Balance à l'aide des autres variables¹.

```
## Chargement des données
setwd("~/Documents/Ens/MachineLearning/Data/")
cr <- read.csv("Credit.csv")
cr$Own <- factor(cr$Own)
cr$Student <- factor(cr$Student)
cr$Married <- factor(cr$Married)
cr$Region <- factor(cr$Region)
summary(cr)

## Comme les données contiennent des variables qualitatives
## une façon de réaliser leur codage disjontif complet
## consiste à utiliser la fonction model.matrix()
## et à supprimer la première colonne
x <- model.matrix(Balance ~ ., data = cr)[,-1]

## Vecteur de valeurs de lambda et graine aléatoire
lambda.val <- exp(seq(from = -2, to = 4, by = 0.1))
set.seed(123)

## Validation croisée pour la régression ridge
vc.rr <- cv.glmnet(x = x, y = cr$Balance, alpha = 0, lambda = lambda.val)
## Validation croisée pour la régression Lasso
vc.rl <- cv.glmnet(x = x, y = cr$Balance, alpha = 1, lambda = lambda.val)
par(mfrow = c(1, 2))
## Modèles ordonnés selon log(lambda)
## Attention, lambda n'a pas la même signification dans les deux graphes
plot(vc.rr)
plot(vc.rl)
## Valeurs "optimales" selon la règle d'un écart-type
lambda.rr <- vc.rr$lambda.1se
lambda.rl <- vc.rl$lambda.1se
```

En regardant `vc.rr$cvm` et `vc.rl$cvm`, on voit que les deux modèles “optimaux” sont très proches en terme d’erreur de généralisation. L’approche Lasso conduit néanmoins à un modèle avec uniquement 6 variables explicatives (contre 11 pour la régression ridge). Il reste à réajuster les modèles avec les λ “optimaux” :

```
## Les modèles finaux
rr.final <- glmnet(x = x, y = cr$Balance, alpha = 0, lambda = lambda.rr)
```

1. Il ne s’agit pas des mêmes données que celles mentionnées dans les sections précédentes.

```

rl.final <- glmnet(x = x, y = cr$Balance, alpha = 1, lambda = lambda.rl)
## Et leurs coefficients
coef(rr.final)
coef(rl.final)
## Vérification
max(abs(coef(rr.final)[1] + x %*% coef(rr.final)[-1] - predict(rr.final, newx = x)))

```

Exercice 40.

Charger les données `Hitters` disponibles sur `Elearn`. Il s'agit de données collectées en 1986-1987 sur 322 joueurs de Baseball. L'objectif est de prédire la variable `Salary`. En adaptant le code précédent, proposer un modèle ridge et un modèle Lasso pour prédire `Salary`. Lequel des deux modèles semble préférable? □

3.4 Exemple R avec le package `tidymodels`

Exercice 41.

Le code ci-après réalise une version de la modélisation demandée dans l'exercice 40 en utilisant `tidymodels`. Exécuter ce code et commenter chaque "bloc de lignes" en expliquant ce qu'il réalise. Adapter ensuite ce code afin de réaliser une modélisation ridge et Lasso pour les données `Credit` utilisées ci-dessus.

```

library(tidyverse)
library(tidymodels)

setwd("~/Documents/Ens/MachineLearning/Data/")
h <- read_csv("Hitters.csv")
head(h)
H <- h |> filter(!is.na(Salary))
H

## Ridge

ridge_spec <- linear_reg(mixture = 0, penalty = 0)|>
  set_mode("regression")|>
  set_engine("glmnet")

ridge_fit <- fit(ridge_spec, Salary ~ ., data = H)

tidy(ridge_fit)
tidy(ridge_fit, penalty = 11498)

```

```

tidy(ridge_fit, penalty = 705)

ridge_fit |> autoplot()

predict(ridge_fit, new_data = H)
predict(ridge_fit, new_data = H, penalty = 500)

set.seed(654)
H_split <- initial_split(H, strata = "Salary")
H_train <- training(H_split)
H_test <- testing(H_split)

H_fold <- vfold_cv(H_train, v = 10)

ridge_recipe <-
  recipe(formula = Salary ~ ., data = H_train)|>
  step_novel(all_nominal_predictors())|>
  step_dummy(all_nominal_predictors())|>
  step_zv(all_predictors())|>
  step_normalize(all_predictors())

ridge_spec <-
  linear_reg(penalty = tune(), mixture = 0)|>
  set_mode("regression")|>
  set_engine("glmnet")

ridge_workflow <- workflow()|>
  add_recipe(ridge_recipe)|>
  add_model(ridge_spec)

penalty_grid <- grid_regular(penalty(range = c(-5, 5)), levels = 50)
penalty_grid

tune_res <- tune_grid(
  ridge_workflow,
  resamples = H_fold,
  grid = penalty_grid
)
tune_res
autoplot(tune_res)
collect_metrics(tune_res)

best_penalty <- select_best(tune_res, metric = "rsq")
best_penalty

```

```

ridge_final <- finalize_workflow(ridge_workflow, best_penalty)
ridge_final_fit <- fit(ridge_final, data = H_train)
augment(ridge_final_fit, new_data = H_test)|>
  rsq(truth = Salary, estimate = .pred)

## Lasso

lasso_recipe <-
  recipe(formula = Salary ~ ., data = H_train)|>
  step_novel(all_nominal_predictors())|>
  step_dummy(all_nominal_predictors())|>
  step_zv(all_predictors())|>
  step_normalize(all_predictors())

lasso_spec <-
  linear_reg(penalty = tune(), mixture = 1)|>
  set_mode("regression")|>
  set_engine("glmnet")

lasso_workflow <- workflow()|>
  add_recipe(lasso_recipe)|>
  add_model(lasso_spec)

penalty_grid <- grid_regular(penalty(range = c(-2, 2)), levels = 50)

tune_res <- tune_grid(
  lasso_workflow,
  resamples = H_fold,
  grid = penalty_grid
)
autoplot(tune_res)

best_penalty <- select_best(tune_res, metric = "rsq")

lasso_final <- finalize_workflow(lasso_workflow, best_penalty)
lasso_final_fit <- fit(lasso_final, data = H_train)

augment(lasso_final_fit, new_data = H_test)|>
  rsq(truth = Salary, estimate = .pred)
lasso_final_fit|>
  # pull_workflow_fit()|>
  tidy()

```

□

Chapitre 4

Arbres de décision, bagging, forêts aléatoires et boosting

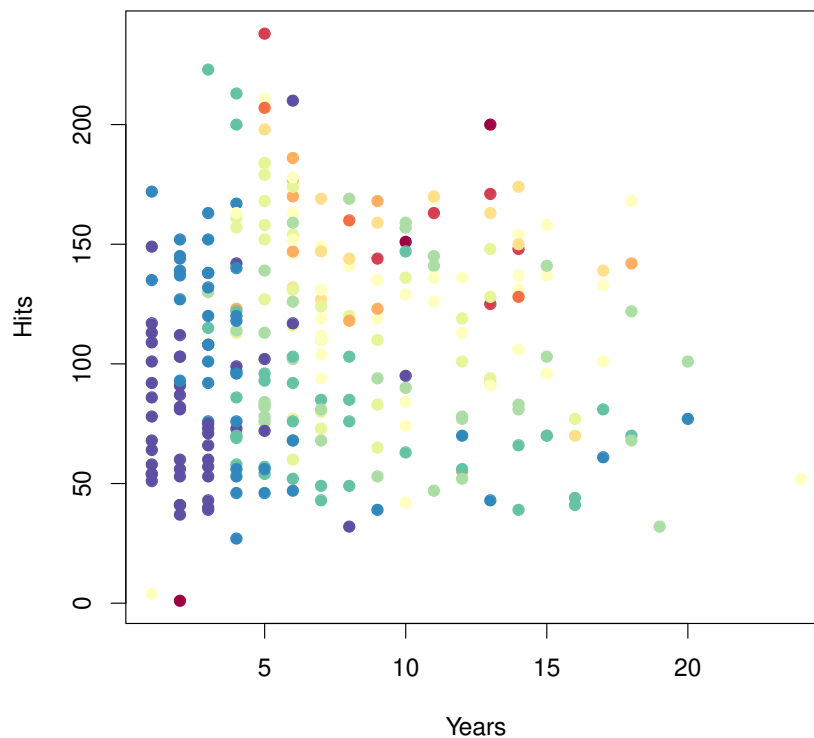
- Nous allons maintenant parler des méthodes d'apprentissage supervisées (régression et classification) à base d'*arbres de décision*.
- Ces méthodes reposent sur la segmentation/le partitionnement de l'espace des variables explicatives en un ensemble de régions de formes simples.
- Comme les segmentations considérées peuvent être résumées sous forme d'arbres, ces méthodes sont connues sous le nom d'*arbres de décision*.

Avantages et inconvénients :

- Les méthodes à base d'arbres sont simples et faciles à interpréter.
- En revanche, elles ne sont typiquement pas compétitives en terme de performance prédictives par rapport aux meilleurs approches pour l'apprentissage supervisé.
- Pour cette raison, nous présenterons également les méthodes connexes de *bagging*, *forêts aléatoires* et *boosting*. Ces dernières consistent à ajuster de nombreux arbres qui sont ensuite combinés afin d'obtenir une prédiction agrégée.
- En combinant de nombreux arbres, il est souvent possible d'améliorer la performance prédictive finale de façon spectaculaire (mais au détriment de l'interprétabilité du modèle final).

4.1 Bases des arbres de décision dans le cas de la régression

- Les arbres de décision peuvent être utilisés pour les problèmes de régression et pour les problèmes de classification.
- Nous allons commencer par les problèmes de régression.
- Afin d'illustrer les principes des arbres de décision, nous considérons un jeu de données concernant le (logarithme du) salaire de joueurs de baseball professionnels. Le graphique ci-dessous représente le salaire ("faible" : bleu, vert ; élevé : jaune, rouge) en fonction du nombre d'années d'ancienneté dans les ligues professionnelles (abscisse) et du nombre de coups de batte de la saison précédente (ordonnée) :

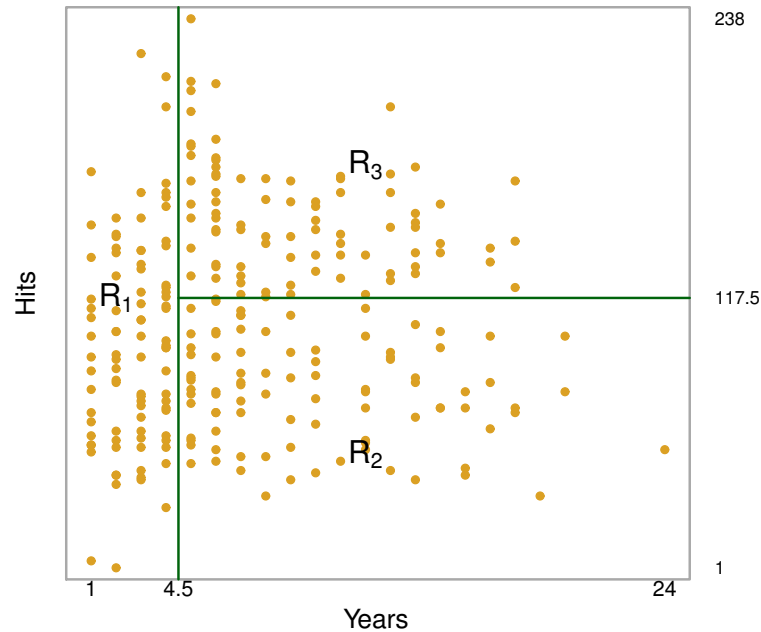


- Une segmentation possible peut alors être résumée à l'aide de l'arbre (au sens informatique du terme) suivant :



Interprétation :

- À un noeud *interne* donné de l'arbre, le label (de la forme $X_j < t_k$) décrit la branche de gauche émanant de cette division de l'espace des variables explicatives. La branche de droite correspond à $X_j \geq t_k$.
- Par exemple, la première division des données (à la racine de l'arbre – en haut) donne deux sous-ensembles. La branche de gauche correspond aux observations telles que $Years < 4.5$, et la branche de droite à celles satisfaisant $Years \geq 4.5$.
- L'arbre a deux noeuds *internes* (aussi appelés *tests*) et trois noeud *terminaux* (également appelés *feuilles*). Le nombre au niveau de chaque feuille correspond à la moyenne de la variable à expliquer pour toutes les observations qui satisfont les tests menant à la feuille, c-à-d, à la moyenne de la variable à expliquer pour toutes les observations qui sont dans la région décrite par les divisions successives de l'espace des variables explicatives.
- Au final, l'arbre partitionne/segmente les joueurs en trois régions de l'espace des variables explicatives : $R_1 = \{X : Years < 4.5\}$, $R_2 = \{X : Years \geq 4.5 \text{ et } Hits < 117.5\}$ et $R_3 = \{X : Years \geq 4.5 \text{ et } Hits \geq 117.5\}$:



- *Years* apparaît comme la variable explicative la plus importante pour déterminer le salaire, et les joueurs moins expérimentés gagnent en général moins que les joueurs plus expérimentés.
- Pour un joueur moins expérimenté, le nombre de coups de batte de la saison précédente n'a apparemment pas (ou très peu) d'influence sur son salaire.
- En revanche, pour un joueur professionnel depuis plus de 5 ans environ, le nombre de coups de batte de la saison précédente affecte son salaire : plus il est élevé, plus le salaire a tendance à être élevé.
- Tout cela est certainement une sur-simplification de la réalité mais, comparé à la plupart des modèles de régression, le résultat de la méthode est facile à représenter, à interpréter et à expliquer.

Principes de construction généraux d'un arbre de décision :

1. L'espace des variables explicatives $X_1, \dots, X_p$ (c-à-d, $\mathbb{R}^p$) est divisé/partitionné en J régions disjointes $R_1, R_2, \dots, R_J$.
2. Pour chaque (nouvelle) observation se retrouvant dans la région R_j , la prédiction sera simplement la moyenne des y_i parmi les x_i de l'ensemble d'apprentissage se trouvant dans R_j .

Plus de détails :

- D'un point de vue théorique, les régions R_j pourraient être de n'importe quelle forme.

- En pratique, dans le cas de la méthode CART (acronyme anglais de *Classification and Regression Trees*) que nous présentons dans ce cours, le choix est d’avoir des hyper-rectangles (plus rigoureusement, des *orthants*) afin de simplifier la construction et l’interprétation du modèle final.
- Un objectif idéal serait de trouver, pour un J fixé, une partition de $\mathbb{R}^p$ en hyper-rectangles $R_1, R_2, \dots, R_J$ tels que

$$\sum_{j=1}^J \sum_{i: x_i \in R_j} (y_i - \hat{y}_{R_j})^2 \quad (4.1)$$

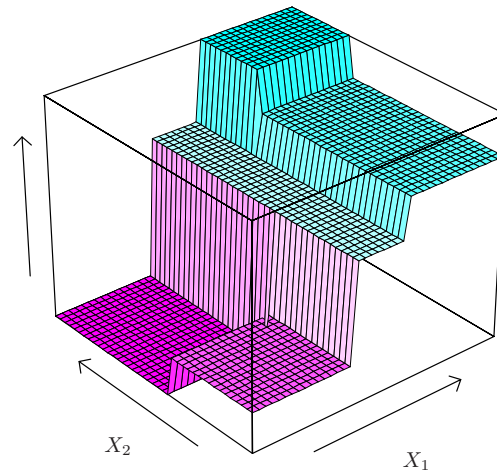
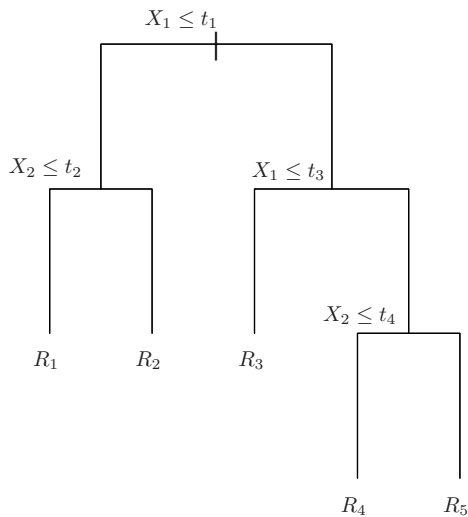
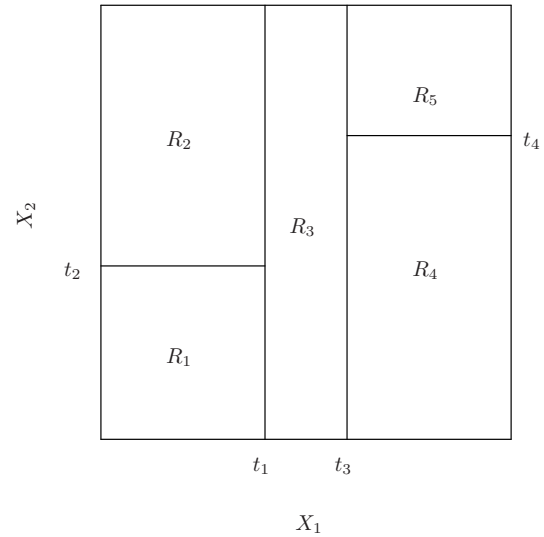
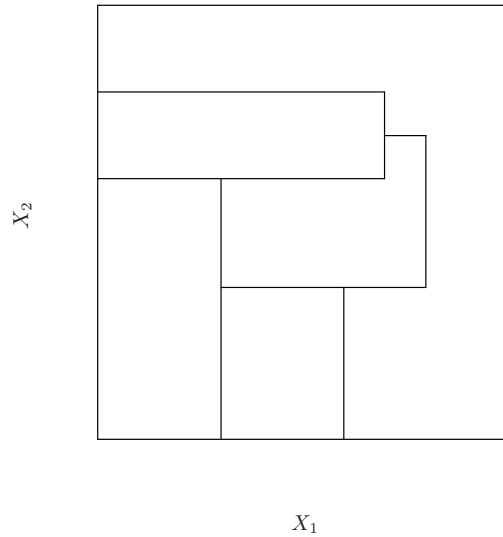
soit minimale, où $\hat{y}_{R_j}$ est la moyenne des y_i parmi les x_i de l’ensemble d’apprentissage se trouvant dans R_j .

- Malheureusement, il n’est pas possible/très difficile en pratique de considérer chaque partition pertinente possible de l’espace des variables explicatives en J hyper-rectangles.
- Pour cette raison, les approches sont généralement fondées sur des heuristiques (algorithmes *gloutons*).
- Pour rappel, un algorithme glouton (*greedy algorithm* en anglais) est un algorithme qui suit le principe de réaliser, à chaque étape, un choix optimal “localement” sans pour autant que l’on puisse s’assurer que la solution finale soit “globalement” optimale.
- L’approche de CART est connue sous le nom de *partitionnement récursif binaire* (*recursive binary splitting* en anglais).
- C’est une approche “descendante” car elle commence à la racine de l’arbre et partitionne ensuite successivement l’espace des variables explicatives en deux (chaque division correspond à un test avec deux branches).
- Elle est gloutonne car à chaque étape de la construction, la “meilleure” division binaire est faite mais cette dernière ne tient pas compte de/ne garantit pas la qualité globale de l’arbre final.
- L’algorithme commence par sélectionner une variable explicative X_j et un seuil s tels que le partitionnement de l’espace des variables explicatives en $\{(x_1, \dots, x_p) \in \mathbb{R}^p : x_j < s\}$ et $\{(x_1, \dots, x_p) \in \mathbb{R}^p : x_j \geq s\}$ conduise à la plus grande réduction possible des moindres carrés donnés dans (4.1).
- Cette idée est ensuite répétée récursivement sur les régions obtenues. On passe ainsi de deux régions, à trois, puis à quatre, etc, jusqu’à ce qu’un critère d’arrêt soit satisfait (par exemple, jusqu’à qu’aucune région ne contiennent plus de 5 $x_i \in \mathbb{R}^p$ de l’ensemble d’apprentissage).

Prédictions :

- La prédiction de Y pour une nouvelle observation telle que $X = x$ se fait en trouvant R_j telle que $x \in R_j$ et en renvoyant la moyenne des y_i de l'ensemble d'apprentissage tels que $x_i \in R_j$.

Exemple : avec $p = 2$ et $J = 5$:



- Diagramme en haut à gauche : type de partition impossible avec CART.
- Diagramme en haut à droite : partition composée d'hyper-rectangles à la CART.

- Graphe en bas à gauche : représentation de la partition en haut à droite sous forme d'arbre.
- Graphe en bas à droite : graphe du modèle ajusté ; c'est une fonction (de prévision de $\mathbb{R}^2$ dans $\mathbb{R}$) constante par morceaux.

Exercice 42.

On considère le jeu de données artificiel généré par le code R suivant :

```
set.seed(456)
## Taille du jeu de données
n <- 100
## Génération des observations
z1 <- rbinom(n, size = 1, prob = 0.5)
x1 <- z1 * rnorm(n, mean = -1) + (1 - z1) * rnorm(n, mean = 3)
z2 <- rbinom(n, size = 1, prob = 0.5)
x2 <- z2 * rnorm(n, mean = -2) + (1 - z2) * rnorm(n, mean = 2)
y <- z1 * z2 * rnorm(n, sd = 0.3) + z1 * (1 - z2) * rnorm(n, mean = 1, sd = 0.3) +
  (1 - z1) * (1 - z2) * rnorm(n, mean = 1, sd = 0.3) +
  (1 - z1) * z2 * rnorm(n, mean = 2, sd = 0.3)
## Jeu de données généré
d <- data.frame(x1, x2, y)
```

1. Expliquer ce que fait le code ci-dessus.
2. Soit $s \in \mathbb{R}$. On considère $R_1 = \{(x_1, x_2) \in \mathbb{R}^2 : x_1 < s\}$ et $R_2 = \{(x_1, x_2) \in \mathbb{R}^2 : x_1 \geq s\}$. Écrire une fonction R qui prend en entrée un seuil s et renvoie la valeur de (4.1) pour $J = 2$.
3. “Vectoriser” la fonction précédente à l’aide de la fonction R `Vectorize`.
4. A l’aide de la fonction `curve` et de la fonction vectorisée, représenter la valeur de (4.1) en fonction de s pour $s \in [-5, 5]$. Donner du code R permettant d’obtenir une approximation de la valeur de s minimisant (4.1).
5. En généralisant le code précédent, construire à partir de `d` un arbre de régression de profondeur 2 permettant de prédire y .

□

Élagage :

- L’algorithme précédent peut conduire à du sur-apprentissage si le processus de construction ne s’arrête pas suffisamment tôt (pourquoi au fait ?).

- Un arbre correspondant à un partitionnement en moins de régions conduit souvent à un estimateur avec une variance plus faible (au prix d'un biais un peu plus élevé), et ainsi à un modèle final avec une interprétation plus facile.
- La profondeur/le nombre de divisions d'un arbre contrôle ainsi directement la complexité/flexibilité de cette classe de modèles.
- On pourrait ainsi imaginer qu'une bonne solution serait de continuer de partitionner seulement tant que les moindres carrés de la forme (4.1) décroissent de façon significative. Cette stratégie donnerait clairement des arbres moins profonds. Elle s'avère néanmoins "trop locale" : une division qui semble peu intéressante en terme de moindres carrés (et qui conduirait donc à arrêter la croissance de l'arbre) à une étape donnée pourrait être suivie par une division qui améliorerait les moindres carrés de façon substantielle.
- Une meilleure approche consiste à construire un arbre T_0 très profond et ensuite à l'élaguer pour obtenir un sous-arbre donnant de meilleures performances.
- Ceci fait partie de l'approche CART et est appelé *cost complexity pruning* ou *weakest link pruning*.
- Soit T un sous-arbre de T_0 . Notons cela $T \subset T_0$ et soit $|T|$ le nombre de noeuds terminaux/feuilles de T . Il y a typiquement plusieurs sous-arbres de T_0 avec le même nombre de feuilles.
- Soit α un paramètre réel positif. Notons alors T_α un sous-arbre de T_0 tel que

$$T_\alpha = \arg \min_{T \subset T_0} \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|, \quad (4.2)$$

où R_m est l'hyper-rectangle correspondant au m -ième noeud terminal de l'arbre considéré T avec $|T|$ feuilles et $\hat{y}_{R_m}$ est la moyenne des y_i de l'ensemble d'apprentissage pour lesquels $x_i \in R_m$.

- Remarque : dans la formulation précédente, nous supposons implicitement que la fonction de T minimisée dans (4.2) a un unique minimum pour chaque valeur de α . Ceci n'est clairement pas toujours vrai mais ne pose pas de problèmes en pratique.
- La paramètre α contrôle un compromis entre la complexité d'un sous-arbre (sa profondeur $|T|$) et son ajustement aux données d'apprentissage.
- Lorsque $\alpha = 0$, T_α est clairement égal à T_0 . En revanche, plus le paramètre α sera grand, plus le prix à payer pour avoir de nombreuses feuilles sera élevé, et donc plus T_α aura tendance à avoir moins de feuilles que T_0 .
- La fonction minimisée dans (4.2) n'est pas sans rappeler des critères similaires mini-

misés dans le cadre de la régression Lasso/ridge.

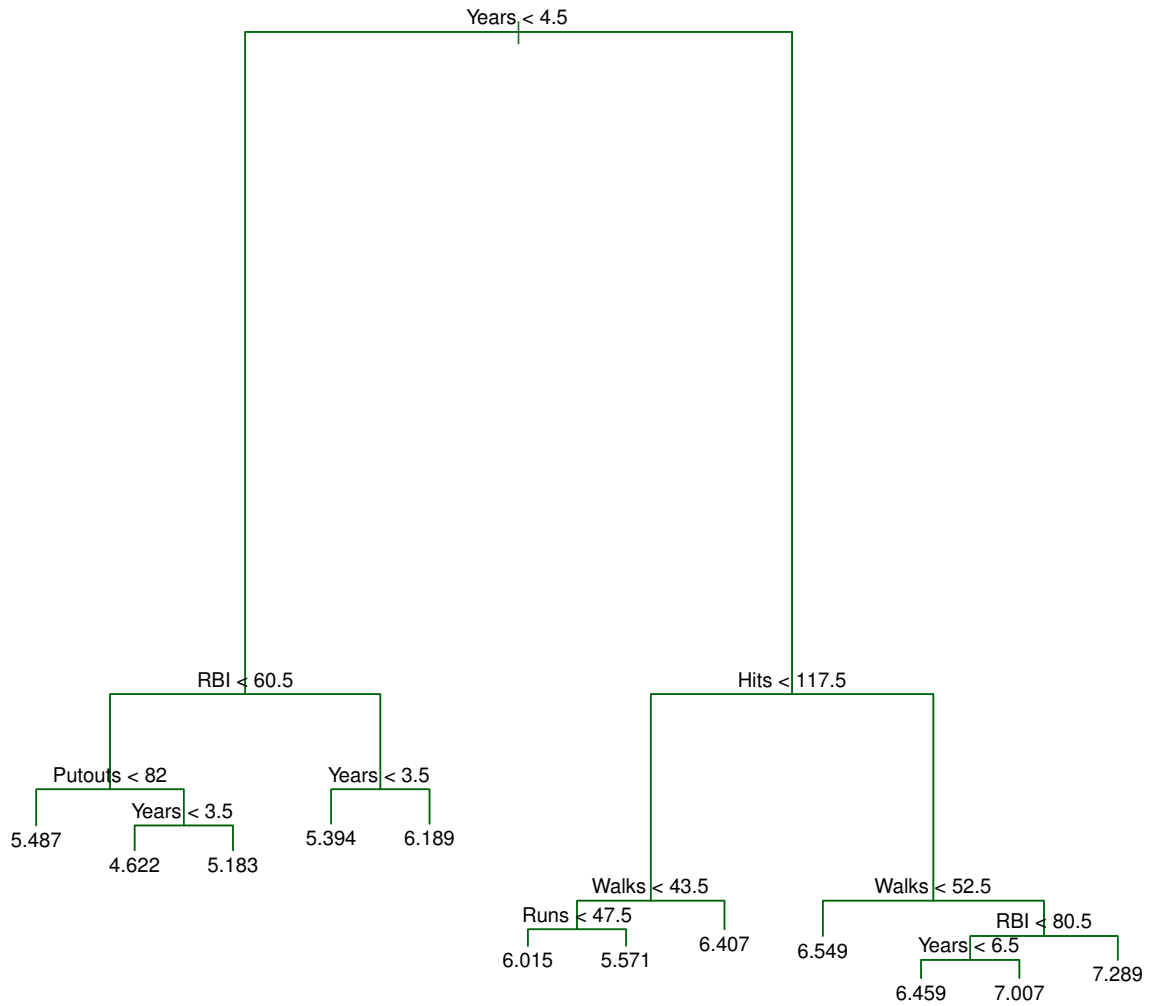
- En variant α , on obtient ainsi une suite de sous-arbres dont la profondeur passe de $|T_0|$ à 1. Il est possible de vérifier que chaque sous-arbre de la suite s’obtient en enlevant un test de l’arbre le précédant dans la suite.
- La valeur “optimale” de α , notée $\hat{\alpha}$, est choisie par validation croisée et $T_{\hat{\alpha}}$ est l’arbre renvoyé.
- Le processus de construction complet est résumé ci-après.

Les principales étapes de l’algorithme de CART :

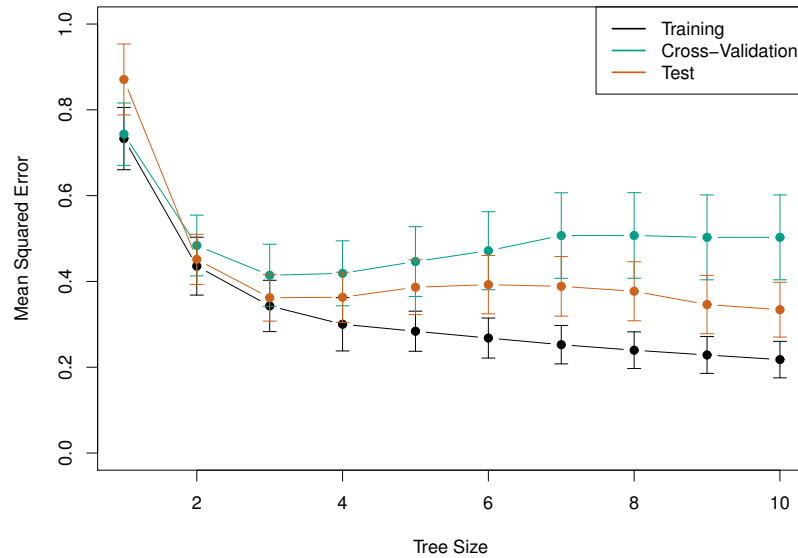
1. Construction d’un arbre profond à l’aide de toutes les données d’apprentissage en n’arrêtant le partitionnement que lorsque qu’il ne reste qu’un faible nombre d’observations (paramètre fixé par l’utilisateur) dans chaque région “terminale” (c-à-d, associée à une feuille).
2. Application du *cost complexity pruning* à l’arbre précédent pour obtenir une suite de “meilleurs” sous-arbres, chacun indexé par α .
3. Utilisation de la validation croisée avec K -blocs pour choisir α . Pour $k = 1, \dots, K$:
 - (a) Répéter les étapes 1 et 2 sur $K - 1$ blocs (en excluant le bloc k).
 - (b) Évaluer les erreurs de généralisation sur le k -ième bloc pour chaque sous-arbre indexé par α .Moyenner les erreurs sur k et choisir pour $\hat{\alpha}$ le α minimisant l’erreur moyenne.
4. Renvoyer le sous-arbre de l’étape 2 correspondant à $\hat{\alpha}$.

Retour sur les données relatives au baseball

- Les données ont été divisées en un ensemble d’apprentissage de 132 observations et un ensemble test de 131 observations.
- Un arbre profond a été ajusté aux données d’apprentissage. Il est représenté ci-après :



- Une suite de sous-arbres (indexés par α) a ensuite été obtenue à partir de l'arbre profond. Les erreurs d'apprentissage (courbe noire) et de test (courbe rouge) des sous-arbres sont données ci-dessous :



- Enfin, la validation croisée avec $K = 6$ a été utilisée sur l'ensemble d'apprentissage pour estimer l'erreur de généralisation de chaque sous-arbre en fonction de α . Les résultats sont donnés par la courbe verte. Nous voyons que les courbes rouge et verte sont raisonnablement proches. L'arbre élagué optimal (avec trois feuilles) est donné ci-dessous :



4.2 Arbres de régression avec le package rpart

Nous commençons par illustrer la mise en œuvre de la méthode CART dans R avec le package `rpart`. Nous considérons le jeu de données `Hitters` déjà utilisé dans le chapitre précédent :

```
library(rpart)
library(rpart.plot) # pour une belle représentation des arbres
```



```

setwd("~/Documents/Ens/MachineLearning/Data/")
h <- read.csv("Hitters.csv")
head(h)
h <- na.omit(h)

## Un premier arbre de régression (method = "anova") pour prédire Salary
ar1 <- rpart(Salary ~ ., data = h, method = "anova")
rpart.plot(ar1)

## Prédiction sur l'ensemble d'apprentissage
pred <- predict(ar1, newdata = h)
## Erreur RMSE sur l'ensemble d'apprentissage
sqrt(mean((pred - h$Salary)^2))

```

Exercice 43.

Interpréter l'arbre affiché à l'aide de `rpart.plot`. □

En réalité, chaque appel à la fonction `rpart` construit une suite de “meilleurs” sous-arbres emboîtés indexés par un paramètre appelé `cp` jouant un rôle similaire au paramètre α dans l'algorithme de CART. La performance de chacun de ces arbres est évaluée par validation croisée avec 10 blocs :

```

## Suite de ``meilleurs'' sous-arbres emboîtés
printcp(ar1)
## CP: paramètre de complexité; plus CP est faible,
## plus l'arbre a de divisions/plus il est profond
## nsplit: nombre de divisions de l'arbre
## xerror: estimation de l'erreur de généralisation
## calculée par validation croisée avec 10 blocs
## xstd: erreur standard associée à l'estimation précédente
plotcp(ar1)

## Quelques arbres correspondant à différentes valeurs de cp
par(mfrow = c(1, 3))
rpart.plot(ar1)
rpart.plot(prune(ar1, cp = 0.02))
rpart.plot(prune(ar1, cp = 0.1))

## Lors de l'appel à rpart, la valeur par défaut de cp est 0.01
## et celle de minsplit est 20
## Voir ?rpart.control pour la signification de minsplit
## Construisons un arbre plus profond
set.seed(321)
ar2 <- rpart(Salary ~ ., data = h, method = "anova", cp = 0.001, minsplit = 2)

```

```

par(mfrow = c(1, 3))
rpart.plot(ar2)
rpart.plot(prune(ar2, cp = 0.01))
rpart.plot(prune(ar2, cp = 0.1))

## Nouvelle suite de sous-arbres emboîtés
printcp(ar2)
par(mfrow = c(1, 1))
plotcp(ar2)

## Sélection de la valeur de cp qui réalise le minimum
cp <- ar2$cptable[, "CP"]
xerror <- ar2$cptable[, "xerror"]
which(xerror == min(xerror))[1]
min.cp <- cp[which(xerror == min(xerror))[1]]
## En général, on peut également utiliser la règle d'un écart-type

## Une possibilité d'arbre final
ar.final <- prune(ar2, cp = min.cp)
rpart.plot(ar.final)

## Prédiction sur l'ensemble d'apprentissage
pred <- predict(ar.final, newdata = h)
## Erreur RMSE sur l'ensemble d'apprentissage
sqrt(mean((pred - h$Salary)^2))

```

4.3 Arbres de décision pour la classification

- En classification, l'objectif est de prédire une variable qualitative (potentiellement codée numériquement).
- Avec un arbre de décision pour la classification, la prédiction de Y pour une nouvelle observation $X = x \in R_j$ est la classe majoritaire dans R_j pour les observations de l'ensemble d'apprentissage.
- Comme en régression, les arbres CART sont construits par partitionnement récursif binaire de l'espace des variables explicatives.
- On pourrait s'attendre à ce qu'il suffisse de remplacer les moindres carrés dans l'algorithme de construction par le taux de mauvaises classifications (lié à la perte 0/1).
- Le taux de mauvaises classifications pour une région R_m est simplement la proportion d'observations de l'ensemble d'apprentissage avec $x_i \in R_m$ qui n'appartiennent pas à

la classe majoritaire, c-à-d,

$$1 - \max_{k \in \{1, \dots, K\}} \hat{p}_{mk}$$

où $\hat{p}_{mk}$ représente la proportion d'observations dans la région R_m de la classe k .

- Ce critère s'avère être sous-optimal et en pratique d'autres mesures sont préférables.
- L'*indice de Gini* de la région R_m est défini par

$$G_m = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

et mesure le *degré d'uniformité* en terme de classes de la région R_m . En particulier, la valeur de G_m sera proche de zéro si un $\hat{p}_{mk}$ est proche de 1 (les autres étant donc proches de 0) et sera maximale si tous les $\hat{p}_{mk}$ sont égaux à $1/K$.

- Ainsi, dans un certain sens, l'indice de Gini G_m permet de mesurer la pureté d'une feuille potentielle correspondant à la région R_m , une faible valeur indiquant que la région ne contient quasiment que des observations de la même classe.
- Une mesure similaire est l'*entropie de Shannon*, appelée parfois *entropie croisée* (*cross-entropy* en anglais) :

$$H_m = - \sum_{i=1}^K \hat{p}_{mk} \log_2 \hat{p}_{mk}.$$

Exercice 44.

Vérifier rapidement que :

- la fonction f définie par $f(x) = -x \log_2 x - (1-x) \log_2 (1-x)$ peut être considérée comme bien définie sur $[0, 1]$, atteint son minimum (qui vaut 0) pour $x = 0$ et $x = 1$ et atteint son maximum (qui vaut 1) pour $x = 1/2$;
- la fonction g définie par $g(x) = 2x(1-x)$ pour $x \in [0, 1]$ atteint son minimum (qui vaut 0) pour $x = 0$ et $x = 1$ et atteint son maximum (qui vaut 1/2) pour $x = 1/2$;
- ces deux fonctions sont symétriques par rapport à $x = 1/2$.

Supposer maintenant que $K = 2$ et, pour une région donnée R_m , exprimer G_m et H_m en fonction de f et g . En quoi cela permet-il d'interpréter G_m et H_m en tant que mesures d'uniformité/de pureté de la région R_m .

Retour sur les principes de construction :

- On commence par calculer, pour chaque variable X_j et chaque seuil candidat s , le *gain entropique* ou *gain d'impureté* :

$$\Gamma(X_j, s) = G_0 - (\hat{\pi}_1 G_1 + \hat{\pi}_2 G_2), \quad (4.3)$$

où G_0 est l'indice de Gini de $R_0 = \mathbb{R}^p$, G_1 (resp. G_2) est l'indice de Gini de la région $R_1 = \{(x_1, \dots, x_p) \in \mathbb{R}^p : x_j < s\}$ (resp. $R_2 = \{(x_1, \dots, x_p) \in \mathbb{R}^p : x_j \geq s\}$), et $\hat{\pi}_1$ (resp. $\hat{\pi}_2$) est la proportion d'observations de l'ensemble d'apprentissage dans la région R_1 (resp. R_2).

- Le premier partitionnement se fait pour la variable X_j et le seuil s pour lesquels $\Gamma(X_j, s)$ est maximal.
- Si l'on souhaite travailler avec l'entropie de Shannon, il suffit de remplacer G_i par H_i dans les calculs précédents.
- Le même principe est ensuite utilisé récursivement sur les régions (R_1 et R_2 après la première étape) obtenues jusqu'à obtenir des régions ne contenant (quasiment) que des observations de la même classe.

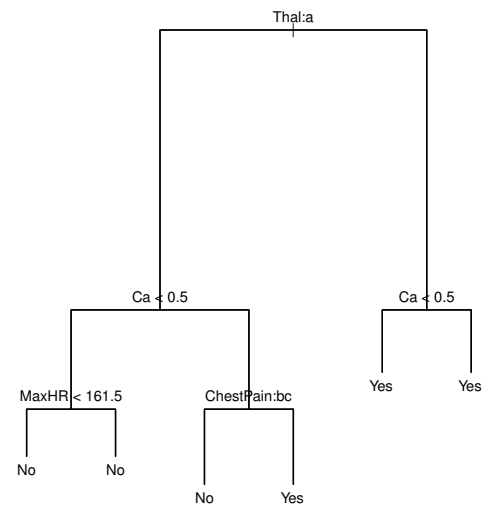
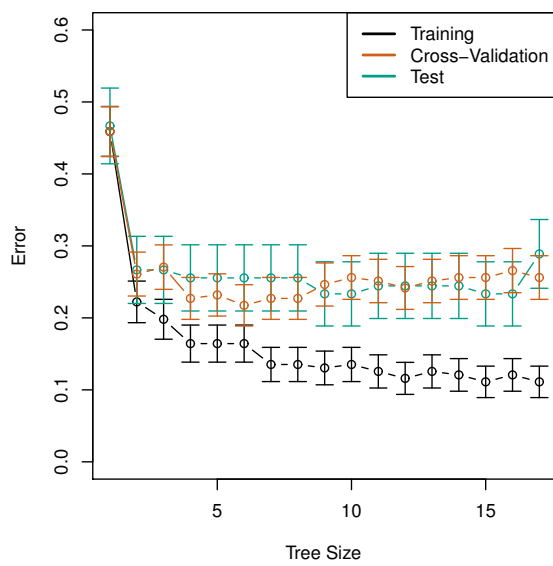
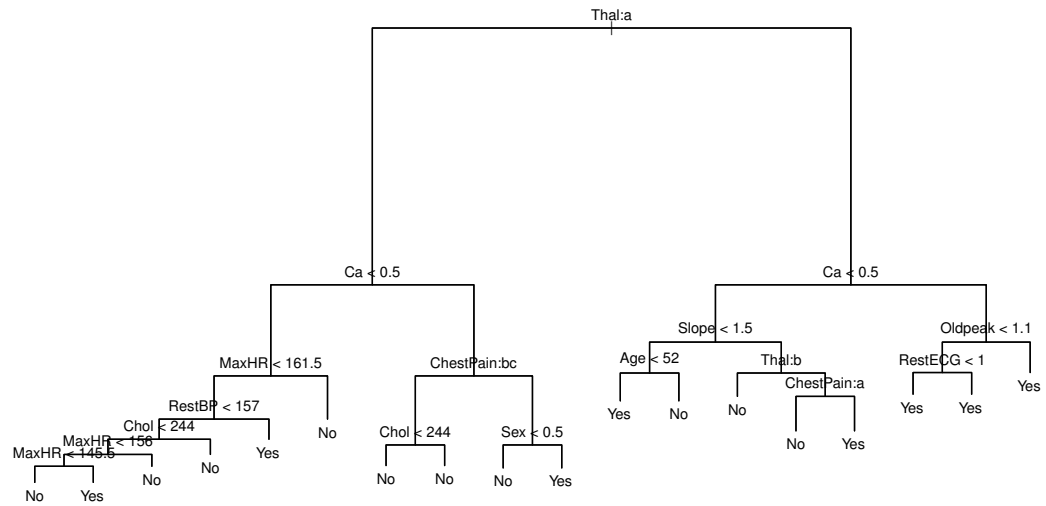
Exercice 45.

1. Expliquer pourquoi (4.3) peut être interprété comme un gain entropique / d'impureté.
2. Adapter de la façon la plus immédiate possible le code donné au début de l'exercice 42 afin de générer des données simples pour la classification (avec $K = 2$).
3. Programmer (de façon rapide et sous-optimale) les principes de construction précédents et les appliquer pour construire un arbre de classification pour prédire y de profondeur 2.

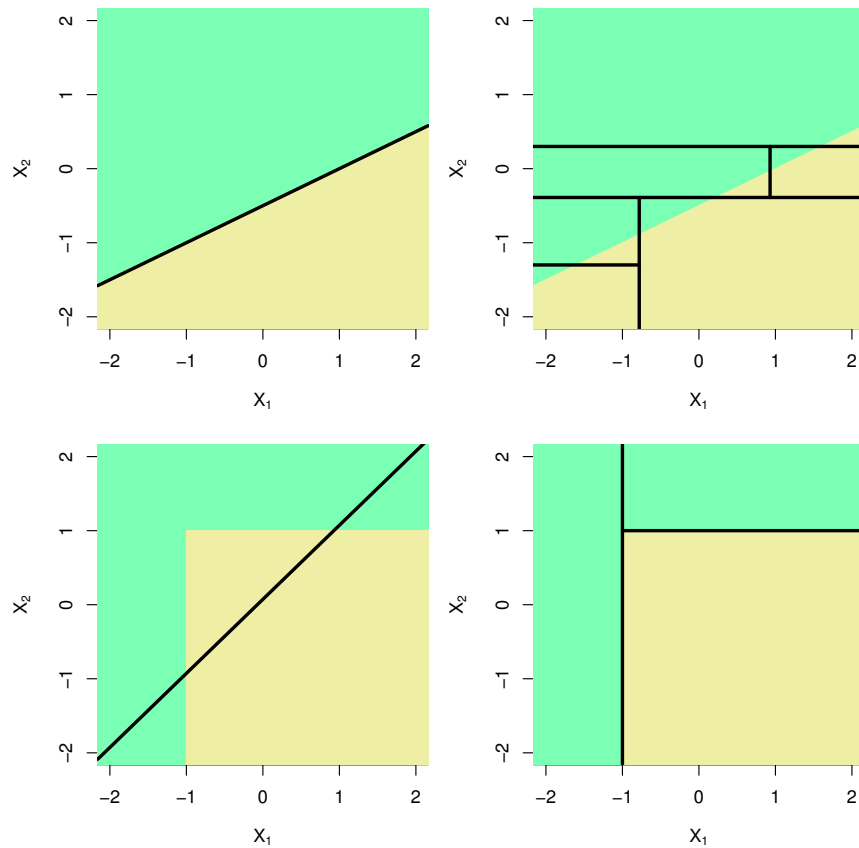
□

Exemple sur des données relatives à des problèmes cardiaques :

- Ces données contiennent des mesures d'une variable à expliquer binaire HD pour 303 patients avec des symptômes de douleurs thoraciques.
- HD = Yes (resp. HD = No) indique la présence (resp. l'absence) d'une pathologie cardiaque suite à un test angiographique.
- Il y a 13 variables potentiellement explicatives parmi lesquelles Age, Sex, Chol (une mesure de cholestérol) et d'autres variables liées au fonctionnement du coeur et des poumons.
- Le *cost complexity pruning* avec validation croisée conduit à un arbre de classification avec 6 feuilles :



Arbres de classification versus modèle linéaire :



- Première ligne de graphiques : frontière de décision linéaire.
- Deuxième ligne de graphiques : frontière de décision non linéaire.
- Graphiques de la première colonne : analyse discriminante linéaire ou similaire.
- Deuxième colonne de graphiques : arbre de classification.

4.4 Arbres de classification avec le package rpart

On considère des données relatives à des problèmes cardiaques similaires à celles présentées ci-dessus. La variable à expliquer à expliquer est AHD. Comme ci-dessus, AHD = Yes (resp. AHD = No) indique la présence (resp. l'absence) d'une pathologie cardiaque.

```
library(rpart)
library(rpart.plot)

## Chargement des données
setwd("~/Documents/Ens/MachineLearning/Data/")
```

```
h <- read.csv("Heart.csv", row.names = 1)
head(h)

## Arbre de classification (method = "class")
ar1 <- rpart(AHD ~ ., data = h, parms = list(split = "gini"), method = "class")
rpart.plot(ar1)
```

Exercice 46.

Interpréter l'arbre représenté. En vous inspirant de la section 4.2, donner du code supplémentaire permettant d'ajuster un arbre plus profond et de l'élaguer de façon "optimale" à la CART. Que constatez-vous ? Calculer son erreur de classification sur les données d'apprentissage.

4.5 Avantages et inconvénients des arbres de décision

- + Les arbres de décision sont particulièrement simples à interpréter. Les arbres de régression sont même plus simples à interpréter que les modèles de régression linéaire.
- + Certains chercheurs considèrent que les arbres de décision représentent mieux les processus de décision humains que les autres méthodes de régression et de classification vues dans les chapitres précédents.
- + Les arbres peuvent être représentés graphiquement et sont facilement interprétables (surtout s'ils sont peu profonds) même par des non experts du domaine sous-jacent.
- + Les méthodes à bases arbres peuvent gérer le cas de variables explicatives qualitatives sans qu'il soit nécessaire de procéder à du codage disjonctif complet.
- Malheureusement, les arbres de décision n'ont généralement pas la même performance prédictive que les autres méthodes de classification et de régression vues dans les chapitres précédents.
- De plus, ces méthodes d'apprentissage sont connues pour être "instables", sensibles à de légères perturbations des données d'apprentissage.

Néanmoins, en agrégeant/combinant de nombreux arbres de décision, la performance prédictive peut être grandement améliorée. Cela est l'objet des prochaines sections. Le dernier point négatif ci-dessus sera même un atout dans ce contexte.

4.6 Bagging

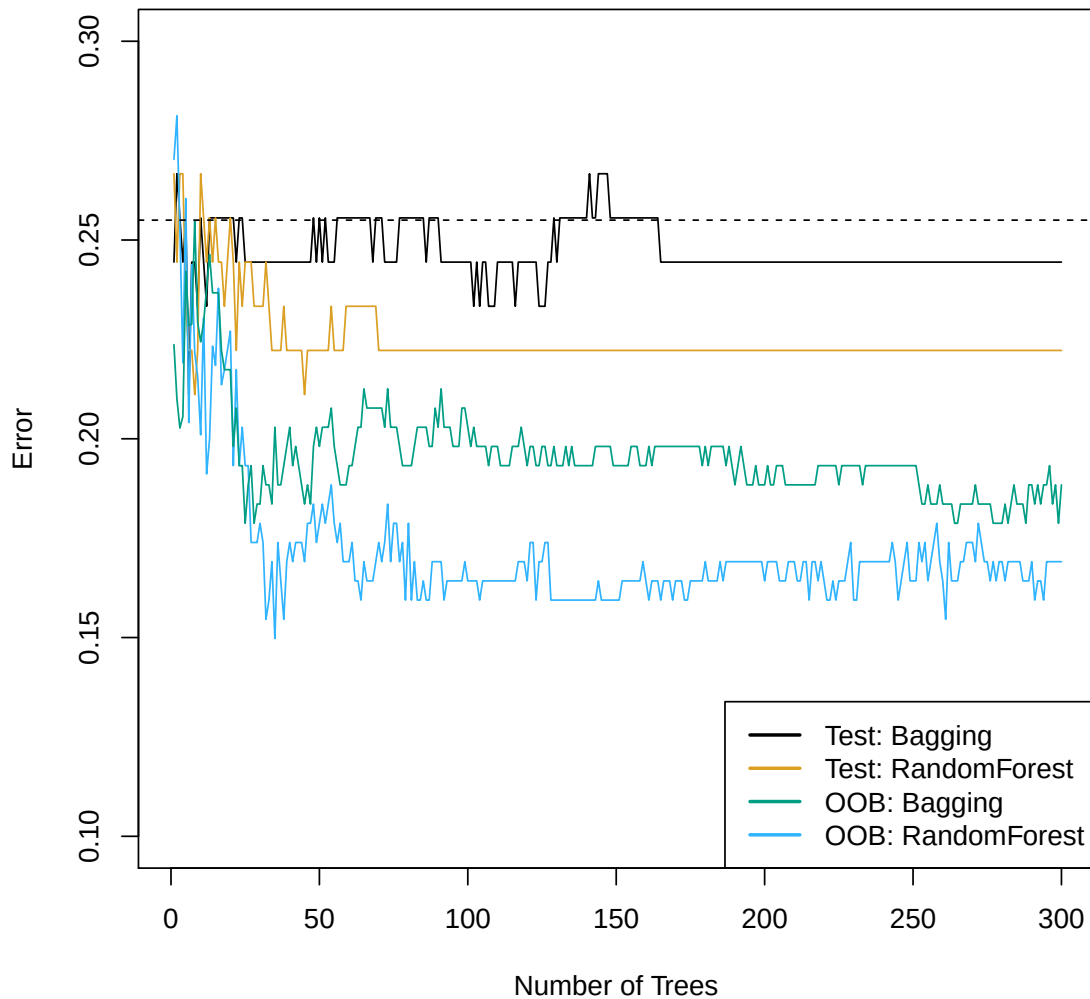
- Le *Bootstrap aggregation* ou *bagging* est une approche générale pour réduire la variance d'une méthode d'apprentissage supervisée. Elle est introduite ci-après parce qu'elle est particulièrement utile dans le contexte des arbres de décision.
- Rappelez-vous qu'étant donné un échantillon aléatoire $Z_1, \dots, Z_n$ d'une variable aléatoire Z telle que $\mathbb{V}[Z] = \sigma^2 < \infty$, la variance de $\bar{Z}_n = \frac{1}{n} \sum_{i=1}^n Z_i$ est σ^2/n .
- En conséquence, l'utilisation de la **moyenne arithmétique** peut avoir tendance à **réduire la variance**. Cela ne paraît pas forcément d'un intérêt pratique dans le cadre de l'apprentissage supervisé car, en général, on ne dispose pas de jeux de données multiples.
- Il est néanmoins possible de **ré-échantillonner** les données disponibles (*bootstrapping* en anglais).
- À partir des données disponibles d_n , on génère un échantillon bootstrap d_n^* en tirant n fois avec remise une observation (x_i, y_i) de d_n . L'échantillon d_n^* peut ainsi contenir plusieurs fois une observation de d_n et toutes les observations de d_n ne sont pas forcément dans d_n^* .
- Pour un modèle d'apprentissage à ajuster $\hat{f}_{D_n}$, au lieu d'utiliser $\hat{f}_{d_n}$ pour faire des prédictions, l'idée du bagging consiste à utiliser, dans le cas de la régression,

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_{d_n^{*,(b)}}(x),$$

où $d_n^{*,(1)}, \dots, d_n^{*,(B)}$ sont B échantillons bootstrap obtenus à partir de d_n .

- Dans le cas de classification, la moyenne est remplacée par un vote à la majorité : la classe prédite pour Y lorsque $X = x$ est celle qui est la plus fréquente parmi les prédictions $\hat{f}_{d_n^{*,(1)}}(x), \dots, \hat{f}_{d_n^{*,(B)}}(x)$.

Exemple sur les données relatives à des problèmes cardiaques :



- L'erreur de test (en noir) est donnée en fonction de B , le nombre d'échantillons bootstrap.
- La courbe noire en pointillés correspond à l'erreur de test d'un seul arbre de classification.
- La courbe verte donne l'erreur *out-of-bag* (OOB), qui est considérablement plus faible.

Estimation de l'erreur *out-of-bag* :

- Il s'avère qu'il est particulièrement simple d'estimer l'erreur de généralisation d'un modèle d'apprentissage auquel on a appliqué le principe du bagging.
- On peut montrer qu'en moyenne un modèle estimé $\hat{f}_{d_n^{*(b)}}$ utilise approximativement $2/3$ des observations se trouvant dans d_n .

- Les observations restantes (un tiers approximativement en moyenne) qui n’a pas été utilisé pour obtenir $\hat{f}_{d_n^{*,(b)}}$ sont appelées les observations *out-of-bag* (OOB).
- L’idée pour prédire la i ème observation est alors de n’utiliser que les modèles ajustés $\hat{f}_{d_n^{*,(b)}}$ pour lesquels cette observation était OOB. Cela donne environ $B/3$ prédictions pour la i ème observation, que l’on peut ensuite moyenner.
- Lorsque B est grand, il est possible de vérifier que cette estimation est approximativement l’erreur LOOCV (*leave-one out cross-validation*) pour le bagging.

Exercice 47.

Soient $T_1(x), \dots, T_B(x)$ B “modèles aléatoires” évalués en $x \in \mathbb{R}^p$ fixé et “identiquement distribués” ($T_1(x), \dots, T_B(x)$ seront ainsi vus comme B variables aléatoires identiquement distribuées). On suppose de plus que

$$\text{Cor}[T_i(x), T_j(x)] = \rho(x) \geq 0, \quad \text{pour tout } i \neq j.$$

On note

$$T(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

1. Vérifier que $\mathbb{E}[T(x)] = \mathbb{E}[T_1(x)]$ et que

$$\mathbb{V}[T(x)] = \rho(x) \mathbb{V}[T_1(x)] + \frac{1 - \rho(x)}{B} \mathbb{V}[T_1(x)].$$

2. Cela permet-il de garantir que, nécessairement, le “bagging réduit la variance” ? Pour différentes valeurs de $\rho(x)$, représenter la réduction de variance en fonction de B .

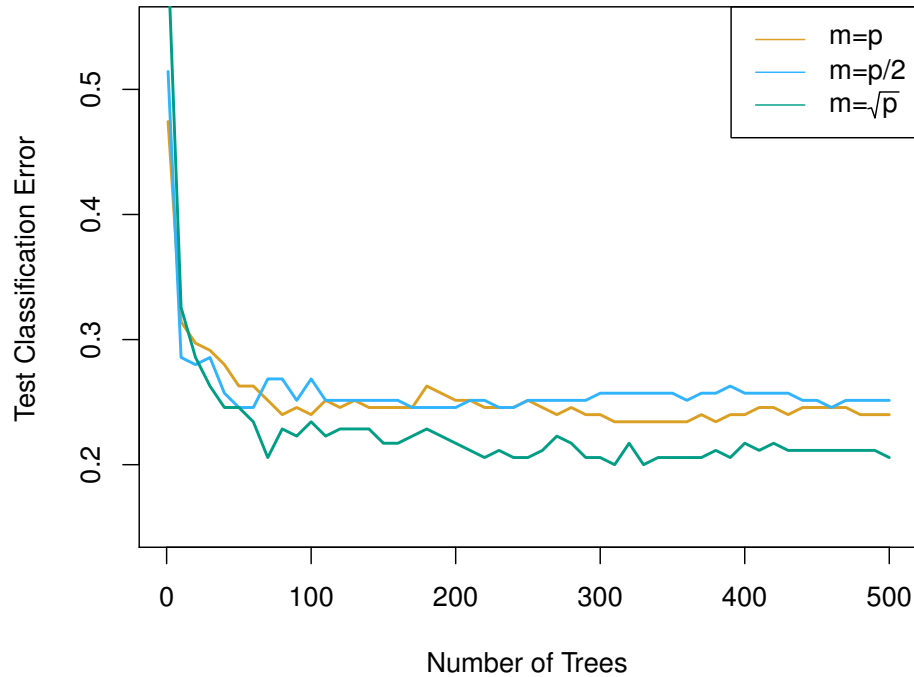
4.7 Forêts aléatoires

- Les *forêts aléatoires* (*random forests* en anglais) peuvent être vues comme une amélioration du bagging pour les arbres reposant sur un petit ajustement qui décorrèle les arbres à agréger et réduit ainsi la variance du modèle final.
- Comme pour le bagging, B arbres de décision sont estimés sur B échantillons bootstrap obtenus à partir de d_n .
- En revanche, lors de la construction de ces arbres, à chaque fois qu’il faut réaliser une division/choisir un test, la division est réalisée en choisissant la variable explicative pour la division dans un sous-ensemble de $m \leq p$ variables explicatives candidates choisies aléatoirement.

- Le sous-ensemble de $m \leq p$ variables explicatives candidates change avec chaque division à réaliser et, typiquement, $m = \sqrt{p}$.
- Par exemple, pour les données relatives à des problèmes cardiaques, chaque division ne peut porter que sur 4 des variables potentiellement explicatives.

Exemple sur des données relatives à l'expression de gènes :

- Les forêts aléatoires ont été appliquées à un jeu de données biologique de grande dimension contenant 4718 variables relatives à l'expression de gènes mesurées sur de prélèvements faits sur 349 patients.
- Il y a environ 20 000 gènes chez les humains, et les gènes d'un individu ont différents niveaux d'activité ou d'expression en fonction du type de la cellule, etc, et des conditions biologiques.
- Chacun des prélèvements est affecté à une classe : prélèvement normal ou 14 différents types de cancers.
- Les forêts aléatoires ont été utilisées pour prédire le type de cancer en se fondant sur les 500 variables d'expression de gènes qui ont la plus grande variance dans l'ensemble d'apprentissage.
- Les observations ont été aléatoirement divisées en ensemble d'apprentissage et ensemble de test. Les forêts aléatoires ont été appliquées avec trois valeurs différentes du paramètre m (taille du sous-ensemble de variables explicatives pour une division).
- La figure ci-après donne les résultats ($p = 500$, $K = 15$) :



- L'erreur de test est représenté en fonction du nombre d'arbres. Chaque couleur correspond à une valeur différente de m .
- Les forêts aléatoires ($m < p$) semblent ainsi conduire à une légère amélioration par rapport au bagging ($m = p$). L'erreur de classification d'un seul arbre est de 45.7%.

4.8 Boosting

- Comme le bagging, le *boosting* est une approche générale qui peut être appliquée à de nombreux modèles d'apprentissage statistique pour la régression ou la classification. Cette technique ne sera néanmoins présentée que dans le cadre des arbres de décision.
- Rappelez-vous que le bagging consiste à créer des échantillons bootstrap des données disponibles, à ajuster un arbre à chaque échantillon bootstrap et à combiner leurs résultats pour créer un seul modèle prédictif. Chaque arbre est en particulier construit à partir d'un échantillon bootstrap indépendamment des autres arbres.
- Le boosting n'utilise pas le bootstrap mais construit des arbres séquentiellement : chaque arbre est construit en utilisant des informations provenant des arbres précédemment construits : le premier arbre est construit sur les données initiales, le second arbre est construit de façon à prédire les résidus de l'arbre précédent, etc.

Algorithme du boosting pour des arbres de régression :

1. Mettre $\hat{f} = 0$ et $r_i = y_i$ pour tout $i \in \{1, \dots, n\}$.
2. Pour $b = 1, \dots, B$, répéter :
 - (a) Ajuster un arbre $\hat{f}_{(b)}$ avec d divisions (et donc $d + 1$ feuilles) à l'ensemble d'apprentissage $\{(x_i, r_i)\}_{i \in \{1, \dots, n\}}$.
 - (b) Ajuster $\hat{f}$ en lui rajoutant une version “réduite” de $\hat{f}_{(b)}$:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}_{(b)}(x).$$

- (c) Mettre les résidus à jour :

$$r_i \leftarrow r_i - \lambda \hat{f}_{(b)}(x_i).$$

3. Renvoyer le modèle boosté :

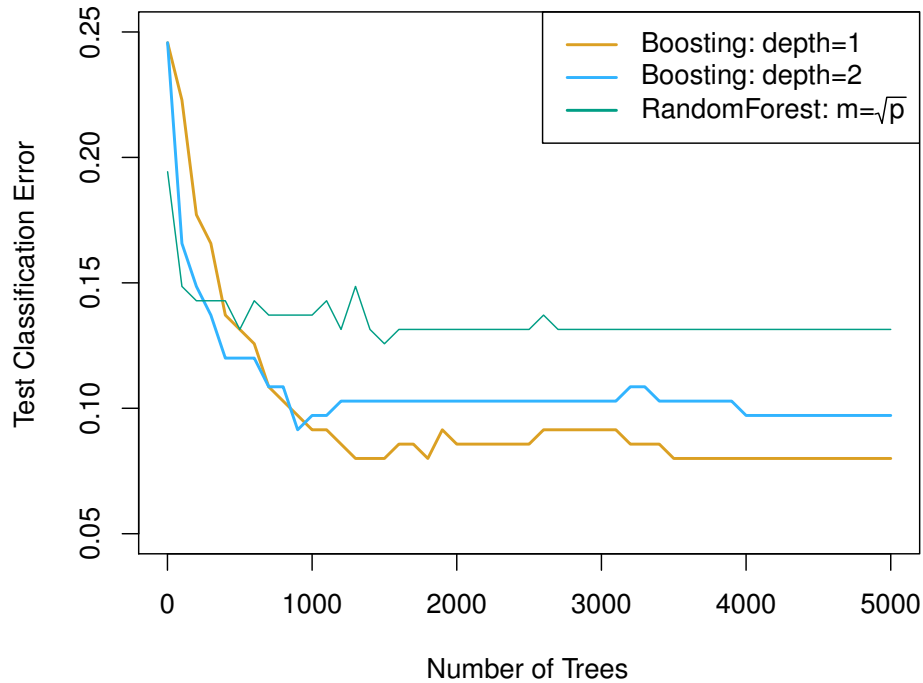
$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}_{(b)}(x).$$

Intuitions derrière le boosting :

- Contrairement à l'ajustement d'un seul arbre de décision (utilisation de toutes les données et potentiellement sur-apprentissage), le boosting conduit à un apprentissage “lent”.
- Étant donné le modèle courant, un nouvel arbre est ajusté aux résidus du modèle. Le nouvel arbre est ensuite “rajouté” au modèle “final” et les résidus sont mis à jour.
- Les arbres sont généralement plutôt peu profonds, avec juste quelques feuilles, ce qui est déterminé par le paramètre d de l'algorithme.
- En ajustant de petits arbres aux résidus, le modèle “final” $\hat{f}$ est lentement amélioré pour les x pour lesquels il n'est pas précis. Le paramètre/coefficient λ ralentit le processus d'amélioration permettant à un grand nombre d'arbres différents de potentiellement “réduire” les résidus.
- Le boosting pour des arbres de classification est similaire mais plus complexe. Il ne sera pas présenté dans ce cours.
- Le package `gbm` (*gradient boosted models*) par exemple implémente le boosting pour une variété de modèles de classification et de régression.

Retour sur les données relatives à l'expression de gènes :

- Résultats du boosting pour le problème ramené de $K = 15$ à deux classes (`cancer` versus `normal`) :



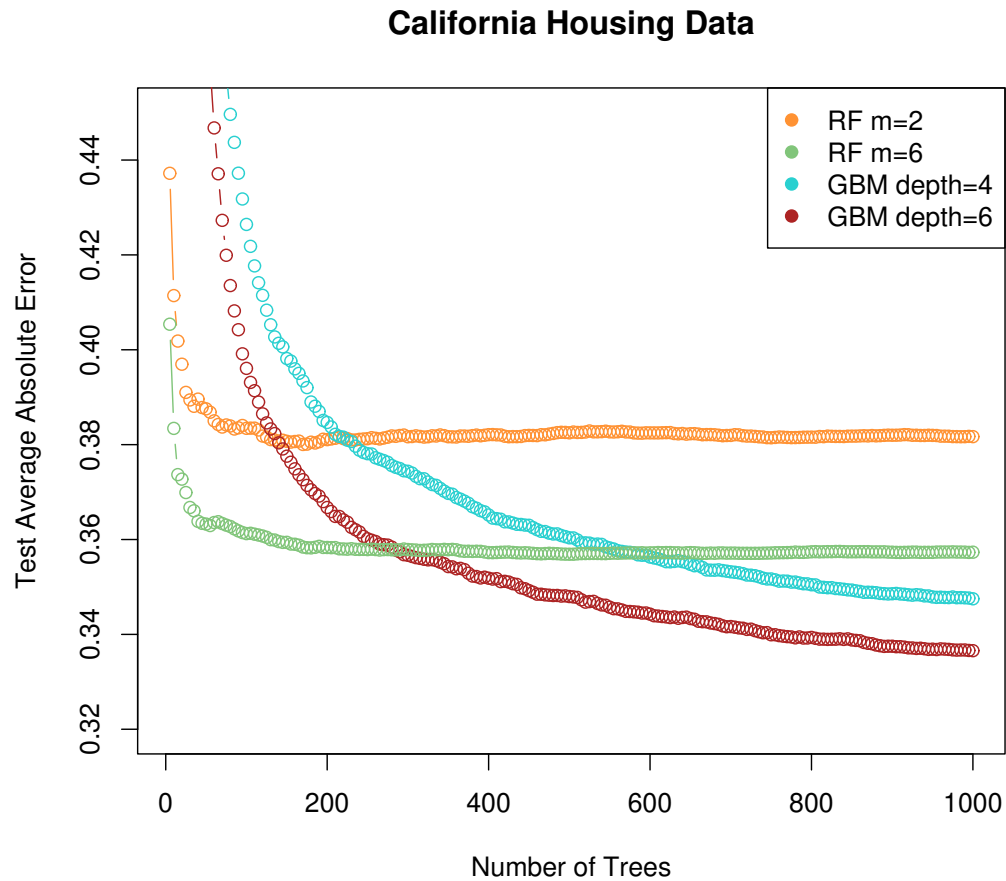
- L’erreur de test est représentée en fonction du nombre d’arbres. Pour les deux modèles boostés, $\lambda = 0.01$. L’utilisation d’arbres de profondeur 1 ($d = 1$) conduit à une légère amélioration par rapport à l’utilisation d’arbres de profondeur 2 ($d = 2$).
- Les deux modèles boostés sont légèrement meilleurs que les forêts aléatoires même si l’erreur standard de l’erreur de classification est autour de 2% (rendant les différences non significatives au final).
- L’erreur de classification d’un seul arbre est de 24%.

Calibrage des paramètres du boosting :

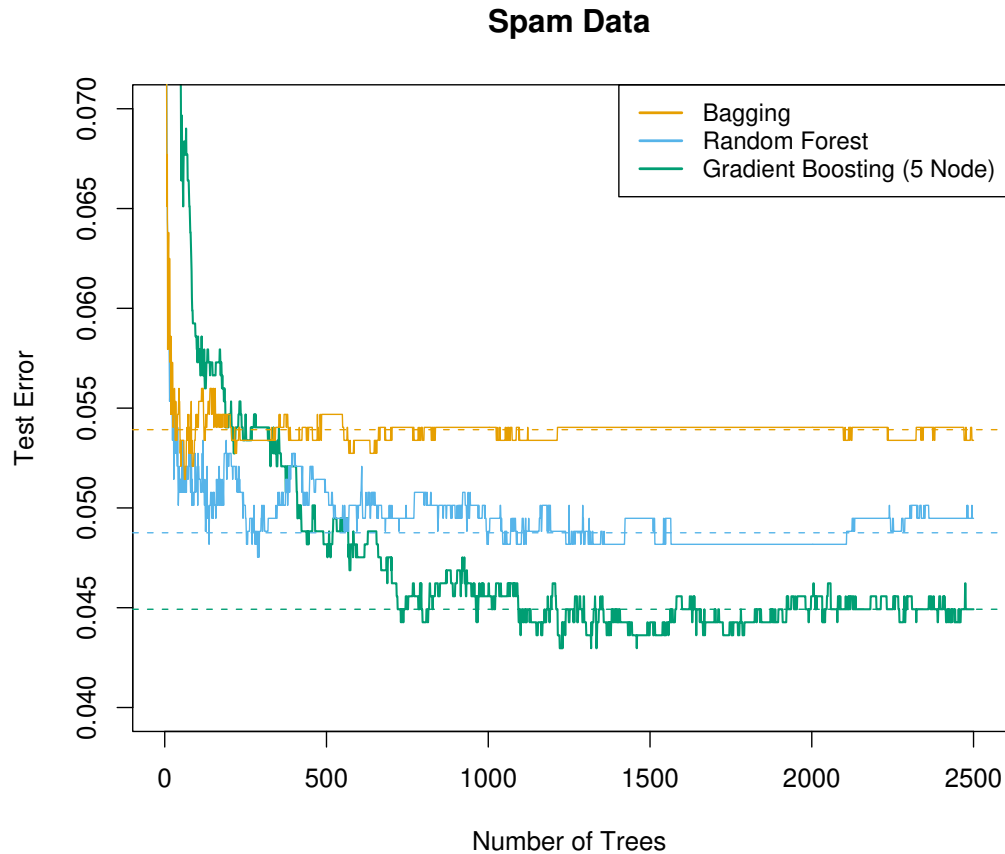
1. Le nombre d’arbres B : Contrairement au bagging et aux forêts aléatoires, le boosting peut conduire à du sur-apprentissage si B est trop grand (même si cela n’arriverait typiquement que très lentement). La valeur de B est typiquement sélectionnée par validation croisée.
2. La paramètre de *shrinkage* λ : Il contrôle la vitesse d’apprentissage du boosting. Des valeurs typiques sont 0.01 ou 0.001, et “une bonne valeur” peut dépendre du problème. Une très petite valeur de λ peut nécessiter une très grande valeur de B pour atteindre une bonne performance.
3. Le nombre de divisions d de chaque arbre : Ce paramètre contrôle la complexité du modèle boosté. Souvent $d = 1$ donne de bons résultats. Dans ce cas, le modèle boosté

s'avère être un modèle additif. Plus généralement, d peut être vu comme contrôlant le degré d'interaction des variables explicatives dans le modèle boosté.

Exemple sur des données relatives au logement en Californie :



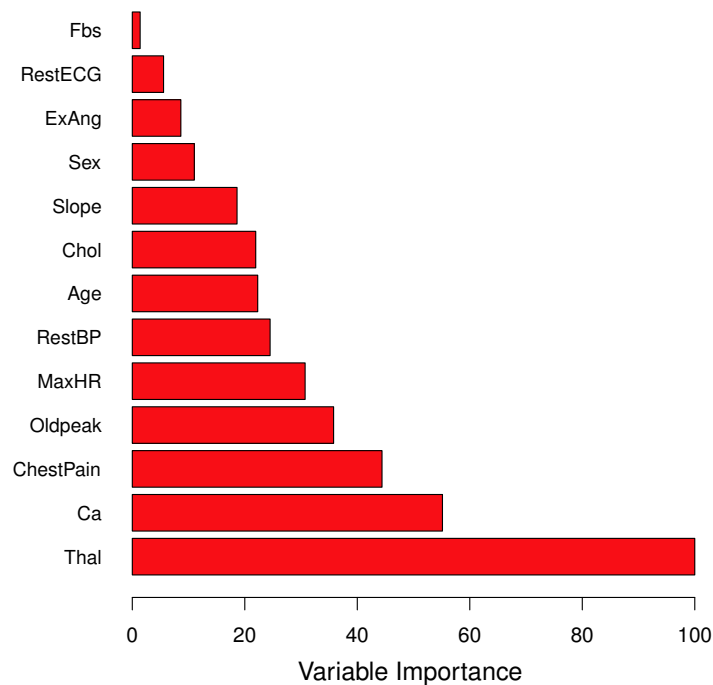
Exemple sur des données relatives au spam :



Mesure de l'importance des variables potentiellement explicatives :

1. Dans le cas de la régression, pour le bagging et les forêts aléatoires, afin de mesurer l'importance d'une variable potentiellement explicative, l'idée consiste à mesurer à quel point les moindres carrés diminuent suite à des divisions concernant cette variable (en moyenne sur les B arbres). Une grande valeur indique que la variable est importante.
2. De façon similaire, dans le cas de la classification, afin de mesurer l'importance d'une variable potentiellement explicative, l'idée est de mesurer le total de la décroissance de l'indice de Gini suite à des divisions concernant cette variable (en moyenne sur les B arbres).

Exemple sur les données relatives à des problèmes cardiaques :



4.9 Bagging, forêts aléatoires et boosting avec tidymodels

```
library(tidyverse)
library(tidymodels)
library(xgboost)

## Bagging #####

bagging_mod <- rand_forest(mtry = .cols()) |>
  set_engine("randomForest", importance = TRUE) |>
  set_mode("regression")

setwd("~/Documents/Ens/MachineLearning/Data/")
Boston <- read_csv("Boston.csv")
set.seed(1234)
Boston_split <- initial_split(Boston)

Boston_train <- training(Boston_split)
Boston_test <- testing(Boston_split)

bagging_fit <- fit(bagging_mod, medv ~ ., data = Boston_train)
```

```

augment(bagging_fit, new_data = Boston_test) |> rmse(truth = medv, estimate = .pred)

augment(bagging_fit, new_data = Boston_test) |>
  ggplot(aes(medv, .pred)) +
  geom_abline() +
  geom_point(alpha = 0.5)

## Forêts aléatoires #####

rf_mod <- rand_forest(mtry = 6)|>
  set_engine("randomForest", importance = TRUE) |>
  set_mode("regression")

rf_fit <- fit(rf_mod, medv ~ ., data = Boston_train)

augment(rf_fit, new_data = Boston_test) |> rmse(truth = medv, estimate = .pred)

augment(rf_fit, new_data = Boston_test) |>
  ggplot(aes(medv, .pred)) +
  geom_abline() +
  geom_point(alpha = 0.5)

## Boosting #####

boost_mod <- boost_tree(trees = 5000, tree_depth = 4) |>
  set_engine("xgboost") |>
  set_mode("regression")

boost_fit <- fit(boost_mod, medv ~ ., data = Boston_train)

augment(boost_fit, new_data = Boston_test) |>
  rmse(truth = medv, estimate = .pred)

augment(boost_fit, new_data = Boston_test) |>
  ggplot(aes(medv, .pred)) +
  geom_abline() +
  geom_point(alpha = 0.5)

```

4.10 Résumé

- Les arbres de décision sont des modèles de régression et de classification simples et interprétables.
- Ils ne sont néanmoins généralement pas compétitifs en terme de performance prédictive

en comparaison à d'autres modèles.

- Le bagging, les forêts aléatoires et le boosting sont de bonnes approches permettant d'améliorer la performance prédictive des arbres. Ces méthodes consistent à ajuster un grand nombre d'arbres et à combiner les prédictions des arbres estimés.
- Les forêts aléatoires et le boosting font partie de l'état de l'art en apprentissage supervisé. Leur résultats peuvent néanmoins être difficiles à interpréter.